PyCuDAL

VALIDATION PROTOCOL

PyCuDAL — Content Uniformity and Dissolution Acceptance Limits (Python Implementation)

Approval

Role	Name (printed)	Signature	Date
Performed By	Moaz El-Essawey		
Reviewed By			
Approved By			

TABLE OF CONTENTS

Appendices

Forms

1.0 PURPOSE

This Validation Protocol defines the approach, responsibilities, and acceptance criteria for validating the PyCuDAL (Content Uniformity and Dissolution Acceptance Limits) Python application. The purpose of this validation is to provide documented evidence that the Program consistently produces accurate, reliable, and reproducible results when used as intended, in accordance with applicable USP <905> Content Uniformity and USP <711> Dissolution requirements.

2.0 SCOPE

This protocol applies to the PyCuDAL Python application, including the Content Uniformity (Sampling Plan 1 and Sampling Plan 2) and Dissolution (Sampling Plan 1 and Sampling Plan 2) acceptance-limit calculation modules described in Appendix F.

Validation activities under this protocol cover: installation and startup verification, primary window/function navigation, review of the program's calculation strategy against its Python code implementation, and agreement between program output and predefined test data sets.

This protocol does not cover the underlying USP <905>/<711> statistical methodology itself, network or IT infrastructure qualification, or end-user training, which are addressed by separate documentation.

3.0 REFERENCES

- USP General Chapter <905> — Uniformity of Dosage Units
- USP General Chapter <711> — Dissolution
- CUSP2 Acceptance-Limit Discrepancy Investigation (Technical Note)
- Program source code listing (Appendix A)

4.0 DEFINITIONS AND ABBREVIATIONS

Term	Definition
PyCuDAL	Python – Content Uniformity and Dissolution Acceptance Limits (the Program)
USP	United States Pharmacopeia
CU	Content Uniformity (USP <905>)
AV	Acceptance Value, as defined in USP <905>
SE	Within-location standard deviation (CUSP2 / DISP2)
SM	Between-location standard deviation (CUSP2 / DISP2)
MEANL / MEANU	Lower / upper acceptable bound on the sample mean
LBOUND	Required lower bound on the probability of passing the USP test
CI	Confidence level / confidence interval

5.0 RESPONSIBILITIES

Role	Responsibility
Validation Lead / Author	Prepares this protocol, coordinates execution, and compiles the validation summary report.
Performer / Tester	Executes the test forms in this protocol and records results, evidence, and any deviations.
Reviewer	Reviews completed test forms and supporting documentation for completeness, accuracy, and compliance.
Approver	Approves this protocol prior to execution and approves the final validation summary upon completion.
System Owner / SME	Provides subject-matter support on program functionality and resolves technical deviations.

6.0 SYSTEM DESCRIPTION

PyCuDAL is a Python application that calculates parametric (tolerance-interval) acceptance limits for USP <905> Content Uniformity and USP <711> Dissolution testing. It is a translation of a legacy SAS/AF application of the same name. A complete description of the Program's modules, functions, and intended use is provided in Appendix F.

7.0 VALIDATION APPROACH

Validation will be performed using a combination of: installation and startup verification (Form 1); navigation checks across the Program's primary windows/functions (Form 2); review of the Program's calculation strategy against its Python code implementation (Form 3); and comparison of Program output against the predefined test data sets in Appendix G (Form 4). Any discrepancy identified during execution will be documented and evaluated per Section 9 using Form 5.

8.0 ACCEPTANCE CRITERIA

Validation is considered successful when all of the following are met:

- The Program loads and runs without error (Form 1).
- All primary windows/functions are accessible and navigate as intended (Form 2).
- The Program's calculation strategy is confirmed to be correctly reflected in the Python code implementation (Form 3).
- Program output agrees with the predefined test data sets in Appendix G, within the acceptance tolerance defined for each test case (Form 4).
- All test forms are completed, reviewed, and approved, and any deviations are closed prior to final approval of this protocol.

9.0 DEVIATION HANDLING

Any unexpected result, discrepancy, or issue identified during execution of this protocol shall be documented on Form 5 (Problem/Request Report). Each deviation shall be assessed for impact on validation status, assigned a severity, and tracked to resolution. Deviations must be closed and approved prior to final protocol closure.

10.0 DOCUMENTATION AND RETENTION

All completed forms, supporting data, and this protocol shall be retained. The completed protocol package, together with its appendices and forms, constitutes the validation record for the Program.

APPENDIX A — PROGRAM LISTING

pycudal/cudal/core.py

```
"""
cudal.core
==========

Shared statistical building blocks for the CuDAL (Content Uniformity and
Dissolution Acceptance Limits) system, translated from the original SAS
macros (CuDAL.sas, Cusp1.sas, Cusp2.sas, Disp1.sas, Disp2.sas).

SAS -> Python function mapping
------------------------------
    PROBNORM(x)        -> norm.cdf(x)
    PROBIT(p)          -> norm.ppf(p)
    PROBCHI(x, df)     -> chi2.cdf(x, df)
    CINV(p, df)        -> chi2.ppf(p, df)

Two probability "cores" appear repeatedly in the SAS code, always with the
exact same formulas (only the values fed into them differ):

  * ``c1calc`` (cusp1.sas) / ``cullu`` + ``cuulu`` (Cusp2.sas)
    -> :func:`content_uniformity_bound` here.
  * ``COMPUTE`` (Disp1.sas / Disp2.sas -- byte-for-byte identical formula
    in both files)
    -> :func:`dissolution_bound` here.

Both are implemented as **numpy ufunc-style, broadcastable** functions:
``mu``/``sigma`` (or ``llu``/``sigma``) may be scalars, or arrays of any
mutually-broadcastable shape, and the result has the broadcast shape.
This lets every acceptance-limit search evaluate an entire grid (e.g. all
300 rows of a MEAN table, or an 80x80 SE/SM grid) in a handful of numpy
calls instead of one Python-level call per grid point.

Performance notes
-----------------
This module deliberately avoids two patterns that make the equivalent
SAS code (and a naive line-by-line port of it) slow:

1. **Per-point root finding.** The SAS code steps a candidate value
   (a standard deviation, or a mean) in tiny fixed increments, one
   value at a time, until a probability threshold is crossed. Doing
   this with a Python ``for`` loop -- even calling a fast vectorized
   probability function inside it -- pays the interpreter-loop
   overhead once per grid point. :func:`batched_root_find` instead
   solves *all* grid points' roots simultaneously with a vectorized
   bisection: every iteration is a single numpy expression over the
   whole grid, not a Python loop over the grid.

2. **Scalar broadcasting.** :func:`content_uniformity_bound` and
   :func:`dissolution_bound` accept arrays directly, so a table with
   ``M`` mean values, or a grid with ``G = len(SE) * len(SM)`` cells,
   is evaluated as one ``M``- or ``G``-length numpy computation rather
   than ``M`` (or ``G``) separate scalar calls.

Together, building e.g. the CUSP2 acceptance-limit table for an 80x80
SE/SM grid drops from tens of thousands of scalar probability
evaluations to a few dozen vectorized ones.
"""

```

```
from __future__ import annotations

from typing import Callable, Tuple

import numpy as np
from scipy.special import gammainc, gammaincinv, ndtr, ndtri

__all__ = [
    "probnorm",
    "probit",
    "probchi",
    "cinv",
    "sas_do_range",
    "content_uniformity_bound",
    "dissolution_bound",
    "batched_root_find",
    "batched_two_sided_bounds",
]


# ---------------------------------------------------------------------------
# Direct SAS function equivalents (already numpy ufuncs -> broadcast for free)
# ---------------------------------------------------------------------------


def probnorm(x):
    """
    SAS PROBNORM(x): standard normal CDF.

    Implemented directly via ``scipy.special.ndtr`` rather than
    ``scipy.stats.norm.cdf`` -- numerically identical, but avoids the
    ~5-7x overhead of the generic frozen-distribution machinery, which
    matters a lot here since this is called on arrays with millions of
    elements during the acceptance-limit table searches.
    """
    return ndtr(x)


def probit(p):
    """SAS PROBIT(p): standard normal inverse CDF (quantile function)."""
    return ndtri(p)


def probchi(x, df):
    """
    SAS PROBCHI(x, df): chi-square CDF with `df` degrees of freedom.

    Implemented via the regularized lower incomplete gamma function
    (``chi2.cdf(x, df) == gammainc(df/2, x/2)``) rather than
    ``scipy.stats.chi2.cdf`` -- numerically identical, ~2.5x faster.
    """
    return gammainc(df / 2.0, x / 2.0)


def cinv(p, df):
    """
    SAS CINV(p, df): chi-square inverse CDF (quantile function).

    ``chi2.ppf(p, df) == 2 * gammaincinv(df/2, p)``.
    """
    return 2.0 * gammaincinv(df / 2.0, p)


def sas_do_range(start: float, stop: float, step: float) -> np.ndarray:
```

```
    """
    Reproduce the index values visited by a SAS ``DO start TO stop BY step;``
    loop (inclusive of `stop`, subject to floating point rounding the same
    way SAS handles it in practice for these fixed step sizes).
    """
    if step == 0:
        raise ValueError("step must be non-zero")
    n = int(round((stop - start) / step)) + 1
    if n <= 0:
        return np.array([])
    return start + step * np.arange(n)


# ---------------------------------------------------------------------------
# Content uniformity core  (c1calc / cullu / cuulu in the SAS source)
# ---------------------------------------------------------------------------


def _cu_stage_prob(mu: np.ndarray, sigma: np.ndarray, E, n: int, k: float) -
> np.ndarray:
    """
    One "stage" of the content-uniformity probability calculation
    (n=10, k=2.4 for stage 1; n=30, k=2.0 for stage 2), reproducing the
    int1/int2/int3 (or iint1/iint2/iint3) block of the SAS macro exactly.

    ``mu``, ``sigma`` and ``E`` may be scalars or arrays of a common
    broadcast shape; the result has that broadcast shape.
    """
    h = 0.05
    L1 = 15.0

    mu = np.asarray(mu, dtype=float)
    sigma = np.asarray(sigma, dtype=float)
    E = np.asarray(E, dtype=float)

    z1 = (E - mu) * np.sqrt(n) / sigma
    z2 = (98.5 - mu) * np.sqrt(n) / sigma
    chi_a = probchi((n - 1) * L1**2 / (k * sigma) ** 2, n - 1)
    int1 = (probnorm(z1) - probnorm(z2)) * chi_a

    # Broadcast the fixed integration grid against mu/sigma/E by adding a
    # trailing axis to the (broadcast) parameter arrays and summing over it.
    mu_e, sigma_e, E_e = np.broadcast_arrays(mu, sigma, E)
    mu_e = mu_e[..., None]
    sigma_e = sigma_e[..., None]
    E_e = E_e[..., None]

    # int2 : do x = E to (E + 15 - h) by h;   (E varies per element, so we
    # build the grid relative to E_e directly rather than via sas_do_range)
    n_steps = int(round(15 / h))  # number of h-steps spanning the 15-unit window
    j = np.arange(n_steps)  # 0 .. n_steps-1
    xs = E_e + j * h
    x1 = (xs - mu_e) * np.sqrt(n) / sigma_e
    x2 = (xs + h - mu_e) * np.sqrt(n) / sigma_e
    chi_args = (n - 1) * (E_e + 15 - xs - h / 2) ** 2 / (k * sigma_e) ** 2
    int2 = np.sum((probnorm(x2) - probnorm(x1)) * probchi(chi_args, n - 1), axis=
-1)

    # int3 : do x = (98.5 - 15) to (98.5 - h) by h;   (fixed grid, no E dependenc
e)
    xs3 = sas_do_range(98.5 - 15, 98.5 - h, h)
    x1b = (xs3 - mu_e) * np.sqrt(n) / sigma_e
    x2b = (xs3 + h - mu_e) * np.sqrt(n) / sigma_e
    chi_args3 = (n - 1) * (15 - 98.5 + xs3 + h / 2) ** 2 / (k * sigma_e) ** 2
```

```
183     int3 = np.sum((probnorm(x2b) - probnorm(x1b)) * probchi(chi_args3, n - 1), ax
is=-1)
184
185     return int1 + int2 + int3
186
187
188def content_uniformity_bound(mu, sigma, target):
189     """
190     Replicates the ``c1calc`` / ``cullu`` / ``cuulu`` SAS macro.
191
192     Estimated probability that a batch with true (population) mean ``mu``
193     and true standard deviation ``sigma`` passes the USP <905> Content
194     Uniformity test, for a label claim / target of ``target`` (%).
195
196     Fully broadcastable: ``mu``, ``sigma``, ``target`` may be Python
197     scalars or numpy arrays of a common broadcast shape (e.g. evaluate an
198     entire table of candidate means in one call). The return value has
199     that broadcast shape (a plain float is returned for scalar inputs).
200
201     Parameters
202     ----------
203     mu : float or array_like
204         True population mean (% of label claim), e.g. LLU or ULU.
205     sigma : float or array_like
206         True population standard deviation (% of label claim).
207     target : float or array_like
208         Target / label claim value used by the USP test (usually 100).
209
210     Returns
211     -------
212     float or numpy.ndarray
213         The "OVERBD" value: max(P1, P2), where P1 is the stage-1 (n=10)
214         pass probability and P2 combines stage-2 (n=30) with an
215         additional 30-unit compound tail term.
216     """
217     mu = np.asarray(mu, dtype=float)
218     sigma = np.asarray(sigma, dtype=float)
219     target = np.asarray(target, dtype=float)
220     E = np.where(target <= 101.5, 101.5, target)
221
222     # ---- stage 1 (n=10, k=2.4) -> P1 ------------------------------------
223     P1 = _cu_stage_prob(mu, sigma, E, n=10, k=2.4)
224
225     # ---- stage 2 (n=30, k=2.0) -> P2a + tail term P2b -------------------
226     P2a = _cu_stage_prob(mu, sigma, E, n=30, k=2.0)
227
228     zzz1 = (123.125 - mu) / sigma
229     zzz2 = np.where(target <= 101.5, (101.5 - 24.625 - mu) / sigma, (target - 24.
625 - mu) / sigma)
230     P2b = (probnorm(zzz1) - probnorm(zzz2)) ** 30
231     P2 = np.maximum(0.0, P2a + P2b - 1)
232
233     result = np.maximum(P1, P2)
234     return result if result.ndim else result.item()
235
236
237# ---------------------------------------------------------------------------
238# Dissolution core (COMPUTE macro in Disp1.sas / Disp2.sas -- identical)
239# ---------------------------------------------------------------------------
240
241
242def dissolution_bound(llu, sigma):
243     """
244     Replicates the ``COMPUTE`` SAS macro (identical in Disp1.sas and
```

```
    Disp2.sas).

    Estimated probability of passing the USP <711> Dissolution test
    (stage 1: n=6, stage 2: n=12, stage 3: n=24), given a lower adjusted
    bound on the population mean (``llu``) and population standard
    deviation ``sigma``. Fully broadcastable, like
    :func:`content_uniformity_bound`.

    Returns
    -------
    float or numpy.ndarray
        The "OVERBD" value: max(F1, F2, F3) across the three USP <711>
        stages.
    """
    llu = np.asarray(llu, dtype=float)
    sigma = np.asarray(sigma, dtype=float)

    F1 = (1 - probnorm((5 - llu) / sigma)) ** 6

    sn2 = np.sqrt(12)
    pm2 = probnorm(sn2 * (-llu) / sigma)
    pb2 = 1 - probnorm((-15 - llu) / sigma)
    F2 = pb2**12 - pm2

    sn3 = np.sqrt(24)
    pm3 = probnorm(sn3 * (-llu) / sigma)
    p2 = probnorm((-15 - llu) / sigma) - probnorm((-25 - llu) / sigma)
    p3 = 1 - probnorm((-15 - llu) / sigma)
    F3 = p3**24 + 24 * p2 * p3**23 + 276 * p2**2 * p3**22 - pm3

    result = np.maximum(np.maximum(F1, F2), F3)
    return result if result.ndim else result.item()


# ---------------------------------------------------------------------------
# Vectorized batch root finding -- replaces per-point brentq/GOTO search
# ---------------------------------------------------------------------------


def batched_root_find(
    func: Callable[[np.ndarray], np.ndarray],
    lo,
    hi,
    scan_points: int = 12,
    bisect_iters: int = 22,
    which: str = "first",
) -> Tuple[np.ndarray, np.ndarray]:
    """
    Solve ``func(x) == 0`` independently for every element of a grid, in
    one vectorized pass, instead of one root-find per grid point.

    ``func`` must accept an array of shape ``(S, 1)`` (or any shape that
    broadcasts against the grid's own parameter arrays baked into the
    closure) and return an array of shape ``(S, G)``, and must also
    accept a shape-``(G,)`` array (during bisection refinement) and
    return shape ``(G,)``. This is automatic for anything built from
    :func:`content_uniformity_bound` / :func:`dissolution_bound` plus
    elementwise arithmetic -- see cudal.cusp1/cusp2/disp1/disp2 for the
    closures passed in.

    Parameters
    ----------
    func : callable
        Elementwise function; ``func(x)`` broadcasts ``x`` against the
```

```
        grid parameters captured in the closure.
    lo, hi : float
        Scalar bounds of the search interval (the same interval is
        scanned for every grid point -- pass a wide-enough range).
    scan_points : int
        Number of points used to bracket the root before refining. The
        function is assumed to have exactly one sign crossing of
        interest between ``lo`` and ``hi`` at this resolution (this
        mirrors the same monotonicity assumption the original SAS
        stepped search relies on).
    bisect_iters : int
        Number of vectorized bisection halving steps used to refine each
        bracket. The SAS code this replaces used a fixed step of
        0.001-0.2 depending on macro; matching that resolution needs on
        the order of log2((hi-lo)/step) ~= 13-20 iterations, so the
        default of 22 already has comfortable headroom while staying
        cheap (each extra iteration re-evaluates the full probability
        integral for the whole grid).
    which : {"first", "last"}
        If the scanned curve has more than one sign crossing (e.g. the
        content-uniformity bound is bump-shaped in the mean), "first"
        finds the lowest-x crossing and "last" finds the highest-x one.
        This mirrors the SAS code's two opposite-direction searches
        (scan up from the low end vs. scan down from the high end).

    Returns
    -------
    (root, found) : (numpy.ndarray, numpy.ndarray[bool])
        ``root`` has shape ``(G,)`` -- the last scanned/bisected value is
        returned even where no sign change was found (caller decides how
        to interpret that; ``found`` flags which entries actually
        bracketed a root).
    """
    xs = np.linspace(lo, hi, scan_points)  # shape (S,)
    xs_col = xs[:, None]  # (S, 1) -- broadcasts against the grid inside func
    vals = np.asarray(func(xs_col))  # shape (S, G)

    sign = np.sign(vals)
    changes = np.diff(sign, axis=0) != 0  # (S-1, G)
    found = changes.any(axis=0)

    if which == "first":
        idx = np.argmax(changes, axis=0)
    else:
        idx = changes.shape[0] - 1 - np.argmax(changes[::-1, :], axis=0)

    g = vals.shape[-1]
    cols = np.arange(g)
    lo_x = np.where(found, xs[idx], lo)
    hi_x = np.where(found, xs[idx + 1], hi)
    lo_val = np.where(found, vals[idx, cols], vals[0, cols])

    # Vectorized bisection refinement of every bracket simultaneously.
    cur_lo, cur_hi, cur_lo_val = lo_x.copy(), hi_x.copy(), lo_val.copy()
    for _ in range(bisect_iters):
        mid = (cur_lo + cur_hi) / 2.0
        f_mid = np.asarray(func(mid))
        same_sign_as_lo = np.sign(f_mid) == np.sign(cur_lo_val)
        cur_lo = np.where(same_sign_as_lo, mid, cur_lo)
        cur_hi = np.where(same_sign_as_lo, cur_hi, mid)
        cur_lo_val = np.where(same_sign_as_lo, f_mid, cur_lo_val)

    root = (cur_lo + cur_hi) / 2.0
    return root, found
```

```python


def batched_two_sided_bounds(
    func_lower: Callable[[np.ndarray], np.ndarray],
    func_upper: Callable[[np.ndarray], np.ndarray],
    lo,
    hi,
    scan_points: int = 24,
    bisect_iters: int = 22,
):
    """
    Convenience wrapper for the CUSP2-style pattern of two searches over
    the same grid: the lower boundary found scanning low->high (first
    crossing) and the upper boundary found scanning high->low (last
    crossing). See :func:`batched_root_find`.

    Returns
    -------
    (lower, lower_found, upper, upper_found)
    """
    lower, lower_found = batched_root_find(
        func_lower, lo, hi, scan_points, bisect_iters, which="first"
    )
    upper, upper_found = batched_root_find(
        func_upper, lo, hi, scan_points, bisect_iters, which="last"
    )
    return lower, lower_found, upper, upper_found

```

pycudal/cudal/cusp1.py

```
"""
cudal.cusp1
===========

Translation of ``cusp1.sas`` -- Content Uniformity, Sampling Plan 1
(single location / composite sample, USP <905>).

Three analyses, matching the three SAS entry points:

  * :func:`acceptance_limit_table`  (``CALCUSP1`` / ``PRTCUSP1``, A1CUSP1=Y)
  * :func:`probability_of_passing`  (``EVCUSP1``, A2CUSP1=Y)
  * :func:`sample_probability`      (``SMPCUSP1``, A3CUSP1=Y)

Performance
-----------
``acceptance_limit_table`` finds, for every candidate MEAN in the grid,
the boundary standard deviation where the pass-probability crosses the
required bound. Rather than looping over means one at a time (each doing
its own root search), all means are solved *simultaneously* with
:func:`cudal.core.batched_root_find` -- a handful of vectorized numpy
calls covering the whole table instead of hundreds of individual ones.

``probability_of_passing`` vectorizes across the full (U, CV) grid at
once using broadcasting, rather than a Python loop over each combination.
"""

from __future__ import annotations

import numpy as np
import pandas as pd

from .core import batched_root_find, cinv, content_uniformity_bound, probchi, prob
it, probnorm


def acceptance_limit_table(
    number: int,
    target: float,
    lbound: float,
    cilevel: float,
    mean_low: float = 85.1,
    mean_high: float = 114.9,
    mean_step: float = 0.1,
) -> pd.DataFrame:
    """
    SAS: ``%CALCUSP1`` (called when A1CUSP1=Y or A2CUSP1=Y).

    For each candidate sample MEAN, finds the largest sample standard
    deviation (expressed as CV%) for which the probability of passing
    USP <905> is still at least ``lbound`` (given as a percentage, e.g.
    95 for 95%) with ``cilevel``% confidence, for ``number`` (N) units
    and a label claim ``target``.

    The whole MEAN grid is solved in one vectorized batch (see module
    docstring) rather than one root-search per mean.

    Returns
    -------
    pandas.DataFrame with columns ["MEAN", "CV"].
    A CV of 0.0 marks a mean where even a (near) zero standard
    deviation batch would not meet the bound -- SAS's "CV = 0" branch.
    """
```

```
    z = probit((1 + np.sqrt(cilevel / 100)) / 2)
    n = number
    chi = cinv(1 - np.sqrt(cilevel / 100), n - 1)
    target_prob = lbound / 100

    means = np.round(np.arange(mean_low, mean_high + mean_step / 2, mean_step), 6)

    def overbd_of_sd(sampsd):
        # sampsd broadcasts against `means` (row vector (S,1) x (M,) -> (S,M),
        # or shape (M,) during the bisection refinement stage).
        sigma = np.sqrt((n - 1) * sampsd**2 / chi)
        llu = means - z * sigma / np.sqrt(n)
        ulu = means + z * sigma / np.sqrt(n)
        overlbd = content_uniformity_bound(llu, sigma, target)
        overubd = content_uniformity_bound(ulu, sigma, target)
        return np.minimum(overlbd, overubd) - target_prob

    sd_lo, sd_hi = 0.01, 7.8
    root, found = batched_root_find(overbd_of_sd, sd_lo, sd_hi)

    floor_fail = overbd_of_sd(np.full_like(means, sd_lo)) < 0  # even minimal SD f
ails -> CV = 0
    sd = np.where(floor_fail, sd_lo, np.where(found, root, sd_hi))
    cv = np.where(floor_fail, 0.0, 100 * sd / means)

    return pd.DataFrame({"MEAN": means, "CV": cv})


def probability_of_passing(
    table: pd.DataFrame,
    number: int,
    u_values,
    cv_values,
) -> pd.DataFrame:
    """
    SAS: ``%EVCUSP1`` / ``%SIGCUSP1`` (A2CUSP1=Y).

    Given the acceptance-limit ``table`` from :func:`acceptance_limit_table`
    (columns MEAN, CV), evaluate -- for every combination of assumed true
     population mean ``U`` and true population CV -- the probability that a
     sample of size ``number`` drawn from that population lands inside the
     acceptance region traced out by the table (a trapezoid-style sum over
     each pair of adjacent MEAN/CV/STD table entries, exactly reproducing
     the running ``PTRAP + PT`` accumulation in ``%SIGCUSP1``).

     The full (row, U, CV) computation is done as one broadcasted numpy
     array rather than a Python loop over (U, CV) pairs.

     Returns
     -------
     pandas.DataFrame with columns ["U", "CV", "PTRAP"] -- PTRAP is the
     estimated probability of passing.
     """
    t = table.sort_values("MEAN").reset_index(drop=True)
    x = t["MEAN"].to_numpy()
    std = (t["MEAN"] * t["CV"] / 100).to_numpy()
    n = number

    u = np.asarray(u_values, dtype=float)[None, :, None]  # (1, U, 1)
    cv = np.asarray(cv_values, dtype=float)[None, None, :]  # (1, 1, CV)
    sigma = u * cv / 100  # (1, U, CV)

    x_hi = x[1:, None, None]  # (R-1, 1, 1)
    x_lo = x[:-1, None, None]
```

```
    std_hi = std[1:, None, None]
    std_lo = std[:-1, None, None]

    pmean = probnorm((x_hi - u) * np.sqrt(n) / sigma) - probnorm((x_lo - u) * np.
sqrt(n) / sigma)
    aveht = (std_hi + std_lo) / 2
    pstd = probchi((n - 1) * aveht**2 / sigma**2, n - 1)
    ptrap = np.sum(pmean * pstd, axis=0)  # (U, CV)

    u_flat = np.asarray(u_values, dtype=float)
    cv_flat = np.asarray(cv_values, dtype=float)
    uu, cc = np.meshgrid(u_flat, cv_flat, indexing="ij")
    return pd.DataFrame({"U": uu.ravel(), "CV": cc.ravel(), "PTRAP": ptrap.ravel(
)})


def sample_probability(
    mean: float,
    cv: float,
    number: int,
    target: float,
    lbound: float,
    cilevel: float,
) -> dict:
    """
    SAS: ``%SMPCUSP1`` (A3CUSP1=Y).

    Given an observed sample ``mean`` (% claim) and ``cv`` (%), directly
    computes the sample standard deviation and the probability
    ("OVERBD") that future samples from that population will pass
    USP <905>, for ``number`` units, label claim ``target``, at
    ``cilevel``% confidence with the requested ``lbound``% lower bound
    (lbound/cilevel are only used to report context; OVERBD itself does
    not depend on lbound, mirroring the SAS code).

    Returns
    -------
    dict with keys: MEAN, CV, SAMPSD, OVERBD
    """
    z = probit((1 + np.sqrt(cilevel / 100)) / 2)
    n = number
    chi = cinv(1 - np.sqrt(cilevel / 100), n - 1)

    sampsd = mean * cv / 100
    sigma = np.sqrt((n - 1) * sampsd**2 / chi)
    llu = mean - z * sigma / np.sqrt(n)
    ulu = mean + z * sigma / np.sqrt(n)

    overlbd = content_uniformity_bound(llu, sigma, target)
    overubd = content_uniformity_bound(ulu, sigma, target)
    overbd = min(overlbd, overubd)

    return {"MEAN": mean, "CV": cv, "SAMPSD": sampsd, "OVERBD": overbd}

```

pycudal/cudal/cusp2.py

```
"""
cudal.cusp2
===========

Translation of ``Cusp2.sas`` -- Content Uniformity, Sampling Plan 2
(multiple locations, each with its own composite sample -- a
within-location / between-location variance-components model, USP <905>).

Three analyses, matching the three SAS entry points:

  * :func:`acceptance_limit_table`  (``CALCUSP2``, A1CUSP2=Y)
  * :func:`probability_of_passing`  (``EVCUSP2`` / ``SIGCUSP2``, A2CUSP2=Y)
  * :func:`sample_probability`      (``SMPCUSP2``, A3CUSP2=Y)

Variance-component notation (mirrors the SAS variable names)
------------------------------------------------------------
    NN   -- number of units assayed per location (SAS &NUM)
    L    -- number of locations (SAS &LOC)
    N    = NN * L
    SE   -- pooled within-location standard deviation
    SM   -- between-location standard deviation
    VAR  -- total variance used as sigma^2 in the content-uniformity core
    MVAR -- upper confidence bound on the between-location variance
            component, used (via sqrt(MVAR/N)) as the standard error of
            the overall mean for the MEANL/MEANU confidence bound

Performance
-----------
The original SAS code (and a naive Python port) loops over every (SE, SM)
grid cell and, for each one, scans/steps a candidate MEAN up from below
(for MEANL) and down from above (for MEANU) one value at a time. Here the
*entire* SE/SM grid is flattened to a single array of length
``G = len(SE) * len(SM)``, and both boundaries for all G cells are solved
in one vectorized pass with :func:`cudal.core.batched_two_sided_bounds`
-- turning what would be thousands of scalar root searches (each costing
hundreds of probability evaluations) into a couple dozen array-wide
evaluations.
"""

from __future__ import annotations

import numpy as np
import pandas as pd

from .core import (
    batched_two_sided_bounds,
    cinv,
    content_uniformity_bound,
    probchi,
    probit,
    probnorm,
)


def _variance_components(se, sm, nn: int, l: int, chierr: float, chiloc: float):
    """
    Shared VAR / MVAR calculation used by all three CUSP2 analyses.
    Works elementwise on scalars or numpy arrays of any shape.
    """
    se2 = se * se
    h2 = l * (nn - 1) / chierr - 1
    sec = ((1 - 1 / nn) * h2 * se2) ** 2
```

```
63
64      sl2 = sm * sm * nn
65      sl2ub = (l - 1) * sl2 / chiloc
66      h1 = (l - 1) / chiloc - 1
67      first = ((1 / nn) * h1 * sl2) ** 2
68
69      ptest = (1 / nn) * sl2 + (1 - 1 / nn) * se2
70      var = ptest + np.sqrt(first + sec)
71      mvar = sl2ub
72      return var, mvar
73
74
75  def acceptance_limit_table(
76      num: int,
77      loc: int,
78      target: float,
79      lbound: float,
80      cilevel: float,
81      se_values,
82      sm_values,
83      mean_search_range=(80.0, 120.0),
84  ) -> pd.DataFrame:
85      """
86      SAS: ``%CALCUSP2`` (A1CUSP2=Y).
87
88      Parameters
89      ----------
90      num : int
91          Units per location (SAS &NUM).
92      loc : int
93          Number of locations (SAS &LOC).
94      target, lbound, cilevel : float
95          Label claim / target, required lower bound (%), confidence
96          level (%) -- same meaning as in :mod:`cudal.cusp1`.
97      se_values, sm_values : iterable of float
98          Grids of within-location (SE) and between-location (SM) standard
99          deviations to evaluate (SAS steps these by D1=0.10).
100     mean_search_range : (float, float)
101         Range to scan for the MEANL/MEANU roots.
102
103     Returns
104     -------
105     pandas.DataFrame with columns SE, SM, MEANL, MEANU (NaN where no
106     acceptable mean range exists for that SE/SM combination -- i.e. the
107     combined variability is too large to ever pass).
108     """
109     nn, l = num, loc
110     n = nn * l
111     z = probit((1 + np.sqrt(cilevel / 100)) / 2)
112     chierr = cinv(1 - np.sqrt(cilevel / 100), l * (nn - 1))
113     chiloc = cinv(1 - np.sqrt(cilevel / 100), l - 1)
114     target_prob = lbound / 100
115     lo, hi = mean_search_range
116
117     se_grid, sm_grid = np.meshgrid(
118         np.asarray(se_values, dtype=float), np.asarray(sm_values, dtype=float), i
ndexing="ij"
119     )
120     se_flat = se_grid.ravel()
121     sm_flat = sm_grid.ravel()
122
123     var, mvar = _variance_components(se_flat, sm_flat, nn, l, chierr, chiloc)
124     sigma = np.sqrt(var)  # shape (G,)
125     se_of_mean = np.sqrt(mvar / n)  # shape (G,)
```

```
126
127     def func_lower(mean):
128         llu = mean - z * se_of_mean
129         return content_uniformity_bound(llu, sigma, target) - target_prob
130
131     def func_upper(mean):
132         ulu = mean + z * se_of_mean
133         return content_uniformity_bound(ulu, sigma, target) - target_prob
134
135     # Bump scan_points/bisect_iters to avoid crossing of overbd_of_sd(sd)
136     meanl, meanl_found, meanu, meanu_found = batched_two_sided_bounds(
137         func_lower,
138         func_upper,
139         lo,
140         hi,
141         scan_points=200,
142         bisect_iters=30,
143     )
144
145     ok = meanl_found & meanu_found & (meanu > meanl)
146     meanl_out = np.where(ok, meanl, np.nan)
147     meanu_out = np.where(ok, meanu, np.nan)
148
149     return pd.DataFrame({"SE": se_flat, "SM": sm_flat, "MEANL": meanl_out, "MEANU
": meanu_out})
150
151
152def probability_of_passing(
153     table: pd.DataFrame,
154     num: int,
155     loc: int,
156     d1: float,
157     u_values,
158     sigse_values,
159     sigsm_values,
160) -> pd.DataFrame:
161     """
162     SAS: ``%EVCUSP2`` / ``%SIGCUSP2`` (A2CUSP2=Y).
163
164     Given the acceptance-limit ``table`` (columns SE, SM, MEANL, MEANU),
165     sums, over every (SE, SM) grid cell, the probability that a batch with
166     assumed true mean ``U`` and true within-/between-location standard
167     deviations (``sigse``, ``sigsm``) both (a) lands its location means in
168     [MEANL, MEANU], (b) has a within-location SD landing in the SE bin,
169     and (c) has a between-location SD landing in the SM bin.
170
171     The full (table row x U x SIGSE x SIGSM) computation is done as one
172     broadcasted numpy array rather than nested Python loops.
173
174     Returns
175     -------
176     pandas.DataFrame with columns ["U", "SIGSE", "SIGSM", "PSUM"].
177     """
178     t = table.dropna(subset=["MEANL", "MEANU"])
179     nn, l = num, loc
180     n = nn * l
181
182     se = t["SE"].to_numpy()[:, None, None, None]
183     sm = t["SM"].to_numpy()[:, None, None, None]
184     meanl = t["MEANL"].to_numpy()[:, None, None, None]
185     meanu = t["MEANU"].to_numpy()[:, None, None, None]
186
187     u = np.asarray(u_values, dtype=float)[None, :, None, None]
188     sigse = np.asarray(sigse_values, dtype=float)[None, None, :, None]
```

```
    sigsm = np.asarray(sigsm_values, dtype=float)[None, None, None, :]

    expse2 = sigse**2
    expsm2 = expse2 + nn * sigsm**2

    pmean = probnorm((meanu - u) * np.sqrt(n / expsm2)) - probnorm(
        (meanl - u) * np.sqrt(n / expsm2)
    )
    pse = probchi(l * (nn - 1) * se**2 / expse2, l * (nn - 1)) - probchi(
        l * (nn - 1) * (se - d1) ** 2 / expse2, l * (nn - 1)
    )
    psm = probchi((l - 1) * nn * sm**2 / expsm2, l - 1) - probchi(
        (l - 1) * nn * (sm - d1) ** 2 / expsm2, l - 1
    )

    psum = np.sum(pmean * pse * psm, axis=0)  # (U, SIGSE, SIGSM)

    uu, ee, mm = np.meshgrid(
        np.asarray(u_values, dtype=float),
        np.asarray(sigse_values, dtype=float),
        np.asarray(sigsm_values, dtype=float),
        indexing="ij",
    )
    return pd.DataFrame(
        {
            "U": uu.ravel(),
            "SIGSE": ee.ravel(),
            "SIGSM": mm.ravel(),
            "PSUM": psum.ravel(),
        }
    )


def sample_probability(
    mean: float,
    se: float,
    sm: float,
    num: int,
    loc: int,
    target: float,
    cilevel: float,
) -> dict:
    """
    SAS: ``%SMPCUSP2`` (A3CUSP2=Y).

    Given an observed sample MEAN, within-location SD (SE) and
    between-location SD (SM), computes the probability ("OVERBD") that
    future samples from that population pass USP <905>.
    """
    nn, l = num, loc
    n = nn * l
    z = probit((1 + np.sqrt(cilevel / 100)) / 2)
    chierr = cinv(1 - np.sqrt(cilevel / 100), l * (nn - 1))
    chiloc = cinv(1 - np.sqrt(cilevel / 100), l - 1)

    var, mvar = _variance_components(se, sm, nn, l, chierr, chiloc)
    sigma = np.sqrt(var)
    se_of_mean = np.sqrt(mvar / n)

    llu = mean - z * se_of_mean
    ulu = mean + z * se_of_mean

    overbdl = content_uniformity_bound(llu, sigma, target)
    overbdu = content_uniformity_bound(ulu, sigma, target)
```

```
253     overbd = min(overbdl, overbdu)
254
255     return {
256         "MEAN": mean,
257         "SE": se,
258         "SM": sm,
259         "VAR": var,
260         "MVAR": mvar,
261         "SIGMA": sigma,
262         "OVERBDL": overbdl,
263         "OVERBDU": overbdu,
264         "OVERBD": overbd,
265     }
```

pycudal/cudal/disp1.py

```
"""
cudal.disp1
===========

Translation of ``Disp1.sas`` -- Dissolution, Sampling Plan 1
(single location, USP <711>).

Three analyses, matching the three SAS entry points:

  * :func:`acceptance_limit_table`  (``CALDISP1`` / ``PRTDISP1``, A1DISP1=Y)
  * :func:`probability_of_passing`  (``EVDISP1`` / ``SIGDISP1``, A2DISP1=Y)
  * :func:`sample_probability`      (``SMPDISP1``, A3DISP1=Y)

Note the dissolution confidence multiplier is one-sided:
``Z = PROBIT(SQRT(CILEVEL/100))``, unlike the two-sided
``PROBIT((1+SQRT(CILEVEL/100))/2)`` used for content uniformity.

Performance: see :mod:`cudal.cusp1` -- the MEAN grid's boundary standard
deviations are all solved in one vectorized batch via
:func:`cudal.core.batched_root_find`, and ``probability_of_passing``
evaluates the full (U, CV) grid as one broadcasted array.
"""

from __future__ import annotations

import numpy as np
import pandas as pd

from .core import batched_root_find, cinv, dissolution_bound, probchi, probit, pro
bnorm


def acceptance_limit_table(
    number: int,
    q: float,
    lbound: float,
    cilevel: float,
    meanadj_step: float = 0.2,
) -> pd.DataFrame:
    """
    SAS: ``%CALDISP1`` (A1DISP1=Y or A2DISP1=Y).

    Parameters
    ----------
    number : int
        Sample size N (SAS &NUMBER).
    q : float
        USP <711> Q value (% dissolved), e.g. 80.
    lbound, cilevel : float
        Required lower bound (%) and confidence level (%).
    meanadj_step : float
        Step size for the internal MEANADJ = MEAN - Q grid (SAS D = 0.2).

    Returns
    -------
    pandas.DataFrame with columns ["MEAN", "CV"] where MEAN ranges from
    just above Q up to 100 (Q + LIM). CV = 0.0 marks a mean adjustment
    where even a (near) zero SD fails to meet the bound.
    """
    lim = 100 - q
    n = number
    z = probit(np.sqrt(cilevel / 100))
```

```
    chi = cinv(1 - np.sqrt(cilevel / 100), n - 1)
    target_prob = lbound / 100

    meanadjs = np.round(np.arange(meanadj_step, lim + meanadj_step / 2, meanadj_st
ep), 6)

    def overbd_of_sd(sampsd):
        sigma = np.sqrt((n - 1) * sampsd**2 / chi)
        llu = meanadjs - z * sigma / np.sqrt(n)
        return dissolution_bound(llu, sigma) - target_prob

    sd_lo, sd_hi = 0.002, 60.0
    root, found = batched_root_find(overbd_of_sd, sd_lo, sd_hi)

    floor_fail = overbd_of_sd(np.full_like(meanadjs, sd_lo)) < 0
    sd = np.where(floor_fail, sd_lo, np.where(found, root, sd_hi))
    mean = meanadjs + q
    cv = np.where(floor_fail, 0.0, 100 * sd / mean)

    return pd.DataFrame({"MEAN": mean, "CV": cv})


def probability_of_passing(
    table: pd.DataFrame,
    number: int,
    u_values,
    cv_values,
) -> pd.DataFrame:
    """
    SAS: ``%EVDISP1`` / ``%SIGDISP1`` (A2DISP1=Y).

    Same trapezoidal-style accumulation as
    :func:`cudal.cusp1.probability_of_passing`, plus the SAS code's extra
    one-sided upper-tail correction applied to every table row whose mean
    exceeds 99.9 (% claim), using that row's own standard deviation rather
    than an average with its neighbour. The full (row, U, CV) grid is
    evaluated as one broadcasted array.

    Returns
     -------
     pandas.DataFrame with columns ["U", "CV", "PTRAP"].
     """
     t = table.sort_values("MEAN").reset_index(drop=True)
     x = t["MEAN"].to_numpy()
     std = (t["MEAN"] * t["CV"] / 100).to_numpy()
     n = number

     u = np.asarray(u_values, dtype=float)[None, :, None]  # (1, U, 1)
     cv = np.asarray(cv_values, dtype=float)[None, None, :]  # (1, 1, CV)
     sigma = u * cv / 100  # (1, U, CV)

     # main pairwise (trapezoid-style) term, rows 2..end
     x_hi, x_lo = x[1:, None, None], x[:-1, None, None]
     std_hi, std_lo = std[1:, None, None], std[:-1, None, None]
     pmean = probnorm((x_hi - u) * np.sqrt(n) / sigma) - probnorm((x_lo - u) * np.
sqrt(n) / sigma)
     aveht = (std_hi + std_lo) / 2
     pstd = probchi((n - 1) * aveht**2 / sigma**2, n - 1)
     ptrap = np.sum(pmean * pstd, axis=0)  # (U, CV)

     # extra upper-tail correction for every row with X > 99.9
     tail_mask = x > 99.9
     if np.any(tail_mask):
         xt = x[tail_mask][:, None, None]
```

```
        stdt = std[tail_mask][:, None, None]
        pmean_t = 1 - probnorm((xt - u) * np.sqrt(n) / sigma)
        pstd_t = probchi((n - 1) * stdt**2 / sigma**2, n - 1)
        ptrap = ptrap + np.sum(pmean_t * pstd_t, axis=0)

    u_flat = np.asarray(u_values, dtype=float)
    cv_flat = np.asarray(cv_values, dtype=float)
    uu, cc = np.meshgrid(u_flat, cv_flat, indexing="ij")
    return pd.DataFrame({"U": uu.ravel(), "CV": cc.ravel(), "PTRAP": ptrap.ravel(
)})


def sample_probability(
    mean: float,
    cv: float,
    number: int,
    q: float,
    cilevel: float,
) -> dict:
    """
    SAS: ``%SMPDISP1`` (A3DISP1=Y).

    Given an observed sample mean (% dissolved) and CV (%), computes the
    probability ("OVERBD") that future samples pass USP <711>.
    """
    n = number
    z = probit(np.sqrt(cilevel / 100))
    chi = cinv(1 - np.sqrt(cilevel / 100), n - 1)

    meanadj = mean - q
    sampsd = mean * cv / 100
    sigma = np.sqrt((n - 1) * sampsd**2 / chi)
    llu = meanadj - z * sigma / np.sqrt(n)
    overbd = dissolution_bound(llu, sigma)

    return {"MEAN": mean, "CV": cv, "SAMPSD": sampsd, "OVERBD": overbd}
```

pycudal/cudal/disp2.py

```
"""
cudal.disp2
===========

Translation of ``Disp2.sas`` -- Dissolution, Sampling Plan 2
(multiple locations, variance-components model, USP <711>).

Unlike the content-uniformity Sampling Plan 2 (:mod:`cudal.cusp2`), the
dissolution test only has a *lower* bound (Q value), so there is a single
``MEANL`` boundary per (SE, SM) combination, not a [MEANL, MEANU] pair.

Three analyses:

  * :func:`acceptance_limit_table`  (``CALDISP2``, A1DISP2=Y)
  * :func:`probability_of_passing`  (``EVDISP2`` / ``SIGDISP2``, A2DISP2=Y)
  * :func:`sample_probability`      (``SMPDISP2``, A3DISP2=Y)

Performance: as in :mod:`cudal.cusp2`, the whole (SE, SM) grid is
flattened and solved for MEANL in one vectorized batch via
:func:`cudal.core.batched_root_find`, instead of a per-cell scalar search.
"""

from __future__ import annotations

import numpy as np
import pandas as pd

from .core import batched_root_find, cinv, dissolution_bound, probchi, probit, pro
bnorm
from .cusp2 import _variance_components


def acceptance_limit_table(
    num: int,
    loc: int,
    q: float,
    lbound: float,
    cilevel: float,
    se_values,
    sm_values,
    meanadj_search_range=(-20.0, 100.0),
) -> pd.DataFrame:
    """
    SAS: ``%CALDISP2`` (A1DISP2=Y).

    For each (SE, SM) combination, finds the smallest acceptable sample
    mean (MEAN = MEANL + Q) such that the probability of passing USP
    <711> is still at least ``lbound``%. The whole SE/SM grid is solved
    in one vectorized batch (see module docstring).

    Returns
    -------
    pandas.DataFrame with columns ["SE", "SM", "MEAN"] (NaN MEAN where no
    acceptable mean exists in the search range for that SE/SM).
    """
    nn, l = num, loc
    n = nn * l
    lim = 100 - q
    z = probit(np.sqrt(cilevel / 100))
    chierr = cinv(1 - np.sqrt(cilevel / 100), l * (nn - 1))
    chiloc = cinv(1 - np.sqrt(cilevel / 100), l - 1)
    target_prob = lbound / 100
```

```
    lo, hi = meanadj_search_range
    hi = min(hi, lim)

    se_grid, sm_grid = np.meshgrid(
        np.asarray(se_values, dtype=float), np.asarray(sm_values, dtype=float), in
dexing="ij"
    )
    se_flat = se_grid.ravel()
    sm_flat = sm_grid.ravel()

    var, mvar = _variance_components(se_flat, sm_flat, nn, l, chierr, chiloc)
    sigma = np.sqrt(var)
    se_of_mean = np.sqrt(mvar / n)

    def func(meanadj):
        llu = meanadj - z * se_of_mean
        return dissolution_bound(llu, sigma) - target_prob

    meanl, found = batched_root_find(func, lo, hi, which="first")
    mean = np.where(found, meanl + q, np.nan)

    return pd.DataFrame({"SE": se_flat, "SM": sm_flat, "MEAN": mean})


def probability_of_passing(
    table: pd.DataFrame,
    num: int,
    loc: int,
    dse: float,
    dsm: float,
    u_values,
    sigse_values,
    sigsm_values,
) -> pd.DataFrame:
    """
    SAS: ``%EVDISP2`` / ``%SIGDISP2`` (A2DISP2=Y).

    One-sided analogue of :func:`cudal.cusp2.probability_of_passing`:
    ``PMEAN = 1 - PROBNORM((MEAN - U) * SQRT(N / EXPSM2))`` since there is
     only a lower acceptance bound on the mean. The full grid is evaluated
     as one broadcasted array.

     Returns
     -------
     pandas.DataFrame with columns ["U", "SIGSE", "SIGSM", "PSUM"].
     """
     t = table.dropna(subset=["MEAN"])
     nn, l = num, loc
     n = nn * l

     se = t["SE"].to_numpy()[:, None, None, None]
     sm = t["SM"].to_numpy()[:, None, None, None]
     mean = t["MEAN"].to_numpy()[:, None, None, None]

     u = np.asarray(u_values, dtype=float)[None, :, None, None]
     sigse = np.asarray(sigse_values, dtype=float)[None, None, :, None]
     sigsm = np.asarray(sigsm_values, dtype=float)[None, None, None, :]

     expse2 = sigse**2
     expsm2 = expse2 + nn * sigsm**2

     pmean = 1 - probnorm((mean - u) * np.sqrt(n / expsm2))
     pse = probchi(l * (nn - 1) * se**2 / expse2, l * (nn - 1)) - probchi(
         l * (nn - 1) * (se - dse) ** 2 / expse2, l * (nn - 1)
```

```
    )
    psm = probchi((l - 1) * nn * sm**2 / expsm2, l - 1) - probchi(
        (l - 1) * nn * (sm - dsm) ** 2 / expsm2, l - 1
    )

    psum = np.sum(pmean * pse * psm, axis=0)  # (U, SIGSE, SIGSM)

    uu, ee, mm = np.meshgrid(
        np.asarray(u_values, dtype=float),
        np.asarray(sigse_values, dtype=float),
        np.asarray(sigsm_values, dtype=float),
        indexing="ij",
    )
    return pd.DataFrame(
        {
            "U": uu.ravel(),
            "SIGSE": ee.ravel(),
            "SIGSM": mm.ravel(),
            "PSUM": psum.ravel(),
        }
    )


def sample_probability(
    mean: float,
    se: float,
    sm: float,
    num: int,
    loc: int,
    q: float,
    cilevel: float,
) -> dict:
    """
    SAS: ``%SMPDISP2`` (A3DISP2=Y).

    Given an observed sample MEAN, within-location SD (SE) and
    between-location SD (SM), computes the probability ("OVERBD") that
    future samples pass USP <711>.
    """
    nn, l = num, loc
    n = nn * l
    z = probit(np.sqrt(cilevel / 100))
    chierr = cinv(1 - np.sqrt(cilevel / 100), l * (nn - 1))
    chiloc = cinv(1 - np.sqrt(cilevel / 100), l - 1)

    var, mvar = _variance_components(se, sm, nn, l, chierr, chiloc)
    sigma = np.sqrt(var)
    se_of_mean = np.sqrt(mvar / n)

    meanadj = mean - q
    llu = meanadj - z * se_of_mean
    overbd = dissolution_bound(llu, sigma)

    return {
        "MEAN": mean,
        "SE": se,
        "SM": sm,
        "VAR": var,
        "MVAR": mvar,
        "SIGMA": sigma,
        "OVERBD": overbd,
    }
```

pycudal/cudal/gui.py

```
"""
cudal_gui_pyside6.py  --  FINAL integrated PySide6 version

CuDAL GUI: 4 tabs (CU Plan 1/2, Dissolution Plan 1/2) x 3 modes
(table / evaluate / sample), with:

* modern flat "card" design, strict column-aligned parameter forms
* background-thread calculations with real progress bar + status text
* scrollable parameters, live validation, tooltips, Reset defaults
* results table: sorting, zebra stripes, conditional P(pass) coloring,
  Ctrl+C copy, CSV export, modal matplotlib dialog (lines+spline and
  heatmap with 80/90% contours), Save PNG
* settings persistence (parameters, mode, tab, window geometry)
* logo.png + fonts/ (TTF) support on Windows AND Linux
* menu, shortcuts (Ctrl+R/E/P, F1), About dialog, --selftest

Requirements:
    pip install PySide6 numpy pandas scipy matplotlib openpyxl

Run:
    python cudal_gui_pyside6.py            # GUI
    python cudal_gui_pyside6.py --selftest # quick unit checks
"""

from __future__ import annotations

import csv
import json
import math
import os
import sys
import time
import traceback

from PySide6.QtCore import QSize, Qt, QThread, QUrl, Signal
from PySide6.QtGui import (
    QAction,
    QBrush,
    QColor,
    QDesktopServices,
    QFont,
    QFontDatabase,
    QIcon,
    QKeySequence,
    QPainter,
    QPixmap,
    QShortcut,
)
from PySide6.QtWidgets import (
    QAbstractItemView,
    QApplication,
    QButtonGroup,
    QComboBox,
    QDialog,
    QFileDialog,
    QFrame,
    QGridLayout,
    QGroupBox,
    QHBoxLayout,
    QLabel,
    QLineEdit,
    QMainWindow,
```

```
    QMessageBox,
    QProgressBar,
    QPushButton,
    QRadioButton,
    QScrollArea,
    QTableWidget,
    QTableWidgetItem,
    QTabWidget,
    QVBoxLayout,
    QWidget,
)

try:
    from reportlab.lib.pagesizes import landscape, letter
    from reportlab.pdfgen import canvas as rl_canvas

    HAVE_PDF = True
except Exception:  # pragma: no cover
    HAVE_PDF = False

# Heavy / optional dependencies load in stages after the splash appears.
np = pd = None
cusp1 = cusp2 = disp1 = disp2 = None
Figure = FigureCanvas = NavigationToolbar = None
make_interp_spline = None
HAVE_CUDAL = HAVE_MPL = HAVE_SPLINE = HAVE_XLSX = False

REPO_URL = "https://github.com/moazelessawey/pycudal"

# ---------------------------------------------------------------------------
# Constants
# ---------------------------------------------------------------------------
VERSION = "1.0.8 (PySide6, modern UI)"

BG = "#f4f6f9"
PANEL_BG = "#ffffff"
ACCENT = "#2f6fed"
ACCENT_DARK = "#204ea6"
TEXT = "#1c2733"
MUTED = "#64748b"
BORDER = "#e2e8f0"
OK_GREEN = "#1a8754"
ERR_RED = "#c0392b"

SERIES_COLORS = [ACCENT, OK_GREEN, ERR_RED, "#8e44ad", "#e67e22", "#16a085", "#c2
417d", "#5b6470"]

SETTINGS_PATH = os.path.join(os.path.dirname(os.path.abspath(__file__)), "cudal_g
ui_settings.json")

# ---------------------------------------------------------------------------
# SAS-style PDF listing export (reportlab, Courier, column wrapping)
# ---------------------------------------------------------------------------
_PDF_FONT = "Courier"
_PDF_FS = 8.0
_PDF_LEAD = 11.0
_PDF_CHAR = 0.6 * _PDF_FS


def _fmt_num(v):
    """Format a numeric value to 2 decimal places; '*' for NaN/Inf."""
    try:
        if isinstance(v, float) and math.isnan(v):
            return "*"
```

```
        if pd.isna(v):
            return "*"
        f = float(v)
        if math.isnan(f) or math.isinf(f):
            return "*"
        return f"{f:.2f}"
    except Exception:
        return str(v)


def _cell(v):
    return _fmt_num(v)


def write_sas_pdf(path, df, title_lines):
    """Write `df` to `path` as a SAS-listing-style PDF (Courier, wrapped).

    The report title block AND the table header are repeated on every page.
    """
    page_w, page_h = landscape(letter)
    margin = 36.0
    usable = int((page_w - 2 * margin) / _PDF_CHAR)

    c = rl_canvas.Canvas(path, pagesize=landscape(letter))
    c.setTitle("CuDAL results")
    st = {"y": page_h - margin, "table_header_fn": None}

    # -- low-level drawing (no page checks, used inside new_page) ------------
    def raw(text, indent=0):
        c.setFont(_PDF_FONT, _PDF_FS)
        c.drawString(margin + indent * _PDF_CHAR, st["y"], text)
        st["y"] -= _PDF_LEAD

    def raw_centered(text):
        raw(text, max(0, (usable - len(text)) // 2))

    def draw_title():
        for t in title_lines:
            raw_centered(t)
        raw("")

    def new_page():
        c.showPage()
        st["y"] = page_h - margin
        draw_title()  # title on every page
        if st["table_header_fn"] is not None:  # table header on every page
            st["table_header_fn"]()

    # -- page-checking drawing -----------------------------------------------
    def put(text, indent=0):
        if st["y"] < margin:
            new_page()
        raw(text, indent)

    def centered(text):
        put(text, max(0, (usable - len(text)) // 2))

    def blank(n=1):
        for _ in range(n):
            put("")

    def need(n):
        """Force a page break unless `n` more lines fit on this page."""
        if st["y"] - n * _PDF_LEAD < margin:
```

```
            new_page()

    draw_title()  # first-page title

    cols = [str(x) for x in df.columns]
    low = {cl.lower(): cl for cl in cols}

    # ------------ Plan-2 style matrix: SE rows x SM groups of LL/UL --------
    if {"se", "sm", "ll", "ul"} <= set(low):
        se_c, sm_c, ll_c, ul_c = low["se"], low["sm"], low["ll"], low["ul"]
        tmp = df.copy()
        tmp["_se"] = pd.to_numeric(tmp[se_c], errors="coerce")
        tmp["_sm"] = pd.to_numeric(tmp[sm_c], errors="coerce")
        ses = sorted(tmp["_se"].dropna().unique())
        sms = sorted(tmp["_sm"].dropna().unique())

        get = {}
        for r in tmp.itertuples():
            if pd.notna(r._se) and pd.notna(r._sm):
                get[(r._se, r._sm)] = (_fmt_num(r[ll_c]), _fmt_num(r[ul_c]))

        ll_w = max([2] + [len(v[0]) for v in get.values()])
        ul_w = max([2] + [len(v[1]) for v in get.values()])
        grp_w = ll_w + 1 + ul_w
        se_w = max([2] + [len(_fmt_num(s)) for s in ses])
        gap = 3
        per_page = max(1, (usable - se_w + gap) // (grp_w + gap))

        def header(seg):
            centered("STANDARD DEVIATION OF LOCATION MEANS")
            blank()
            line = " " * se_w
            for sm in seg:
                line += (" " * gap) + f"{_fmt_num(sm):>{grp_w}}"
            put(line)
            line = f"{'SE':>{se_w}}"
            for _ in seg:
                line += (" " * gap) + f"{'LL':>{ll_w}} {'UL':>{ul_w}}"
            put(line)
            blank()

        HDR_LINES = 5  # banner + blank + SM row + LL/UL row + blank

        for start in range(0, len(sms), per_page):  # column wrapping
            seg = sms[start : start + per_page]
            st["table_header_fn"] = None  # don't repeat old header
            need(HDR_LINES + 1)
            header(seg)
            st["table_header_fn"] = lambda seg=seg: header(seg)
            for se in ses:
                line = f"{_fmt_num(se):>{se_w}}"
                for sm in seg:
                    ll, ul = get.get((se, sm), ("*", "*"))
                    line += (" " * gap) + f"{ll:>{ll_w}} {ul:>{ul_w}}"
                put(line)  # auto page-break repeats
            blank()  # title + segment header
        st["table_header_fn"] = None
        c.save()
        return

    # ------------ Plan-1 style: long rows wrapped into side-by-side blocks --
    body = [[_cell(v) for v in row] for _, row in df.iterrows()]
    widths = [
```

```
252         max(len(cols[j]), max((len(r[j]) for r in body), default=0)) for j in ran
ge(len(cols))
253     ]
254
255     hdr = []
256     for j, h in enumerate(cols):
257         parts = h.split(" ", 1)
258         if len(parts) == 2 and len(parts[0]) <= widths[j] and len(parts[1]) <= wi
dths[j]:
259             hdr.append((parts[0], parts[1]))
260         else:
261             hdr.append((h, ""))
262     hdr_lines = 2 if any(b for _a, b in hdr) else 1
263
264     gap, block_gap = 3, 4
265     block_w = sum(widths) + gap * (len(widths) - 1)
266     n_blocks = max(1, (usable + block_gap) // (block_w + block_gap))
267     rpb = max(1, -(-len(body) // n_blocks))  # rows per block
268     chunks = [body[i : i + rpb] for i in range(0, len(body), rpb)]
269
270     def header_line(li):
271         return (" " * gap).join(f"{hdr[j][li]:^{widths[j]}}" for j in range(len(c
ols)))
272
273     def block_line(chunk, i):
274         parts = []
275         for j in range(len(cols)):
276             val = chunk[i][j] if i < len(chunk) else ""
277             parts.append(f"{val:>{widths[j]}}")
278         return (" " * gap).join(parts)
279
280     for p in range(0, len(chunks), n_blocks):
281         page_chunks = chunks[p : p + n_blocks]
282
283         def draw_hdr(_pc=page_chunks):
284             for li in range(hdr_lines):
285                 put((" " * block_gap).join(header_line(li) for _ in _pc))
286             blank()
287
288         st["table_header_fn"] = None  # don't repeat old header
289         need(hdr_lines + 2)
290         draw_hdr()
291         st["table_header_fn"] = draw_hdr  # repeat on page breaks
292         for i in range(rpb):
293             put((" " * block_gap).join(block_line(ch, i) for ch in page_chunks))
294         blank()
295     st["table_header_fn"] = None
296
297     c.save()
298
299
300def resource_path(relative: str) -> str:
301     """Resolve bundled files both from source and from a PyInstaller bundle."""
302     base = getattr(sys, "_MEIPASS", os.path.dirname(os.path.abspath(__file__)))
303     return os.path.join(base, relative)
304
305
306def _register_local_fonts() -> str | None:
307     """Load every TTF/OTF in fonts/ into the Qt application font database."""
308     font_dir = resource_path("fonts")
309     if not os.path.isdir(font_dir):
310         return None
311     family = None
312     for f in sorted(os.listdir(font_dir)):
```

```
        if f.lower().endswith((".ttf", ".otf")):
            fid = QFontDatabase.addApplicationFont(os.path.join(font_dir, f))
            if fid != -1 and family is None:
                fams = QFontDatabase.applicationFontFamilies(fid)
                if fams:
                    family = fams[0]
    return family


def build_stylesheet(family: str) -> str:
    """Balanced 'classic-modern' theme: familiar desktop look, clean palette."""
    return f"""
    QMainWindow, QDialog {{ background: {BG}; }}
    QWidget {{ background: {BG}; color: #1f2937; font-family: "{family}"; }}

    QLabel {{ background: transparent; }}
    QLabel#header {{ font-size: 17px; font-weight: 700; }}
    QLabel#subheader {{ color: {MUTED}; }}
    QLabel#muted {{ color: {MUTED}; font-size: 9pt; }}
    QLabel#section {{ color: {ACCENT_DARK}; font-weight: 700; }}
    QLabel#fieldlabel {{ color: #2f3a4d; }}

    /* panels & group boxes: light cards with a classic frame */
    QFrame#panel, QGroupBox#card {{
        background: {PANEL_BG};
        border: 1px solid #cfd8e3;
        border-radius: 6px;
    }}
    QGroupBox#card {{ margin-top: 12px; font-weight: 600; color: #2f3a4d; }}
    QGroupBox#card::title {{
        subcontrol-origin: margin; left: 10px; padding: 0 4px;
        color: {ACCENT_DARK};
    }}

    /* radio buttons: classic circle + dot, accent colour */
    QRadioButton {{ spacing: 7px; padding: 3px; background: transparent; }}
    QRadioButton::indicator {{
        width: 15px; height: 15px;
        border: 1px solid #8b97a8; border-radius: 8px; background: white;
    }}
    QRadioButton::indicator:hover {{ border-color: {ACCENT}; }}
    QRadioButton::indicator:checked {{
        border: 1px solid {ACCENT_DARK};
        background: qradialgradient(cx:0.5, cy:0.5, radius:0.45, fx:0.5, fy:0.5,
                    stop:0 {ACCENT}, stop:0.55 {ACCENT},
                    stop:0.56 white, stop:1 white);
    }}

    /* inputs: white with a firm border */
    QLineEdit {{
        background: white; border: 1px solid #aeb9c8; border-radius: 3px;
        padding: 4px 8px; selection-background-color: {ACCENT};
    }}
    QLineEdit:hover {{ border-color: #8b97a8; }}
    QLineEdit:focus {{ border: 1px solid {ACCENT}; }}
    QLineEdit[invalid="true"] {{ background: #fdf1f1; border: 1px solid {ERR_RED}
; color: {ERR_RED}; }}

    /* buttons: subtle gradient like classic toolkits */
    QPushButton {{
        background: qlineargradient(x1:0, y1:0, x2:0, y2:1, stop:0 #fdfefe, stop:
1 #e7ebf2);
        color: #2f3a4d; border: 1px solid #aeb9c8; border-radius: 4px;
        padding: 7px 14px;
```

```
    }}
    QPushButton:hover {{
        background: qlineargradient(x1:0, y1:0, x2:0, y2:1, stop:0 #f4f8ff, stop:
1 #dfe8f7);
        border-color: {ACCENT}; color: {ACCENT_DARK};
    }}
    QPushButton:pressed {{ background: #dfe4ec; border-color: #8b97a8; }}
    QPushButton:disabled {{ background: #eef1f5; color: #9aa7b8; border-
color: #d5dce6; }}
    QPushButton#accent {{
        background: qlineargradient(x1:0, y1:0, x2:0, y2:1, stop:0 #5a8ff2, stop:
1 {ACCENT});
        border: 1px solid {ACCENT_DARK}; color: white; padding: 8px 20px;
    }}
    QPushButton#accent:hover {{
        background: qlineargradient(x1:0, y1:0, x2:0, y2:1, stop:0 #4c82f3, stop:
1 {ACCENT_DARK});
    }}
    QPushButton#accent:disabled {{ background: #b6c6ea; border-
color: #9db1dd; color: #f8fafc; }}
    QPushButton#outline {{ background: white; border: 1px solid {ACCENT}; color:
{ACCENT_DARK}; }}
    QPushButton#outline:hover {{ background: #eaf1fe; }}

    /* progress: framed bar with accent fill, same height as the Run button */
    QProgressBar {{
        background: white;
        border: 1px solid #aeb9c8;
        border-radius: 4px;
        text-align: center;
    }}
    QProgressBar::chunk {{
        background: qlineargradient(x1:0, y1:0, x2:0, y2:1, stop:0 #6ea0f5, stop:
1 {ACCENT});
        border-radius: 3px;
    }}

    /* scrollbars: classic width, soft colours, no arrows */
    QScrollArea {{ border: none; background: transparent; }}
    QScrollBar:vertical {{ background: #eef1f5; width: 12px; border: 1px solid #d
fe4ec; }}
    QScrollBar::handle:vertical {{ background: #c3ccd8; border-radius: 6px; min-
height: 24px; margin: 1px; }}
    QScrollBar::handle:vertical:hover {{ background: #a9b4c4; }}
    QScrollBar::add-line:vertical, QScrollBar::sub-line:vertical {{ height: 0; }}
    QScrollBar:horizontal {{ background: #eef1f5; height: 12px; border: 1px solid
 #dfe4ec; }}
    QScrollBar::handle:horizontal {{ background: #c3ccd8; border-
radius: 6px; min-width: 24px; margin: 1px; }}
    QScrollBar::add-line:horizontal, QScrollBar::sub-
line:horizontal {{ width: 0; }}

    /* tabs: classic raised tabs on a framed pane */
    QTabWidget::pane {{
        border: 1px solid #cfd8e3; background: {PANEL_BG}; border-radius: 4px;
    }}
    QTabBar::tab {{
        background: qlineargradient(x1:0, y1:0, x2:0, y2:1, stop:0 #f2f5f9, stop:
1 #e2e7ee);
        color: {MUTED};
        border: 1px solid #cfd8e3; border-bottom: none;
        border-top-left-radius: 5px; border-top-right-radius: 5px;
        padding: 7px 16px; margin-right: 2px;
    }}
```

```
426     QTabBar::tab:hover {{ color: #2f3a4d; background: #eef3fa; }}
427     QTabBar::tab:selected {{ background: {PANEL_BG}; color: {ACCENT_DARK}; font-
weight: 600; }}
428
429     /* table: classic grid with soft header */
430     QTableWidget {{
431         background: white; alternate-background-color: #f4f7fb;
432         border: 1px solid #cfd8e3; border-radius: 4px;
433         gridline-color: #dde3ec;
434         selection-background-color: #cfdffc; selection-color: #101828;
435     }}
436     QHeaderView::section {{
437         background: qlineargradient(x1:0, y1:0, x2:0, y2:1, stop:0 #fbfcfe, stop:
1 #edf0f5);
438         color: #334155; border: none;
439         border-right: 1px solid #dde3ec; border-bottom: 1px solid #cfd8e3;
440         padding: 5px; font-weight: 600;
441     }}
442
443     QComboBox {{
444         background: white; border: 1px solid #aeb9c8; border-radius: 3px;
445         padding: 4px 8px; min-width: 140px;
446     }}
447     QComboBox:hover {{ border-color: {ACCENT}; }}
448     QComboBox::drop-down {{
449         subcontrol-origin: padding; subcontrol-position: center right;
450         width: 20px; border: none;
451         background: qlineargradient(x1:0, y1:0, x2:0, y2:1, stop:0 #f6f8fb, stop:
1 #e2e7ee);
452     }}
453     QComboBox QAbstractItemView {{
454         background: white; border: 1px solid #aeb9c8;
455         selection-background-color: #cfdffc;
456     }}
457
458     QMenuBar {{ background: #eef1f6; border-bottom: 1px solid #cfd8e3; }}
459     QMenuBar::item {{ background: transparent; padding: 4px 10px; }}
460     QMenuBar::item:selected {{ background: #d7e3fc; border-radius: 3px; }}
461     QMenu {{ background: {PANEL_BG}; border: 1px solid #aeb9c8; padding: 4px; }}
462     QMenu::item {{ padding: 5px 24px; border-radius: 3px; }}
463     QMenu::item:selected {{ background: #cfdffc; }}
464     QMenu::separator {{ height: 1px; background: #dde3ec; margin: 4px 8px; }}
465
466     QStatusBar {{ background: #eef1f6; color: {MUTED}; border-
top: 1px solid #cfd8e3; }}
467
468     /* classic pale-yellow tooltip */
469     QToolTip {{
470         background: #ffffe1; color: #333333; border: 1px solid #767676; padding:
4px;
471     }}
472     QToolBar {{ background: #eef1f6; border: 0; border-bottom: 1px solid #cfd8e3;
473                 spacing: 4px; padding: 4px; }}
474     QToolBar QToolButton {{ background: transparent; border: 1px solid transparen
t;
475                             border-
radius: 4px; padding: 4px 8px; color: #2f3a4d; }}
476     QToolBar QToolButton:hover {{ background: #dfe8f7; border-color: #aeb9c8; }}
477     QToolBar QToolButton:pressed {{ background: #cfdffc; }}
478     """
479
480
481# ---------------------------------------------------------------------------
482# Plot helpers
```

```
# ---------------------------------------------------------------------------
def _spline_xy(xs, ys, samples=300):
    """Smooth cubic spline through (xs, ys); linear fallback w/o scipy."""
    xs = np.asarray(xs, dtype=float)
    ys = np.asarray(ys, dtype=float)
    ux, inv = np.unique(xs, return_inverse=True)
    uy = np.array([ys[inv == i].mean() for i in range(ux.size)]) if ux.size != xs
.size else ys
    order = np.argsort(ux)
    ux, uy = ux[order], uy[order]
    if ux.size < 2:
        return ux, uy
    if HAVE_SPLINE:
        k = min(3, ux.size - 1)
        try:
            spl = make_interp_spline(ux, uy, k=k)
            xx = np.linspace(ux[0], ux[-1], samples)
            return xx, spl(xx)
        except Exception:
            pass
    xx = np.linspace(ux[0], ux[-1], samples)
    return xx, np.interp(xx, ux, uy)


def _grid_columns(df):
    """Detect a 3-column grid and return (x, y, z) column *names*.

    The surface value (z) is the column with the most distinct values
    (it changes on nearly every row); the two axis columns are the rest,
    with x = the axis having more distinct values.  Returns None when the
    DataFrame is not a proper 3-column grid (e.g. the acceptance-limit or
    probability tables in a different column order are still handled).
    """
    num = df.select_dtypes(include=[np.number]).dropna()
    if num.shape[1] != 3 or len(num) < 4:
        return None

    uniq = [num[c].nunique() for c in num.columns]
    if min(uniq) < 2:
        return None

    zcol = num.columns[int(np.argmax(uniq))]
    axes = sorted(
        (c for c in num.columns if c != zcol), key=lambda c: num[c].nunique(), re
verse=True
    )

    # sanity check: the surface must vary more than either axis
    if num[zcol].nunique() <= max(num[axes[0]].nunique(), num[axes[1]].nunique())
:
        return None

    return str(axes[0]), str(axes[1]), str(zcol)


def _is_plan2_table(df):
    """Detect a Plan 2 acceptance limit table (SE, SM, and >=2 value columns)."""
    cols = [str(c).lower() for c in df.columns]
    return "se" in cols and "sm" in cols and len(df.columns) >= 4


def _get_plan2_axes_and_z(df):
    """Return (x_col, y_col, [z_cols]) for Plan 2 tables."""
    cols = [str(c) for c in df.columns]
```

```
    lower_cols = [c.lower() for c in cols]
    x_col = cols[lower_cols.index("se")] if "se" in lower_cols else cols[0]
    y_col = cols[lower_cols.index("sm")] if "sm" in lower_cols else cols[1]
    z_cols = [
        c
        for c, lc in zip(cols, lower_cols)
        if lc not in ("se", "sm") and pd.api.types.is_numeric_dtype(df[c])
    ]
    return x_col, y_col, z_cols


def _plan2_eval_columns(df):
    """Detect a Plan-2 probability-of-passing grid (U x within-SD x between-SD -
> P).

    Returns (u_col, se_col, sm_col, p_col) or None.
    """
    num = df.select_dtypes(include=[np.number])
    if num.shape[1] != 4:
        return None
    cols = [str(c) for c in num.columns]
    low = [c.lower() for c in cols]

    def find(*keys):
        for k in keys:
            for c, lc in zip(cols, low):
                if k in lc:
                    return c
        return None

    p_col = find("prob", "pass", "psum")
    se_col = find("within", "sigse") or next((c for c, lc in zip(cols, low) if lc
 == "se"), None)
    sm_col = find("between", "sigsm") or next((c for c, lc in zip(cols, low) if l
c == "sm"), None)
    if None in (p_col, se_col, sm_col):
        return None
    rest = [c for c in cols if c not in (p_col, se_col, sm_col)]
    if len(rest) != 1:
        return None
    return rest[0], se_col, sm_col, p_col


def build_plan2_eval_plot(fig, df, u_col, se_col, sm_col, p_col):
    """Faceted view of the Plan-2 probability-of-passing surface.

    One panel per between-location SD (SM); inside each panel one spline
    curve per within-location SD (SE):  P(pass) vs true mean U.
    Dashed lines mark the usual 80 % / 90 % coverage levels.
    """
    work = pd.DataFrame(
        {
            "u": pd.to_numeric(df[u_col], errors="coerce"),
            "se": pd.to_numeric(df[se_col], errors="coerce"),
            "sm": pd.to_numeric(df[sm_col], errors="coerce"),
            "p": pd.to_numeric(df[p_col], errors="coerce"),
        }
    ).dropna()

    if work.empty:
        fig.text(0.5, 0.5, "Not enough numeric data to plot.", ha="center", va="c
enter")
        fig.tight_layout()
        return
```

```
604
605     sm_vals = sorted(work["sm"].unique())
606     se_vals = sorted(work["se"].unique())
607     n = len(sm_vals)
608     ncols = min(3, n)
609     nrows = -(-n // ncols)  # ceil without importing math
610     axes = fig.subplots(nrows, ncols, squeeze=False, sharex=True, sharey=True)
611
612     for i, smv in enumerate(sm_vals):
613         ax = axes[i // ncols][i % ncols]
614         panel = work[work["sm"] == smv]
615         for color, sev in zip(SERIES_COLORS, se_vals):
616             sub = panel[panel["se"] == sev].sort_values("u")
617             xs = sub["u"].to_numpy(dtype=float)
618             ys = sub["p"].to_numpy(dtype=float)
619             if xs.size == 0:
620                 continue
621             ax.plot(xs, ys, "o", color=color, ms=3, alpha=0.7)
622             xx, yy = _spline_xy(xs, ys)
623             ax.plot(xx, yy, "-
", color=color, lw=1.6, label=f"{se_col} = {sev:g}")
624         for t in (0.8, 0.9):
625             ax.axhline(t, ls="--", lw=0.8, color="0.5")
626         ax.set_title(f"{sm_col} = {smv:g}", fontsize=9)
627         ax.grid(True, alpha=0.3)
628         if i // ncols == nrows - 1:
629             ax.set_xlabel(u_col, fontsize=9)
630         if i % ncols == 0:
631             ax.set_ylabel(p_col, fontsize=9)
632
633     for j in range(n, nrows * ncols):  # hide unused panels
634         axes[j // ncols][j % ncols].set_axis_off()
635
636     handles, labels = axes[0][0].get_legend_handles_labels()
637     if handles:
638         fig.legend(
639             handles, labels, loc="lower center", ncol=min(len(labels), 4), fontsi
ze=8, frameon=False
640         )
641     fig.suptitle(
642         f"Probability of passing vs {u_col}  (panels: {sm_col}, curves: {se_col})
", fontsize=10
643     )
644     fig.tight_layout(rect=(0, 0.05, 1, 0.95))
645
646
647def build_results_plot_plan2(fig, df):
648     """Family of curves for Plan 2 tables: LL/UL vs SE, grouped by SM."""
649     x_col, y_col, z_cols = _get_plan2_axes_and_z(df)
650     if not z_cols:
651         fig.text(0.5, 0.5, "Not enough data to plot.", ha="center", va="center")
652         return
653
654     n_z = len(z_cols)
655     axes = fig.subplots(1, n_z, squeeze=False)
656
657     for idx, z_col in enumerate(z_cols):
658         ax = axes[0, idx]
659         grouped = df.groupby(y_col, sort=True)
660         for color, (sm_val, sub) in zip(SERIES_COLORS, grouped):
661             xs = sub[x_col].to_numpy(dtype=float)
662             ys = sub[z_col].to_numpy(dtype=float)
663             order = np.argsort(xs)
664             xs, ys = xs[order], ys[order]
```

```
665
666             xx, yy = _spline_xy(xs, ys)
667             ax.plot(xx, yy, "-
", color=color, lw=1.6, label=f"{y_col}={sm_val:g}")
668             ax.plot(xs, ys, "o", color=color, ms=3, alpha=0.7)
669
670         ax.set_xlabel(x_col)
671         ax.set_ylabel(z_col)
672         ax.set_title(f"{z_col} vs {x_col}")
673         ncol = 2 if len(grouped) > 6 else 1
674         ax.legend(fontsize=7, loc="best", ncol=ncol)
675         ax.grid(True, alpha=0.3)
676
677     fig.tight_layout()
678
679
680def build_heatmap_plan2(fig, df, z_col, thresholds=(0.8, 0.9)):
681     """Contour-filled heatmap for Plan 2 tables (SE x SM -> Z)."""
682     x_col, y_col, z_cols = _get_plan2_axes_and_z(df)
683     if not z_cols:
684         fig.text(0.5, 0.5, "Not enough data to plot.", ha="center", va="center")
685         return
686     if z_col not in z_cols:
687         z_col = z_cols[0]
688
689     work = df[[x_col, y_col, z_col]].copy()
690     work = work.apply(pd.to_numeric, errors="coerce").dropna()
691     piv = work.pivot_table(index=y_col, columns=x_col, values=z_col, aggfunc="mea
n")
692     piv = piv.sort_index(axis=0).sort_index(axis=1)
693
694     X = piv.columns.to_numpy(dtype=float)
695     Y = piv.index.to_numpy(dtype=float)
696     Z = np.ma.masked_invalid(piv.to_numpy(dtype=float))
697
698     ax = fig.add_subplot(111)
699     if Z.count() == 0 or X.size < 2 or Y.size < 2:
700         ax.text(0.5, 0.5, "Not enough grid points for a heatmap.", ha="center", v
a="center")
701         fig.tight_layout()
702         return
703
704     cs = ax.contourf(X, Y, Z, levels=min(24, max(6, X.size + Y.size)), cmap="viri
dis")
705     fig.colorbar(cs, ax=ax, label=z_col)
706
707     if X.size <= 15:
708         ax.set_xticks(X)
709     if Y.size <= 15:
710         ax.set_yticks(Y)
711
712     if float(Z.max()) <= 1.01 and float(Z.min()) >= -0.01:
713         for i, t in enumerate(thresholds):
714             if float(Z.min()) <= t <= float(Z.max()):
715                 ax.contour(X, Y, Z, levels=[t], colors="white", linewidths=1.2)
716                 ax.text(
717                     0.02,
718                     0.98 - 0.06 * i,
719                     f"white line = {t:.0%}",
720                     transform=ax.transAxes,
721                     fontsize=8,
722                     color="white",
723                     va="top",
724                 )
```

```
725
726     ax.set_xlabel(x_col)
727     ax.set_ylabel(y_col)
728     ax.set_title(f"{z_col} over {x_col} / {y_col}", fontsize=10)
729     fig.tight_layout()
730
731
732def build_results_plot(fig, df):
733     """Points + spline curves for the results DataFrame."""
734     p2 = _plan2_eval_columns(df)
735     if p2 is not None:
736         build_plan2_eval_plot(fig, df, *p2)
737         return
738
739     ax = fig.add_subplot(111)
740     num = df.select_dtypes(include=[np.number]).replace([np.inf, -
np.inf], np.nan).dropna()
741     num = num.copy()
742     num.columns = [str(c) for c in num.columns]
743     cols = list(num.columns)
744
745     if num.shape[1] >= 2 and len(num) >= 2:
746         grid = _grid_columns(df)
747         if grid:
748             xcol, gcol, ycol = grid
749             for color, (g, sub) in zip(SERIES_COLORS, num.groupby(gcol, sort=True
)):
750                 xs = sub[xcol].to_numpy(dtype=float)
751                 ys = sub[ycol].to_numpy(dtype=float)
752                 order = np.argsort(xs)
753                 xs, ys = xs[order], ys[order]
754                 ax.plot(xs, ys, "o", color=color, ms=3, alpha=0.7, label=f"{gcol}
 = {g:g}")
755                 xx, yy = _spline_xy(xs, ys)
756                 ax.plot(xx, yy, "-", color=color, lw=1.6)
757             ax.set_xlabel(xcol)
758             ax.set_ylabel(ycol)
759             ax.legend(fontsize=8, loc="best")
760         else:
761             xcol = cols[0]
762             xs_all = num[cols[0]].to_numpy(dtype=float)
763             for color, ycol in zip(SERIES_COLORS, cols[1:]):
764                 ys = num[ycol].to_numpy(dtype=float)
765                 ax.plot(xs_all, ys, "o", color=color, ms=3, alpha=0.7, label=ycol
)
766                 xx, yy = _spline_xy(xs_all, ys)
767                 ax.plot(xx, yy, "-", color=color, lw=1.6)
768             ax.set_xlabel(xcol)
769             ax.legend(fontsize=8, loc="best")
770
771         kind = "cubic spline" if HAVE_SPLINE else "linear interpolation"
772         ax.set_title(f"Results with {kind}", fontsize=10)
773     else:
774         ax.text(0.5, 0.5, "Not enough numeric data to plot.", ha="center", va="ce
nter")
775
776     ax.grid(True, alpha=0.3)
777     ax.set_facecolor("#fbfcfe")
778     fig.tight_layout()
779
780
781def build_heatmap(fig, df, thresholds=(0.8, 0.9)):
782     """Contour-filled heatmap for 3-column grids (x, y, z) with correct axes."""
783     ax = fig.add_subplot(111)
```

```
    grid = _grid_columns(df)
    if grid is None:
        ax.text(0.5, 0.5, "Data is not a 3-
column grid.", ha="center", va="center")
        fig.tight_layout()
        return

    xcol, ycol, zcol = grid
    work = df[[c for c in df.columns if str(c) in grid]].copy()
    work.columns = [str(c) for c in work.columns]
    work = work.apply(pd.to_numeric, errors="coerce").dropna()

    piv = work.pivot_table(index=ycol, columns=xcol, values=zcol, aggfunc="mean")
    piv = piv.sort_index(axis=0).sort_index(axis=1)

    X = piv.columns.to_numpy(dtype=float)
    Y = piv.index.to_numpy(dtype=float)
    Z = np.ma.masked_invalid(piv.to_numpy(dtype=float))  # holes -
> masked, not artifacts

    if Z.count() == 0 or X.size < 2 or Y.size < 2:
        ax.text(0.5, 0.5, "Not enough grid points for a heatmap.", ha="center", v
a="center")
        fig.tight_layout()
        return

    cs = ax.contourf(X, Y, Z, levels=min(24, max(6, X.size + Y.size)), cmap="viri
dis")
    fig.colorbar(cs, ax=ax, label=zcol)

    if X.size <= 12:
        ax.set_xticks(X)
    if Y.size <= 12:
        ax.set_yticks(Y)

    if float(Z.max()) <= 1.01:  # probability-scale surface -
> draw 80/90% contours
        for i, t in enumerate(thresholds):
            if float(Z.min()) <= t <= float(Z.max()):
                ax.contour(X, Y, Z, levels=[t], colors="white", linewidths=1.2)
                ax.text(
                    0.02,
                    0.98 - 0.06 * i,
                    f"white line = {t:.0%}",
                    transform=ax.transAxes,
                    fontsize=8,
                    color="white",
                    va="top",
                )

    ax.set_xlabel(xcol)
    ax.set_ylabel(ycol)
    ax.set_title(f"{zcol} over {xcol} / {ycol}", fontsize=10)
    fig.tight_layout()


# ---------------------------------------------------------------------------
# Form fields (strict column alignment)
# ---------------------------------------------------------------------------
class LabeledField:
    """Label + line-edit pair placed into a SHARED grid so every input sits in
    one strict, perfectly aligned column.  Live validation via a dynamic Qt
    property styled in QSS."""

```

```
    EDIT_WIDTH = 120
    EDIT_HEIGHT = 30

    def __init__(self, layout, row, key, label, default, cast=float, tip=None):
        self.key = key
        self.default = default
        self.cast = cast

        self.label = QLabel(label)
        self.label.setObjectName("fieldlabel")
        self.label.setAlignment(Qt.AlignmentFlag.AlignLeft | Qt.AlignmentFlag.Ali
gnVCenter)

        self.edit = QLineEdit(str(default))
        self.edit.setFixedWidth(self.EDIT_WIDTH)
        self.edit.setFixedHeight(self.EDIT_HEIGHT)
        self.edit.setToolTip(tip or f"{label}\nDefault: {default}")

        layout.addWidget(self.label, row, 0)
        layout.addWidget(self.edit, row, 1, alignment=Qt.AlignmentFlag.AlignLeft)

        self.edit.textChanged.connect(self._validate)
        self._validate()

    def _validate(self, *_):
        raw = self.edit.text().strip()
        ok = bool(raw)
        if ok:
            try:
                self.cast(raw)
            except ValueError:
                ok = False
        self.edit.setProperty("invalid", "false" if ok else "true")
        self.edit.style().unpolish(self.edit)
        self.edit.style().polish(self.edit)

    def reset(self):
        self.edit.setText(str(self.default))

    def get(self, cast=None):
        cast = cast or self.cast
        raw = self.edit.text().strip()
        if raw == "":
            raise ValueError(f"'{self.key}' cannot be empty")
        try:
            return cast(raw)
        except ValueError:
            raise ValueError(f"'{self.key}' must be a number, got {raw!r}")


def build_form(layout, specs, start_row=0, registry=None):
    """Fill a shared QGridLayout; returns dict[key] -> LabeledField."""
    fields = {}
    for i, (key, label, default) in enumerate(specs):
        fields[key] = LabeledField(layout, start_row + i, key, label, default)
    if registry is not None:
        registry.update(fields)
    return fields


class NumItem(QTableWidgetItem):
    """Table item that sorts numerically instead of lexicographically."""

    def __lt__(self, other):
```

```
906         a = self.data(Qt.ItemDataRole.UserRole)
907         b = other.data(Qt.ItemDataRole.UserRole)
908         try:
909             return float(a) < float(b)
910         except Exception:
911             return super().__lt__(other)
912
913
914def fmt_num(v, digits=4):
915     try:
916         if pd.isna(v):
917             return ""
918     except Exception:
919         pass
920     if isinstance(v, (float, np.floating)):
921         return f"{v:,.{digits}f}"
922     return str(v)
923
924
925# ---------------------------------------------------------------------------
926# Background worker
927# ---------------------------------------------------------------------------
928class Worker(QThread):
929     progress = Signal(float, str)
930     finished_ok = Signal(object)
931     failed = Signal(str)
932
933     def __init__(self, job, parent=None):
934         super().__init__(parent)
935         self._job = job
936
937     def run(self):
938         try:
939             result = self._job(self.progress.emit)
940             self.finished_ok.emit(result)
941         except Exception:  # noqa: BLE001
942             self.failed.emit(traceback.format_exc())
943
944
945# ---------------------------------------------------------------------------
946# Plot dialog (modal)
947# ---------------------------------------------------------------------------
948class PlotDialog(QDialog):
949     def __init__(self, df, parent=None, save_base=None):
950         super().__init__(parent)
951         self.setWindowTitle("Results plot")
952         self.resize(880, 640)
953         self.setMinimumSize(520, 380)
954         self.setModal(True)
955         self._df = df
956         self._save_base = save_base or "results"
957
958         self._is_plan2 = _is_plan2_table(df)
959         self._can_heat = self._is_plan2 or _grid_columns(df) is not None
960
961         lay = QVBoxLayout(self)
962         top = QHBoxLayout()
963         top.addWidget(QLabel("Plot style:"))
964
965         self.style_cb = QComboBox()
966         self.style_cb.addItems(["Lines + spline"] + (["Heatmap (grid)"] if self._
can_heat else []))
967         top.addWidget(self.style_cb)
968
```

```
969         # Z-axis selector for Plan 2 tables
970         self.z_cb = None
971         if self._is_plan2:
972             _, _, z_cols = _get_plan2_axes_and_z(df)
973             if len(z_cols) > 1:
974                 top.addWidget(QLabel("Z-axis:"))
975                 self.z_cb = QComboBox()
976                 self.z_cb.addItems([str(c) for c in z_cols])
977                 top.addWidget(self.z_cb)
978                 self.z_cb.currentIndexChanged.connect(self._redraw)
979 
980         top.addStretch(1)
981         save_btn = QPushButton("Save PNG")
982         save_btn.clicked.connect(self._save_png)
983         close_btn = QPushButton("Close")
984         close_btn.clicked.connect(self.accept)
985         top.addWidget(save_btn)
986         top.addWidget(close_btn)
987         lay.addLayout(top)
988 
989         self._fig = Figure(dpi=100)
990         self._canvas = FigureCanvas(self._fig)
991         self._toolbar = NavigationToolbar(self._canvas, self)
992         lay.addWidget(self._toolbar)
993         lay.addWidget(self._canvas, 1)
994 
995         self.style_cb.currentIndexChanged.connect(self._redraw)
996         self._redraw()
997 
998     def _redraw(self):
999         self._fig.clear()
1000         style = self.style_cb.currentText()
1001         if self._is_plan2:
1002             if style == "Heatmap (grid)" and self._can_heat:
1003                 z_col = self.z_cb.currentText() if self.z_cb else None
1004                 build_heatmap_plan2(self._fig, self._df, z_col)
1005             else:
1006                 build_results_plot_plan2(self._fig, self._df)
1007         else:
1008             if style == "Heatmap (grid)" and self._can_heat:
1009                 build_heatmap(self._fig, self._df)
1010             else:
1011                 build_results_plot(self._fig, self._df)
1012         self._canvas.draw()
1013 
1014     def _save_png(self):
1015         path, _ = QFileDialog.getSaveFileName(
1016             self, "Save figure", self._save_base + ".png", "PNG image (*.png)"
1017         )
1018         if path:
1019             self._fig.savefig(path, dpi=150)
1020             QMessageBox.information(self, "Saved", f"Figure saved to {path}")
1021 
1022 
1023# ---------------------------------------------------------------------------
1024# Results panel
1025# ---------------------------------------------------------------------------
1026class ResultsPanel(QFrame):
1027     def __init__(self, parent=None):
1028         super().__init__(parent)
1029         self.setObjectName("panel")
1030         self._df = None
1031         self._prob_col = None
1032 
```

```
1033        lay = QVBoxLayout(self)
1034
1035        toolbar = QHBoxLayout()
1036        title = QLabel("Results")
1037        title.setObjectName("section")
1038
1039        self.row_count = QLabel("")
1040        self.row_count.setObjectName("muted")
1041
1042        self.plot_btn = QPushButton("Plot")
1043        self.plot_btn.setEnabled(False)
1044
1045        self.export_btn = QPushButton("Export CSV")
1046        self.export_btn.setEnabled(False)
1047
1048        self.pdf_btn = QPushButton("Export PDF")
1049        self.pdf_btn.setEnabled(False)
1050
1051        self.oc_btn = QPushButton("OC Curve")
1052        # self.oc_btn.setObjectName("outline")
1053        self.oc_btn.setEnabled(False)
1054        self.oc_btn.setToolTip("OC curve: computed plan vs USP <905> (CU tabs)")
1055
1056        toolbar.addWidget(title)
1057        toolbar.addStretch(1)
1058        toolbar.addWidget(self.row_count)
1059        toolbar.addWidget(self.plot_btn)
1060        toolbar.addWidget(self.oc_btn)
1061        toolbar.addWidget(self.export_btn)
1062        toolbar.addWidget(self.pdf_btn)
1063        lay.addLayout(toolbar)
1064
1065        self.table = QTableWidget()
1066        self.table.setAlternatingRowColors(True)
1067        self.table.setEditTriggers(QAbstractItemView.EditTrigger.NoEditTriggers)
1068        self.table.setSelectionBehavior(QAbstractItemView.SelectionBehavior.Sele
ctRows)
1069        self.table.verticalHeader().setVisible(False)
1070        self.table.setSortingEnabled(True)
1071        self.table.setToolTip("Click a header to sort | Ctrl+C copies selected r
ows")
1072        lay.addWidget(self.table, 1)
1073
1074        self.export_btn.clicked.connect(self._export_csv)
1075        self.plot_btn.clicked.connect(self._show_plot)
1076        self.pdf_btn.clicked.connect(self._export_pdf)
1077        self.report_meta = None
1078        QShortcut(QKeySequence.Copy, self.table, activated=self._copy_selection)
1079
1080    # -- population ------------------------------------------------------------
1081    def clear(self):
1082        self.table.clear()
1083        self.table.setRowCount(0)
1084        self.table.setColumnCount(0)
1085        self._df = None
1086        self._prob_col = None
1087        self.export_btn.setEnabled(False)
1088        self.plot_btn.setEnabled(False)
1089        self.pdf_btn.setEnabled(False)
1090        self.row_count.setText("")
1091
1092    def show_dataframe(self, df: pd.DataFrame):
1093        self.clear()
1094        self._df = df
```

```
        cols = [str(c) for c in df.columns]

        self._prob_col = None
        for c in df.columns:
            if any(s in str(c).lower() for s in ("prob", "pass")):
                ser = pd.to_numeric(df[c], errors="coerce").dropna()
                if len(ser) and ser.max() <= 1.0:
                    self._prob_col = c
                    break

        self.table.setSortingEnabled(False)
        self.table.setColumnCount(len(cols))
        self.table.setHorizontalHeaderLabels(cols)
        self.table.setRowCount(len(df))

        for r, (_, row) in enumerate(df.iterrows()):
            for ci, c in enumerate(df.columns):
                v = row[c]
                item = NumItem(fmt_num(v, 4))
                if isinstance(v, (float, np.floating)):
                    item.setData(Qt.ItemDataRole.UserRole, float(v))
                if self._prob_col is not None and c == self._prob_col:
                    try:
                        pv = float(v)
                        if pv < 0.8:
                            item.setForeground(QBrush(QColor(ERR_RED)))
                        elif pv >= 0.9:
                            item.setForeground(QBrush(QColor(OK_GREEN)))
                    except Exception:
                        pass
                self.table.setItem(r, ci, item)

        self.table.setSortingEnabled(True)
        self.table.resizeColumnsToContents()

        self.export_btn.setEnabled(True)
        can_plot = HAVE_MPL and len(df) >= 2 and df.select_dtypes(include=[np.nu
mber]).shape[1] >= 2
        self.plot_btn.setEnabled(bool(can_plot))
        self.pdf_btn.setEnabled(bool(HAVE_PDF))
        self.row_count.setText(f"{len(df)} rows")

    def show_dict(self, d: dict):
        self.clear()
        self._df = pd.DataFrame([d])
        self.table.setSortingEnabled(False)
        self.table.setColumnCount(2)
        self.table.setHorizontalHeaderLabels(["Field", "Value"])
        self.table.setRowCount(len(d))
        for r, (k, v) in enumerate(d.items()):
            self.table.setItem(r, 0, QTableWidgetItem(str(k)))
            self.table.setItem(r, 1, QTableWidgetItem(fmt_num(v, 6)))
        self.table.resizeColumnsToContents()
        self.export_btn.setEnabled(True)
        self.plot_btn.setEnabled(False)
        self.pdf_btn.setEnabled(bool(HAVE_PDF))
        self.row_count.setText("1 result")

    # -- actions ---------------------------------------------------------
--
    def _copy_selection(self):
        rows = sorted({i.row() for i in self.table.selectedItems()})
        if not rows:
            return
```

```
1157        lines = []
1158        for r in rows:
1159            lines.append(
1160                "\t".join(
1161                    self.table.item(r, c).text() if self.table.item(r, c) else "
"
1162                    for c in range(self.table.columnCount())
1163                )
1164            )
1165        QApplication.clipboard().setText("\n".join(lines))
1166
1167    def _show_plot(self):
1168        if not HAVE_MPL:
1169            QMessageBox.critical(
1170                self,
1171                "Plot unavailable",
1172                "matplotlib is required for plotting.\nInstall it with:  pip ins
tall matplotlib",
1173            )
1174            return
1175        if self._df is None:
1176            return
1177        PlotDialog(self._df, self, save_base=self._default_name(suffix="-
plot")).exec()
1178
1179    def _export_csv(self):
1180        if self._df is None:
1181            return
1182        path, _ = QFileDialog.getSaveFileName(
1183            self, "Export CSV", self._default_name(ext=".csv"), "CSV files (*.cs
v)"
1184        )
1185        if not path:
1186            return
1187        self._df.to_csv(path, index=False, quoting=csv.QUOTE_MINIMAL)
1188        QMessageBox.information(self, "Exported", f"Saved to {path}")
1189
1190    def _export_pdf(self):
1191        if not HAVE_PDF:
1192            QMessageBox.critical(
1193                self,
1194                "PDF export unavailable",
1195                "reportlab is required.\nInstall it with:  pip install reportlab
",
1196            )
1197            return
1198        if self._df is None:
1199            return
1200        path, _ = QFileDialog.getSaveFileName(
1201            self, "Export PDF", self._default_name(ext=".pdf"), "PDF files (*.pd
f)"
1202        )
1203        if not path:
1204            return
1205        meta = self.report_meta or {"title": ["CuDAL RESULTS"]}
1206        try:
1207            write_sas_pdf(path, self._df, meta["title"])
1208            QMessageBox.information(self, "Exported", f"Saved to {path}")
1209        except Exception as exc:  # noqa: BLE001
1210            QMessageBox.critical(self, "PDF export failed", str(exc))
1211
1212    def _default_name(self, suffix="", ext=""):
1213        tab = getattr(self, "tab", None)
1214        try:
```

```
            base = tab._export_base_name() if tab is not None else "results"
        except Exception:
            base = "results"

        return base + suffix + ext


# ---------------------------------------------------------------------------
# Base tab
# ---------------------------------------------------------------------------
class BaseTab(QWidget):
    MODES = [
        ("table", "Acceptance limit table"),
        ("evaluate", "Probability of passing"),
        ("sample", "Sample probability"),
    ]

    def __init__(self, title, subtitle, parent=None):
        super().__init__(parent)
        self._worker = None
        self._table_cache = {}
        self.field_registry = {m: {} for m, _ in self.MODES}

        outer = QVBoxLayout(self)
        outer.setContentsMargins(16, 14, 16, 14)

        hdr = QLabel(title)
        hdr.setObjectName("header")
        sub = QLabel(subtitle)
        sub.setObjectName("subheader")
        outer.addWidget(hdr)
        outer.addWidget(sub)

        body = QHBoxLayout()
        outer.addLayout(body, 1)

        # --- left: controls ------------------------------------------------
        controls = QFrame()
        controls.setObjectName("panel")
        controls.setFixedWidth(360)
        cl = QVBoxLayout(controls)
        cl.setContentsMargins(12, 12, 12, 12)
        cl.setSpacing(10)

        mode_box = QGroupBox("Analysis mode")
        mode_box.setObjectName("card")
        ml = QVBoxLayout(mode_box)
        ml.setContentsMargins(14, 16, 14, 12)
        ml.setSpacing(2)
        self.mode_group = QButtonGroup(self)
        self._mode_buttons = {}
        for val, label in self.MODES:
            rb = QRadioButton(label)
            rb.setProperty("mode", val)
            rb.setFixedHeight(26)
            self.mode_group.addButton(rb)
            self._mode_buttons[val] = rb
            ml.addWidget(rb)
        self._mode_buttons["table"].setChecked(True)
        self.mode_group.buttonClicked.connect(self._switch_mode)
        cl.addWidget(mode_box)

        self.scroll = QScrollArea()
```

```
1278        self.scroll.setWidgetResizable(True)
1279        self.scroll.setFrameShape(QFrame.Shape.NoFrame)
1280        content = QWidget()
1281        content.setStyleSheet("background: transparent;")
1282        self._content_layout = QVBoxLayout(content)
1283        self._content_layout.setContentsMargins(0, 0, 0, 0)
1284        self._content_layout.setSpacing(12)
1285        self.mode_frames = {}
1286        for val, _ in self.MODES:
1287            gb = QGroupBox("Parameters")
1288            gb.setObjectName("card")
1289            g = QGridLayout()
1290            g.setContentsMargins(14, 20, 14, 14)  # strict, uniform card padding
1291            g.setHorizontalSpacing(12)
1292            g.setVerticalSpacing(8)
1293            g.setColumnStretch(0, 1)  # labels fill col 0...
1294            g.setColumnMinimumWidth(1, LabeledField.EDIT_WIDTH)  # inputs align
in col 1
1295            gb.setLayout(g)
1296            self.mode_frames[val] = gb
1297            self._content_layout.addWidget(gb)
1298        self._content_layout.addStretch(1)
1299        self.scroll.setWidget(content)
1300        cl.addWidget(self.scroll, 1)
1301
1302        self._build_mode_frames()
1303
1304        run_row = QHBoxLayout()
1305
1306        self.run_btn = QPushButton("Run")
1307        self.run_btn.setObjectName("accent")
1308        self.run_btn.setToolTip("Run the selected analysis (Ctrl+R)")
1309        self.reset_btn = QPushButton("Reset")
1310        self.progress = QProgressBar()
1311        self.progress.setRange(0, 100)
1312        self.progress.setValue(0)
1313
1314        # uniform run row: progress bar as tall as the Run button
1315        for w in (self.run_btn, self.reset_btn, self.progress):
1316            w.setFixedHeight(34)
1317
1318        run_row.addWidget(self.run_btn)
1319        run_row.addWidget(self.reset_btn)
1320        run_row.addWidget(self.progress, 1)
1321        cl.addLayout(run_row)
1322
1323        self.status_label = QLabel("Ready.")
1324        self.status_label.setObjectName("muted")
1325        self.status_label.setWordWrap(True)
1326        cl.addWidget(self.status_label)
1327
1328        body.addWidget(controls)
1329
1330        # --- right: results -----------------------------------------------
--
1331        self.results = ResultsPanel()
1332        # self.results.tab = self
1333        self.results.tab = self
1334        body.addWidget(self.results, 1)
1335        self.results.oc_btn.clicked.connect(self._show_oc)
1336
1337        self.run_btn.clicked.connect(self._on_run)
1338        self.reset_btn.clicked.connect(self._reset_defaults)
1339        self._switch_mode()
```

```
1340
1341    # -- subclass hooks ------------------------------------------------------
---
1342    def _build_mode_frames(self):
1343        raise NotImplementedError
1344
1345    def _run_table(self):
1346        raise NotImplementedError
1347
1348    def _run_evaluate(self):
1349        raise NotImplementedError
1350
1351    def _run_sample(self):
1352        raise NotImplementedError
1353
1354    @staticmethod
1355    def _cache_key(*args):
1356        norm = []
1357        for a in args:
1358            if isinstance(a, (list, tuple)):
1359                norm.append(tuple(round(float(x), 12) for x in a))
1360            else:
1361                norm.append(a)
1362        return tuple(norm)
1363
1364    # -- settings ------------------------------------------------------------
-----
1365    def collect_state(self):
1366        return {
1367            "mode": self._current_mode(),
1368            "fields": {
1369                mode: {k: f.edit.text() for k, f in reg.items()}
1370                for mode, reg in self.field_registry.items()
1371            },
1372        }
1373
1374    def apply_state(self, state):
1375        if not isinstance(state, dict):
1376            return
1377        for mode, reg in self.field_registry.items():
1378            for k, field in reg.items():
1379                val = state.get("fields", {}).get(mode, {}).get(k)
1380                if val is not None:
1381                    field.edit.setText(str(val))
1382        mode = state.get("mode")
1383        if mode in self._mode_buttons:
1384            self._mode_buttons[mode].setChecked(True)
1385            self._switch_mode()
1386
1387    def _reset_defaults(self):
1388        for reg in self.field_registry.values():
1389            for field in reg.values():
1390                field.reset()
1391        self.status_label.setText("Parameters reset to defaults.")
1392
1393    # -- plumbing ------------------------------------------------------------
-----
1394    def _current_mode(self):
1395        return self.mode_group.checkedButton().property("mode")
1396
1397    def _switch_mode(self, *_):
1398        mode = self._current_mode()
1399        for val, gb in self.mode_frames.items():
1400            gb.setVisible(val == mode)
```

```
1401
1402    def _on_run(self):
1403        if self._worker is not None and self._worker.isRunning():
1404            return
1405        try:
1406            job = {
1407                "table": self._run_table,
1408                "evaluate": self._run_evaluate,
1409                "sample": self._run_sample,
1410            }[self._current_mode()]()
1411        except ValueError as exc:
1412            QMessageBox.critical(self, "Invalid input", str(exc))
1413            return
1414        except Exception as exc:  # noqa: BLE001
1415            QMessageBox.critical(self, "Invalid input", str(exc))
1416            return
1417
1418        self.run_btn.setEnabled(False)
1419        self.progress.setValue(0)
1420        self.status_label.setText("Calculating...")
1421
1422        self._worker = Worker(job, self)
1423        self._worker.progress.connect(self._on_progress)
1424        self._worker.finished_ok.connect(self._on_done)
1425        self._worker.failed.connect(self._on_fail)
1426        self._worker.finished.connect(self._worker_finished)
1427        self._worker.start()
1428
1429    def _on_progress(self, frac, msg):
1430        self.progress.setValue(int(min(100.0, frac * 100)))
1431        if msg:
1432            self.status_label.setText(msg)
1433
1434    def _on_done(self, result):
1435        self.progress.setValue(100)
1436        if isinstance(result, pd.DataFrame):
1437            self.results.show_dataframe(result)
1438
1439            self.status_label.setText(f"Done -- {len(result)} row(s) computed.")
1440        else:
1441            self.results.show_dict(result)
1442            self.status_label.setText("Done.")
1443        self.results.report_meta = self._report_meta()
1444
1445        # in _on_done, after showing the results:
1446        self.results.oc_btn.setEnabled(self._oc_available())
1447
1448    def _on_fail(self, tb):
1449        self.progress.setValue(0)
1450        self.status_label.setText("Calculation failed -- see error dialog.")
1451        QMessageBox.critical(self, "Calculation error", tb)
1452
1453    def _worker_finished(self):
1454        self.run_btn.setEnabled(True)
1455        if self._worker is not None:
1456            self._worker.deleteLater()
1457            self._worker = None
1458
1459    # -- SAS-style PDF title block ------------------------------------------
1460    def _fv(self, *keys):
1461        for d in (getattr(self, "table_fields", {}), getattr(self, "sample_field
s", {})):
1462            for k in keys:
1463                if k in d:
```

```
1464                     try:
1465                         return d[k].get(float)
1466                     except Exception:
1467                         return None
1468         return None
1469
1470     def _report_meta(self):
1471         name = type(self).__name__
1472         dom = "DISSOLUTION" if name.startswith("Disp") else "CONTENT UNIFORMITY"
1473         plan = 2 if name.endswith("2Tab") else 1
1474         n = self._fv("number", "num")
1475         loc = self._fv("loc")
1476         target = self._fv("target")
1477         q = self._fv("q")
1478         lbound = self._fv("lbound")
1479         cilevel = self._fv("cilevel")
1480         lines = []
1481         if plan == 1:
1482             key = f"TARGET = {target:.1f}" if target is not None else f"Q = {q:.
1f}"
1483             lines.append(f"ACCEPTANCE LIMITS FOR {dom}(N= {n:.0f}, {key})")
1484             lines.append("SAMPLING PLAN 1")
1485             lines.append(
1486                 f"(MEETING LIMITS GUARANTEES, WITH {cilevel:.1f}% ASSURANCE, THA
T AT LEAST"
1487             )
1488             lines.append(f"{lbound:.1f}% OF SAMPLES TESTED FOR {dom} WILL PASS T
HE USP TEST)")
1489         else:
1490             lines.append(f"ACCEPTANCE LIMITS FOR {dom}")
1491             lines.append("SAMPLING PLAN 2")
1492             base = f"TARGET={target:.1f}" if target is not None else f"Q={q:.1f}
"
1493             lines.append(f"{base}, LOWER BOUND = {lbound:.1f}, CONFIDENCE LEVEL
= {cilevel:.1f}")
1494             lines.append("TABLE ENTRIES ARE LOWER(LL) AND UPPER(UL) LIMITS ON TH
E MEAN")
1495             if n is not None and loc is not None:
1496                 lines.append(
1497                     f"OF {int(n * loc)} ASSAYS:  {int(n)} ASSAYS AT EACH OF "
1498                     f"{int(loc)} DIFFERENT LOCATIONS"
1499                 )
1500             lines.append("SE IS THE POOLED WITHIN LOCATION STANDARD DEVIATION")
1501             lines.append("STANDARD DEVIATIONS AND MEANS ARE EXPRESSED IN % CLAIM
")
1502         if self._current_mode() != "table":
1503             lines.append(f"MODE: {dict(self.MODES)[self._current_mode()].upper()
}")
1504         return {"title": lines}
1505
1506     def _export_base_name(self):
1507         name = type(self).__name__
1508         dom = "DISSOLUTION" if name.startswith("Disp") else "CONTENT UNIFORMITY"
1509         plan = 2 if name.endswith("2Tab") else 1
1510         method = ("CUSP" if "CONTENT" in dom else "DISP") + str(plan)
1511         v = getattr(self, "table_fields", {})
1512
1513         def g(key):
1514             try:
1515                 return float(v[key].get(float))
1516             except Exception:
1517                 return None
1518
1519         toks = [method]
```

```
        q = g("q")
        if q is not None:
            toks.append(f"Q{q:g}")
        lb, ci = g("lbound"), g("cilevel")
        if lb is not None and ci is not None:
            toks.append(f"{lb:g}x{ci:g}")
        if plan == 1:
            n = g("number")
            if n is not None:
                toks.append(f"{n:g}N")
        else:
            loc, num = g("loc"), g("num")
            if loc is not None and num is not None:
                toks.append(f"{loc:g}Lx{num:g}N")
        base = "-".join(toks)
        mode = self._current_mode()
        if mode == "evaluate":
            base += "-EVAL"
        elif mode == "sample":
            base += "-SAMPLE"
        return base

    # new methods:
    def _oc_available(self):
        return False

    def _oc_context(self):
        return None

    def _show_oc(self):
        if not HAVE_MPL:
            QMessageBox.critical(
                self, "Plot unavailable", "matplotlib is required.\npip install matplotlib"
            )
            return
        ctx = self._oc_context()
        if ctx is None:
            QMessageBox.information(self, "OC curve", "Not available for this scenario.")
            return
        OCDialog(ctx, self, save_base=self._export_base_name() + "-OC").exec()


def make_grid(low: float, high: float, step: float, name: str):
    if step <= 0:
        raise ValueError(f"{name} step must be positive")
    if high < low:
        raise ValueError(f"{name} high must be >= {name} low")
    n = int(round((high - low) / step)) + 1
    return [low + i * step for i in range(n)]


# ---------------------------------------------------------------------------
# Tabs
# ---------------------------------------------------------------------------
class Cusp1Tab(BaseTab):
    def __init__(self, parent=None):
        super().__init__(
            "Content Uniformity - Sampling Plan 1", "Single composite sample (USP <905>).", parent
        )
        self._last_table = None

```

```
    def _build_mode_frames(self):
        self.table_fields = build_form(
            self.mode_frames["table"].layout(),
            [
                ("number", "Number of units (N)", 10),
                ("target", "Target / label claim (%)", 100.0),
                ("lbound", "Lower bound (%)", 95.0),
                ("cilevel", "Confidence level (%)", 95.0),
                ("mean_low", "Mean grid low", 85.1),
                ("mean_high", "Mean grid high", 114.9),
                ("mean_step", "Mean grid step", 0.5),
            ],
            registry=self.field_registry["table"],
        )

        lay = self.mode_frames["evaluate"].layout()
        desc = QLabel("Builds the table above, then evaluates:")
        desc.setObjectName("muted")
        desc.setWordWrap(True)
        lay.addWidget(desc, 0, 0, 1, 2)
        self.eval_fields = build_form(
            lay,
            [
                ("u_low", "True mean U -- low", 95.0),
                ("u_high", "True mean U -- high", 105.0),
                ("u_step", "True mean U -- step", 2.5),
                ("cv_low", "True CV(%) -- low", 1.0),
                ("cv_high", "True CV(%) -- high", 4.0),
                ("cv_step", "True CV(%) -- step", 1.0),
            ],
            start_row=1,
            registry=self.field_registry["evaluate"],
        )
        for k in ("number", "target", "lbound", "cilevel"):
            self.eval_fields[k] = self.table_fields[k]

        self.sample_fields = build_form(
            self.mode_frames["sample"].layout(),
            [
                ("mean", "Sample mean (%)", 100.0),
                ("cv", "Sample CV (%)", 2.0),
                ("number", "Number of units (N)", 10),
                ("target", "Target / label claim (%)", 100.0),
                ("lbound", "Lower bound (%)", 95.0),
                ("cilevel", "Confidence level (%)", 95.0),
            ],
            registry=self.field_registry["sample"],
        )

    def _run_table(self):
        v = self.table_fields
        number = v["number"].get(int)
        target = v["target"].get(float)
        lbound = v["lbound"].get(float)
        cilevel = v["cilevel"].get(float)
        mean_low = v["mean_low"].get(float)
        mean_high = v["mean_high"].get(float)
        mean_step = v["mean_step"].get(float)
        key = self._cache_key(number, target, lbound, cilevel, mean_low, mean_hi
gh, mean_step)

        def job(progress):
            hit = self._table_cache.get(key)
            if hit is not None:
```

```
1644                 progress(0.5, "Using cached table...")
1645                 return hit
1646             progress(0.2, "Computing acceptance table...")
1647             table = cusp1.acceptance_limit_table(
1648                 number, target, lbound, cilevel, mean_low, mean_high, mean_step
1649             )
1650             self._table_cache[key] = table
1651             self._last_table = table
1652             progress(1.0, "Table complete.")
1653             return table
1654 
1655         return job
1656 
1657     def _run_evaluate(self):
1658         v = self.table_fields
1659         number = v["number"].get(int)
1660         target = v["target"].get(float)
1661         lbound = v["lbound"].get(float)
1662         cilevel = v["cilevel"].get(float)
1663         u_vals = make_grid(
1664             self.eval_fields["u_low"].get(float),
1665             self.eval_fields["u_high"].get(float),
1666             self.eval_fields["u_step"].get(float),
1667             "U",
1668         )
1669         cv_vals = make_grid(
1670             self.eval_fields["cv_low"].get(float),
1671             self.eval_fields["cv_high"].get(float),
1672             self.eval_fields["cv_step"].get(float),
1673             "CV",
1674         )
1675         key = self._cache_key(number, target, lbound, cilevel)
1676 
1677         def job(progress):
1678             table = self._table_cache.get(key)
1679             if table is None:
1680                 progress(0.2, "Building acceptance table...")
1681                 table = cusp1.acceptance_limit_table(number, target, lbound, cil
evel)
1682                 self._table_cache[key] = table
1683                 self._last_table = table
1684             else:
1685                 progress(0.2, "Using cached table...")
1686             progress(0.6, "Evaluating probability grid...")
1687             return cusp1.probability_of_passing(table, number, u_vals, cv_vals)
1688 
1689         return job
1690 
1691     def _run_sample(self):
1692         v = self.sample_fields
1693         mean = v["mean"].get(float)
1694         cv = v["cv"].get(float)
1695         number = v["number"].get(int)
1696         target = v["target"].get(float)
1697         lbound = v["lbound"].get(float)
1698         cilevel = v["cilevel"].get(float)
1699 
1700         def job(progress):
1701             progress(0.4, "Computing sample probability...")
1702             return cusp1.sample_probability(mean, cv, number, target, lbound, ci
level)
1703 
1704         return job
1705 
```

```
    def _oc_available(self):
        return True

    def _oc_context(self):
        v = self.table_fields
        number = v["number"].get(int)
        target = v["target"].get(float)
        lbound = v["lbound"].get(float)
        cilevel = v["cilevel"].get(float)
        table = (
            self._last_table
            if self._last_table is not None
            else cusp1.acceptance_limit_table(number, target, lbound, cilevel)
        )

        def computed(xk, xs, fx):
            if xk == "cv":
                res = cusp1.probability_of_passing(table, number, [fx["U"]], [fl
oat(x) for x in xs])
            else:
                res = cusp1.probability_of_passing(
                    table, number, [float(x) for x in xs], [fx["CV"]]
                )
            return _prob_series(res)

        def make_units(xk, x, fx, rng, reps):
            U = x if xk == "u" else fx["U"]
            CV = x if xk == "cv" else fx["CV"]
            return rng.normal(U, U * CV / 100.0, (reps, 30))

        return make_oc_context(
            "cu",
            target,
            computed,
            make_units,
            [("cv", "True CV (%)  [U fixed]"), ("u", "True mean U (%)  [CV fixed
]")],
            {"cv": (0.5, 10.0, 0.25), "u": (85.0, 115.0, 1.0)},
            {"cv": [("U", target)], "u": [("CV", 2.0)]},
        )


class Cusp2Tab(BaseTab):
    def __init__(self, parent=None):
        super().__init__(
            "Content Uniformity -- Sampling Plan 2",
            "Multiple locations, within/between-
location variance components (USP <905>).",
            parent,
        )
        self._last_table = None

    def _build_mode_frames(self):
        self.table_fields = build_form(
            self.mode_frames["table"].layout(),
            [
                ("num", "Units per location", 6),
                ("loc", "Number of locations", 10),
                ("target", "Target / label claim (%)", 100.0),
                ("lbound", "Lower bound (%)", 95.0),
                ("cilevel", "Confidence level (%)", 95.0),
                ("se_low", "Within-loc SD -- low", 0.5),
                ("se_high", "Within-loc SD -- high", 4.0),
                ("se_step", "Within-loc SD -- step", 0.5),
```

```
                ("sm_low", "Between-loc SD -- low", 0.5),
                ("sm_high", "Between-loc SD -- high", 4.0),
                ("sm_step", "Between-loc SD -- step", 0.5),
            ],
            registry=self.field_registry["table"],
        )

        lay = self.mode_frames["evaluate"].layout()
        desc = QLabel("Builds the table above, then evaluates:")
        desc.setObjectName("muted")
        desc.setWordWrap(True)
        lay.addWidget(desc, 0, 0, 1, 2)
        self.eval_fields = build_form(
            lay,
            [
                ("u_low", "True mean U -- low", 95.0),
                ("u_high", "True mean U -- high", 105.0),
                ("u_step", "True mean U -- step", 2.5),
                ("sigse_low", "True within-loc SD -- low", 1.0),
                ("sigse_high", "True within-loc SD -- high", 3.0),
                ("sigse_step", "True within-loc SD -- step", 1.0),
                ("sigsm_low", "True between-loc SD -- low", 1.0),
                ("sigsm_high", "True between-loc SD -- high", 3.0),
                ("sigsm_step", "True between-loc SD -- step", 1.0),
            ],
            start_row=1,
            registry=self.field_registry["evaluate"],
        )

        self.sample_fields = build_form(
            self.mode_frames["sample"].layout(),
            [
                ("mean", "Sample mean (%)", 100.0),
                ("se", "Sample within-loc SD", 2.2),
                ("sm", "Sample between-loc SD", 2.46),
                ("num", "Units per location", 6),
                ("loc", "Number of locations", 10),
                ("target", "Target / label claim (%)", 100.0),
                ("cilevel", "Confidence level (%)", 95.0),
            ],
            registry=self.field_registry["sample"],
        )

    def _table_args(self):
        v = self.table_fields
        num = v["num"].get(int)
        loc = v["loc"].get(int)
        target = v["target"].get(float)
        lbound = v["lbound"].get(float)
        cilevel = v["cilevel"].get(float)
        se_vals = make_grid(
            v["se_low"].get(float), v["se_high"].get(float), v["se_step"].get(fl
oat), "SE"
        )
        sm_vals = make_grid(
            v["sm_low"].get(float), v["sm_high"].get(float), v["sm_step"].get(fl
oat), "SM"
        )
        return num, loc, target, lbound, cilevel, se_vals, sm_vals

    def _run_table(self):
        num, loc, target, lbound, cilevel, se_vals, sm_vals = self._table_args()
        key = self._cache_key(num, loc, target, lbound, cilevel, se_vals, sm_val
s)
```

```
1828         self._last_table = None
1829 
1830         def job(progress):
1831             hit = self._table_cache.get(key)
1832             if hit is not None:
1833                 progress(0.5, "Using cached table...")
1834                 return hit
1835             progress(0.2, "Computing acceptance table (Plan 2)...")
1836             table = cusp2.acceptance_limit_table(
1837                 num, loc, target, lbound, cilevel, se_vals, sm_vals
1838             )
1839             self._table_cache[key] = table
1840             self._last_table = table
1841             progress(1.0, "Table complete.")
1842             return table
1843 
1844         return job
1845 
1846     def _run_evaluate(self):
1847         num, loc, target, lbound, cilevel, se_vals, sm_vals = self._table_args()
1848         u_vals = make_grid(
1849             self.eval_fields["u_low"].get(float),
1850             self.eval_fields["u_high"].get(float),
1851             self.eval_fields["u_step"].get(float),
1852             "U",
1853         )
1854         sigse_vals = make_grid(
1855             self.eval_fields["sigse_low"].get(float),
1856             self.eval_fields["sigse_high"].get(float),
1857             self.eval_fields["sigse_step"].get(float),
1858             "within-loc SD",
1859         )
1860         sigsm_vals = make_grid(
1861             self.eval_fields["sigsm_low"].get(float),
1862             self.eval_fields["sigsm_high"].get(float),
1863             self.eval_fields["sigsm_step"].get(float),
1864             "between-loc SD",
1865         )
1866         key = self._cache_key(num, loc, target, lbound, cilevel, se_vals, sm_val
s)
1867 
1868         def job(progress):
1869             table = self._table_cache.get(key)
1870             if table is None:
1871                 progress(0.2, "Building acceptance table (Plan 2)...")
1872                 table = cusp2.acceptance_limit_table(
1873                     num, loc, target, lbound, cilevel, se_vals, sm_vals
1874                 )
1875                 self._last_table = table
1876                 self._table_cache[key] = table
1877                 self._last_table = table
1878             else:
1879                 progress(0.2, "Using cached table...")
1880             d1 = se_vals[1] - se_vals[0] if len(se_vals) > 1 else 0.1
1881             progress(0.6, "Evaluating probability grid...")
1882             return cusp2.probability_of_passing(table, num, loc, d1, u_vals, sig
se_vals, sigsm_vals)
1883 
1884         return job
1885 
1886     def _run_sample(self):
1887         v = self.sample_fields
1888         mean = v["mean"].get(float)
1889         se = v["se"].get(float)
```

```
1890         sm = v["sm"].get(float)
1891         num = v["num"].get(int)
1892         loc = v["loc"].get(int)
1893         target = v["target"].get(float)
1894         cilevel = v["cilevel"].get(float)
1895 
1896         def job(progress):
1897             progress(0.4, "Computing sample probability...")
1898             return cusp2.sample_probability(mean, se, sm, num, loc, target, cile
vel)
1899 
1900         return job
1901 
1902     def _oc_available(self):
1903         return True
1904 
1905     def _oc_context(self):
1906         v = self.table_fields
1907         num, loc = v["num"].get(int), v["loc"].get(int)
1908         target = v["target"].get(float)
1909         lbound = v["lbound"].get(float)
1910         cilevel = v["cilevel"].get(float)
1911         se_vals = make_grid(
1912             v["se_low"].get(float), v["se_high"].get(float), v["se_step"].get(fl
oat), "SE"
1913         )
1914         sm_vals = make_grid(
1915             v["sm_low"].get(float), v["sm_high"].get(float), v["sm_step"].get(fl
oat), "SM"
1916         )
1917         d1 = se_vals[1] - se_vals[0] if len(se_vals) > 1 else 0.1
1918         table = (
1919             self._last_table
1920             if self._last_table is not None
1921             else cusp2.acceptance_limit_table(num, loc, target, lbound, cilevel,
 se_vals, sm_vals)
1922         )
1923 
1924         def computed(xk, xs, fx):
1925             U = [float(x) for x in xs] if xk == "u" else [fx["U"]]
1926             SE = [float(x) for x in xs] if xk == "se" else [fx["SE"]]
1927             return _prob_series(
1928                 cusp2.probability_of_passing(table, num, loc, d1, U, SE, [fx["SM
"]])
1929             )
1930 
1931         def make_units(xk, x, fx, rng, reps):
1932             U = x if xk == "u" else fx["U"]
1933             SE = x if xk == "se" else fx["SE"]
1934             return U + rng.normal(0.0, fx["SM"], (reps, 1)) + rng.normal(0.0, SE
, (reps, 30))
1935 
1936         return make_oc_context(
1937             "cu",
1938             target,
1939             computed,
1940             make_units,
1941             [("se", "True within-
loc SD  [U, SM fixed]"), ("u", "True mean U  [SE, SM fixed]")],
1942             {"se": (0.5, 10.0, 0.25), "u": (85.0, 115.0, 1.0)},
1943             {"se": [("U", target), ("SM", 2.2)], "u": [("SE", 2.2), ("SM", 2.2)]
},
1944         )
1945 
```

```
1946
1947class Disp1Tab(BaseTab):
1948    def __init__(self, parent=None):
1949        super().__init__("Dissolution -
- Sampling Plan 1", "Single location (USP <711>).", parent)
1950        self._last_table = None
1951
1952    def _build_mode_frames(self):
1953        self.table_fields = build_form(
1954            self.mode_frames["table"].layout(),
1955            [
1956                ("number", "Number of units (N)", 6),
1957                ("q", "Q value (%)", 80.0),
1958                ("lbound", "Lower bound (%)", 95.0),
1959                ("cilevel", "Confidence level (%)", 95.0),
1960                ("meanadj_step", "Mean grid step", 1.0),
1961            ],
1962            registry=self.field_registry["table"],
1963        )
1964
1965        lay = self.mode_frames["evaluate"].layout()
1966        desc = QLabel("Builds the table above, then evaluates:")
1967        desc.setObjectName("muted")
1968        desc.setWordWrap(True)
1969        lay.addWidget(desc, 0, 0, 1, 2)
1970        self.eval_fields = build_form(
1971            lay,
1972            [
1973                ("u_low", "True mean U -- low", 90.0),
1974                ("u_high", "True mean U -- high", 100.0),
1975                ("u_step", "True mean U -- step", 2.5),
1976                ("cv_low", "True CV(%) -- low", 1.0),
1977                ("cv_high", "True CV(%) -- high", 4.0),
1978                ("cv_step", "True CV(%) -- step", 1.0),
1979            ],
1980            start_row=1,
1981            registry=self.field_registry["evaluate"],
1982        )
1983
1984        self.sample_fields = build_form(
1985            self.mode_frames["sample"].layout(),
1986            [
1987                ("mean", "Sample mean (%)", 90.0),
1988                ("cv", "Sample CV (%)", 3.0),
1989                ("number", "Number of units (N)", 6),
1990                ("q", "Q value (%)", 80.0),
1991                ("cilevel", "Confidence level (%)", 95.0),
1992            ],
1993            registry=self.field_registry["sample"],
1994        )
1995
1996    def _run_table(self):
1997        v = self.table_fields
1998        number = v["number"].get(int)
1999        q = v["q"].get(float)
2000        lbound = v["lbound"].get(float)
2001        cilevel = v["cilevel"].get(float)
2002        meanadj_step = v["meanadj_step"].get(float)
2003        key = self._cache_key(number, q, lbound, cilevel, meanadj_step)
2004
2005        def job(progress):
2006            hit = self._table_cache.get(key)
2007            if hit is not None:
2008                progress(0.5, "Using cached table...")
```

```
2009                 return hit
2010             progress(0.2, "Computing acceptance table...")
2011             table = disp1.acceptance_limit_table(number, q, lbound, cilevel, mea
nadj_step)
2012             self._table_cache[key] = table
2013             self._last_table = table
2014             progress(1.0, "Table complete.")
2015             return table
2016
2017         return job
2018
2019     def _run_evaluate(self):
2020         v = self.table_fields
2021         number = v["number"].get(int)
2022         q = v["q"].get(float)
2023         lbound = v["lbound"].get(float)
2024         cilevel = v["cilevel"].get(float)
2025         u_vals = make_grid(
2026             self.eval_fields["u_low"].get(float),
2027             self.eval_fields["u_high"].get(float),
2028             self.eval_fields["u_step"].get(float),
2029             "U",
2030         )
2031         cv_vals = make_grid(
2032             self.eval_fields["cv_low"].get(float),
2033             self.eval_fields["cv_high"].get(float),
2034             self.eval_fields["cv_step"].get(float),
2035             "CV",
2036         )
2037         key = self._cache_key(number, q, lbound, cilevel)
2038
2039         def job(progress):
2040             table = self._table_cache.get(key)
2041             if table is None:
2042                 progress(0.2, "Building acceptance table...")
2043                 table = disp1.acceptance_limit_table(number, q, lbound, cilevel)
2044                 self._table_cache[key] = table
2045                 self._last_table = table
2046             else:
2047                 progress(0.2, "Using cached table...")
2048             progress(0.6, "Evaluating probability grid...")
2049             return disp1.probability_of_passing(table, number, u_vals, cv_vals)
2050
2051         return job
2052
2053     def _run_sample(self):
2054         v = self.sample_fields
2055         mean = v["mean"].get(float)
2056         cv = v["cv"].get(float)
2057         number = v["number"].get(int)
2058         q = v["q"].get(float)
2059         cilevel = v["cilevel"].get(float)
2060
2061         def job(progress):
2062             progress(0.4, "Computing sample probability...")
2063             return disp1.sample_probability(mean, cv, number, q, cilevel)
2064
2065         return job
2066
2067     def _oc_available(self):
2068         return True
2069
2070     def _oc_context(self):
2071         v = self.table_fields
```

```
        number = v["number"].get(int)
        q = v["q"].get(float)
        lbound = v["lbound"].get(float)
        cilevel = v["cilevel"].get(float)
        table = (
            self._last_table
            if self._last_table is not None
            else disp1.acceptance_limit_table(number, q, lbound, cilevel)
        )

        def computed(xk, xs, fx):
            if xk == "cv":
                res = disp1.probability_of_passing(table, number, [fx["U"]], [fl
oat(x) for x in xs])
            else:
                res = disp1.probability_of_passing(
                    table, number, [float(x) for x in xs], [fx["CV"]]
                )
            return _prob_series(res)

        def make_units(xk, x, fx, rng, reps):
            U = x if xk == "u" else fx["U"]
            CV = x if xk == "cv" else fx["CV"]
            return rng.normal(U, U * CV / 100.0, (reps, 24))

        return make_oc_context(
            "disp",
            q,
            computed,
            make_units,
            [("cv", "True CV (%)  [U fixed]"), ("u", "True mean U (%)  [CV fixed
]")],
            {"cv": (0.5, 25.0, 0.5), "u": (70.0, 120.0, 1.0)},
            {"cv": [("U", 100.0)], "u": [("CV", 3.0)]},
        )


class Disp2Tab(BaseTab):
    def __init__(self, parent=None):
        super().__init__(
            "Dissolution -- Sampling Plan 2",
            "Multiple locations, within/between-
location variance components (USP <711>).",
            parent,
        )
        self._last_table = None

    def _build_mode_frames(self):
        self.table_fields = build_form(
            self.mode_frames["table"].layout(),
            [
                ("num", "Units per location", 6),
                ("loc", "Number of locations", 5),
                ("q", "Q value (%)", 80.0),
                ("lbound", "Lower bound (%)", 95.0),
                ("cilevel", "Confidence level (%)", 95.0),
                ("se_low", "Within-loc SD -- low", 1.0),
                ("se_high", "Within-loc SD -- high", 5.0),
                ("se_step", "Within-loc SD -- step", 1.0),
                ("sm_low", "Between-loc SD -- low", 1.0),
                ("sm_high", "Between-loc SD -- high", 5.0),
                ("sm_step", "Between-loc SD -- step", 1.0),
            ],
            registry=self.field_registry["table"],
```

```
2133         )
2134
2135         lay = self.mode_frames["evaluate"].layout()
2136         desc = QLabel("Builds the table above, then evaluates:")
2137         desc.setObjectName("muted")
2138         desc.setWordWrap(True)
2139         lay.addWidget(desc, 0, 0, 1, 2)
2140         self.eval_fields = build_form(
2141             lay,
2142             [
2143                 ("u_low", "True mean U -- low", 90.0),
2144                 ("u_high", "True mean U -- high", 100.0),
2145                 ("u_step", "True mean U -- step", 2.5),
2146                 ("sigse_low", "True within-loc SD -- low", 1.0),
2147                 ("sigse_high", "True within-loc SD -- high", 3.0),
2148                 ("sigse_step", "True within-loc SD -- step", 1.0),
2149                 ("sigsm_low", "True between-loc SD -- low", 1.0),
2150                 ("sigsm_high", "True between-loc SD -- high", 3.0),
2151                 ("sigsm_step", "True between-loc SD -- step", 1.0),
2152             ],
2153             start_row=1,
2154             registry=self.field_registry["evaluate"],
2155         )
2156
2157         self.sample_fields = build_form(
2158             self.mode_frames["sample"].layout(),
2159             [
2160                 ("mean", "Sample mean (%)", 90.0),
2161                 ("se", "Sample within-loc SD", 2.2),
2162                 ("sm", "Sample between-loc SD", 2.46),
2163                 ("num", "Units per location", 6),
2164                 ("loc", "Number of locations", 5),
2165                 ("q", "Q value (%)", 80.0),
2166                 ("cilevel", "Confidence level (%)", 95.0),
2167             ],
2168             registry=self.field_registry["sample"],
2169         )
2170
2171     def _table_args(self):
2172         v = self.table_fields
2173         num = v["num"].get(int)
2174         loc = v["loc"].get(int)
2175         q = v["q"].get(float)
2176         lbound = v["lbound"].get(float)
2177         cilevel = v["cilevel"].get(float)
2178         se_vals = make_grid(
2179             v["se_low"].get(float), v["se_high"].get(float), v["se_step"].get(fl
oat), "SE"
2180         )
2181         sm_vals = make_grid(
2182             v["sm_low"].get(float), v["sm_high"].get(float), v["sm_step"].get(fl
oat), "SM"
2183         )
2184         return num, loc, q, lbound, cilevel, se_vals, sm_vals
2185
2186     def _run_table(self):
2187         num, loc, q, lbound, cilevel, se_vals, sm_vals = self._table_args()
2188         key = self._cache_key(num, loc, q, lbound, cilevel, se_vals, sm_vals)
2189
2190         def job(progress):
2191             hit = self._table_cache.get(key)
2192             if hit is not None:
2193                 progress(0.5, "Using cached table...")
2194                 return hit
```

```
2195            progress(0.2, "Computing acceptance table (Plan 2)...")
2196            table = disp2.acceptance_limit_table(num, loc, q, lbound, cilevel, s
e_vals, sm_vals)
2197            self._last_table = table
2198            self._table_cache[key] = table
2199            progress(1.0, "Table complete.")
2200            return table
2201
2202        return job
2203
2204    def _run_evaluate(self):
2205        num, loc, q, lbound, cilevel, se_vals, sm_vals = self._table_args()
2206        dse = se_vals[1] - se_vals[0] if len(se_vals) > 1 else 1.0
2207        dsm = sm_vals[1] - sm_vals[0] if len(sm_vals) > 1 else 1.0
2208        u_vals = make_grid(
2209            self.eval_fields["u_low"].get(float),
2210            self.eval_fields["u_high"].get(float),
2211            self.eval_fields["u_step"].get(float),
2212            "U",
2213        )
2214        sigse_vals = make_grid(
2215            self.eval_fields["sigse_low"].get(float),
2216            self.eval_fields["sigse_high"].get(float),
2217            self.eval_fields["sigse_step"].get(float),
2218            "within-loc SD",
2219        )
2220        sigsm_vals = make_grid(
2221            self.eval_fields["sigsm_low"].get(float),
2222            self.eval_fields["sigsm_high"].get(float),
2223            self.eval_fields["sigsm_step"].get(float),
2224            "between-loc SD",
2225        )
2226        key = self._cache_key(num, loc, q, lbound, cilevel, se_vals, sm_vals)
2227
2228        def job(progress):
2229            table = self._table_cache.get(key)
2230            if table is None:
2231                progress(0.2, "Building acceptance table (Plan 2)...")
2232                table = disp2.acceptance_limit_table(num, loc, q, lbound, cileve
l, se_vals, sm_vals)
2233                self._table_cache[key] = table
2234                self._last_table = table
2235            else:
2236                progress(0.2, "Using cached table...")
2237            progress(0.6, "Evaluating probability grid...")
2238            return disp2.probability_of_passing(
2239                table, num, loc, dse, dsm, u_vals, sigse_vals, sigsm_vals
2240            )
2241
2242        return job
2243
2244    def _run_sample(self):
2245        v = self.sample_fields
2246        mean = v["mean"].get(float)
2247        se = v["se"].get(float)
2248        sm = v["sm"].get(float)
2249        num = v["num"].get(int)
2250        loc = v["loc"].get(int)
2251        q = v["q"].get(float)
2252        cilevel = v["cilevel"].get(float)
2253
2254        def job(progress):
2255            progress(0.4, "Computing sample probability...")
2256            return disp2.sample_probability(mean, se, sm, num, loc, q, cilevel)
```

```
2257
2258        return job
2259
2260    def _oc_available(self):
2261        return True
2262
2263    def _oc_context(self):
2264        v = self.table_fields
2265        num, loc = v["num"].get(int), v["loc"].get(int)
2266        q = v["q"].get(float)
2267        lbound = v["lbound"].get(float)
2268        cilevel = v["cilevel"].get(float)
2269        se_vals = make_grid(
2270            v["se_low"].get(float), v["se_high"].get(float), v["se_step"].get(fl
oat), "SE"
2271        )
2272        sm_vals = make_grid(
2273            v["sm_low"].get(float), v["sm_high"].get(float), v["sm_step"].get(fl
oat), "SM"
2274        )
2275        dse = se_vals[1] - se_vals[0] if len(se_vals) > 1 else 1.0
2276        dsm = sm_vals[1] - sm_vals[0] if len(sm_vals) > 1 else 1.0
2277        table = (
2278            self._last_table
2279            if self._last_table is not None
2280            else disp2.acceptance_limit_table(num, loc, q, lbound, cilevel, se_v
als, sm_vals)
2281        )
2282
2283        def computed(xk, xs, fx):
2284            U = [float(x) for x in xs] if xk == "u" else [fx["U"]]
2285            SE = [float(x) for x in xs] if xk == "se" else [fx["SE"]]
2286            return _prob_series(
2287                disp2.probability_of_passing(table, num, loc, dse, dsm, U, SE, [
fx["SM"]])
2288            )
2289
2290        def make_units(xk, x, fx, rng, reps):
2291            U = x if xk == "u" else fx["U"]
2292            SE = x if xk == "se" else fx["SE"]
2293            return U + rng.normal(0.0, fx["SM"], (reps, 1)) + rng.normal(0.0, SE
, (reps, 24))
2294
2295        return make_oc_context(
2296            "disp",
2297            q,
2298            computed,
2299            make_units,
2300            [("se", "True within-
loc SD  [U, SM fixed]"), ("u", "True mean U  [SE, SM fixed]")],
2301            {"se": (0.5, 25.0, 0.5), "u": (70.0, 120.0, 2.0)},
2302            {"se": [("U", 100.0), ("SM", 2.2)], "u": [("SE", 2.2), ("SM", 2.2)]}
,
2303        )
2304
2305
2306# ---------------------------------------------------------------------------
2307# OC-curve engine (unified): Monte-Carlo probability of passing the
2308# compendial test itself -- USP <905> (2 stages) or USP <711> (3 stages).
2309# ---------------------------------------------------------------------------
2310def _oc_cu_pass(units: np.ndarray, target: float) -> float:
2311    """Two-stage USP <905> decision. units: (reps, 30) -> P(pass)."""
2312    """
2313    This checks for an absolute shift of 25.0 units rather than 25% of $M$.
```

```
    - If $M = 98.5$, the lower bound should be $73.875$ ($98.5 \times 0.75$), meaning a deviation of at most $24.625$.
    - code allows a deviation up to $25.0$ (down to $73.5$), falsely passing extreme outliers.
    - If $M = 101.5$, the upper bound should be $126.875$, meaning a deviation up to $25.375$.
    - code caps it strictly at $25.0$ ($126.5$), falsely failing valid units.
    """
    passed = np.zeros(units.shape[0], dtype=bool)
    hi = target if target > 101.5 else 101.5

    def M(m):
        return np.where(m <= 100.0, np.maximum(98.5, m), np.minimum(hi, m))

    # --- Stage 1 (10 units) ---
    x1 = units[:, :10]
    m1, s1 = x1.mean(axis=1), x1.std(axis=1, ddof=1)

    p1 = (np.abs(M(m1) - m1) + 2.4 * s1) <= 15.0
    passed |= p1

    # --- Stage 2 (30 units) ---
    live = np.where(~p1)[0]
    if live.size:
        x30 = units[live]
        m2, s2 = x30.mean(axis=1), x30.std(axis=1, ddof=1)
        M2 = M(m2)

        av_ok = (np.abs(M2 - m2) + 2.0 * s2) <= 15.0
        # FIXED: 0.25 * M2 instead of hardcoded 25.0
        within_ok = np.abs(x30 - M2[:, None]).max(axis=1) <= (0.25 * M2)

        passed[live[av_ok & within_ok]] = True

    return float(passed.mean())


def _oc_disp_pass(units, q):
    """Three-stage USP <711> decision. units: (reps, 24) -> P(pass)."""
    passed = np.zeros(units.shape[0], dtype=bool)

    x6 = units[:, :6]  # Stage 1: all >= Q+5
    p = np.all(x6 >= q + 5.0, axis=1)
    passed |= p
    live = np.where(~p)[0]

    if live.size:  # Stage 2: 12 units
        x12 = units[live][:, :12]
        # FIXED: Changed > to >= for Q-15 boundary
        p = (x12.mean(axis=1) >= q) & np.all(x12 >= q - 15.0, axis=1)
        passed[live[p]] = True
        live = live[~p]

    if live.size:  # Stage 3: 24 units
        x24 = units[live][:, :24]
        ok_mean = x24.mean(axis=1) >= q
        # FIXED: Changed <= to < to match "less than Q-15%"
        n_l15 = (x24 < q - 15.0).sum(axis=1)
        # FIXED: Changed <= to < to match "less than Q-25%"
        any_l25 = (x24 < q - 25.0).any(axis=1)

        p = ok_mean & (n_l15 <= 2) & ~any_l25
        passed[live[p]] = True

```

```
    return float(passed.mean())


def _prob_series(df):
    for c in df.columns:
        if any(s in str(c).lower() for s in ("prob", "pass", "ptrap")):
            return pd.to_numeric(df[c], errors="coerce").to_numpy(dtype=float)
    return pd.to_numeric(df.iloc[:, -1], errors="coerce").to_numpy(dtype=float)


def make_oc_context(test, ref, computed, make_units, x_choices, grid, fixed_specs):
    """Single factory for every tab's OC context.

    test       : "cu"  -> USP <905> two-stage decision,  ref = target
                 "disp"-> USP <711> three-stage decision, ref = Q
    computed   : fn(xk, xs, fx) -> analytic P(pass table)   (per-tab)
    make_units : fn(xk, x, fx, rng, reps) -> (reps, n_units) simulated units
    """
    decision = _oc_cu_pass if test == "cu" else _oc_disp_pass

    def usp(xk, xs, fx, reps):
        out = []
        for x in xs:
            rng = np.random.default_rng(12345)  # seeded -> reproducible
            out.append(decision(make_units(xk, float(x), fx, rng, reps), ref))
        return np.array(out)

    return {
        "test": test,
        "x_choices": x_choices,
        "grid": grid,
        "fixed_specs": fixed_specs,
        "computed": computed,
        "usp": usp,
    }


class OCDialog(QDialog):
    """OC curve: computed acceptance-limit plan vs the USP <905> test itself."""

    def __init__(self, ctx, parent=None, save_base=None):
        super().__init__(parent)
        self._ctx = ctx
        self._usp_label = (
            "USP <905> two-stage test (Monte Carlo)"
            if ctx["test"] == "cu"
            else "USP <711> three-stage test (Monte Carlo)"
        )
        self._title = "OC Curve -- computed plan vs " + (
            "USP <905>" if ctx["test"] == "cu" else "USP <711>"
        )
        self.setWindowTitle(self._title)
        self.resize(900, 640)
        self.setModal(True)
        self._save_base = save_base or "oc"

        lay = QVBoxLayout(self)
        top = QGridLayout()
        top.addWidget(QLabel("X axis:"), 0, 0)
        self.x_cb = QComboBox()
        for key, label in ctx["x_choices"]:
            self.x_cb.addItem(label, key)
        top.addWidget(self.x_cb, 0, 1)
```

```
2438        top.addWidget(QLabel("low/high/step:"), 0, 2)
2439        self.g_lo, self.g_hi, self.g_st = (QLineEdit() for _ in range(3))
2440        for w in (self.g_lo, self.g_hi, self.g_st):
2441            w.setFixedWidth(70)
2442        row = QHBoxLayout()
2443        row.addWidget(self.g_lo)
2444        row.addWidget(self.g_hi)
2445        row.addWidget(self.g_st)
2446        top.addLayout(row, 0, 3)
2447        top.addWidget(QLabel("MC reps:"), 0, 4)
2448        self.rep_ed = QLineEdit("2000")
2449        self.rep_ed.setFixedWidth(70)
2450        top.addWidget(self.rep_ed, 0, 5)
2451
2452        self.fixed_box = QWidget()
2453        self.fixed_layout = QHBoxLayout(self.fixed_box)
2454        self.fixed_layout.setContentsMargins(0, 0, 0, 0)
2455        redraw = QPushButton("Redraw")
2456        redraw.setObjectName("accent")
2457        redraw.clicked.connect(self._redraw)
2458        save = QPushButton("Save PNG")
2459        save.clicked.connect(self._save_png)
2460        close = QPushButton("Close")
2461        close.clicked.connect(self.accept)
2462        wrap = QHBoxLayout()
2463        wrap.addWidget(self.fixed_box, 1)
2464        wrap.addWidget(redraw)
2465        wrap.addWidget(save)
2466        wrap.addWidget(close)
2467        top.addLayout(wrap, 1, 0, 1, 6)
2468        lay.addLayout(top)
2469
2470        self._fig = Figure(dpi=100)
2471        self._canvas = FigureCanvas(self._fig)
2472        lay.addWidget(NavigationToolbar(self._canvas, self))
2473        lay.addWidget(self._canvas, 1)
2474
2475        self.x_cb.currentIndexChanged.connect(lambda _i: (self._build_fixed(), s
elf._redraw()))
2476        self._build_fixed()
2477        self._redraw()
2478
2479    def _build_fixed(self):
2480        while self.fixed_layout.count():
2481            it = self.fixed_layout.takeAt(0)
2482            if it.widget():
2483                it.widget().deleteLater()
2484        self._fixed_edits = {}
2485        xk = self.x_cb.currentData()
2486        lbl = QLabel("Fixed:")
2487        self.fixed_layout.addWidget(lbl)
2488        for name, d in self._ctx["fixed_specs"][xk]:
2489            self.fixed_layout.addWidget(QLabel(f"{name} ="))
2490            ed = QLineEdit(str(d))
2491            ed.setFixedWidth(70)
2492            self.fixed_layout.addWidget(ed)
2493            self._fixed_edits[name] = ed
2494        lo, hi, st = self._ctx["grid"][xk]
2495        self.g_lo.setText(str(lo))
2496        self.g_hi.setText(str(hi))
2497        self.g_st.setText(str(st))
2498
2499    def _inputs(self):
2500        xk = self.x_cb.currentData()
```

```
        lo, hi, st = (float(w.text()) for w in (self.g_lo, self.g_hi, self.g_st)
)
        xs = np.arange(lo, hi + st / 2.0, st)
        fx = {n: float(w.text()) for n, w in self._fixed_edits.items()}
        reps = max(200, int(self.rep_ed.text() or 2000))
        return xk, xs, fx, reps

    def _redraw(self):
        xk, xs, fx, reps = self._inputs()
        p_comp = self._ctx["computed"](xk, xs, fx)
        p_usp = self._ctx["usp"](xk, xs, fx, reps)
        self._fig.clear()
        ax = self._fig.add_subplot(111)
        ax.plot(
            xs,
            p_comp,
            "o-",
            color=SERIES_COLORS[0],
            lw=1.8,
            ms=4,
            label="Computed plan (acceptance-limit table)",
        )
        ax.plot(xs, p_usp, "s--
", color=SERIES_COLORS[2], lw=1.8, ms=4, label=self._usp_label)
        for t in (0.8, 0.9):
            ax.axhline(t, ls=":", lw=0.8, color="0.5")
        ax.set_ylim(0.0, 1.05)
        ax.set_xlabel(self.x_cb.currentText())
        ax.set_ylabel("Probability of passing")
        ax.set_title(self._title)
        ax.legend(fontsize=9, loc="best")
        ax.grid(True, alpha=0.3)
        self._fig.tight_layout()
        self._canvas.draw()

    def _save_png(self):
        path, _ = QFileDialog.getSaveFileName(
            self, "Save figure", self._save_base + ".png", "PNG image (*.png)"
        )
        if path:
            self._fig.savefig(path, dpi=150)
            QMessageBox.information(self, "Saved", f"Figure saved to {path}")


class SplashScreen(QDialog):
    """Frameless startup splash: logo, note, progress bar, developer footer."""

    def __init__(self):
        super().__init__(
            None, Qt.WindowType.FramelessWindowHint | Qt.WindowType.WindowStaysO
nTopHint
        )
        self.setFixedSize(620, 400)
        self.setStyleSheet("""
            QDialog      { background:#0d2b55; }
            QLabel       { background:transparent; }
            #title { color:white; font-size:26px; font-weight:700; }
            #note  { color:#bcd4f5; font-size:10pt; }
            #status{ color:#9fc3f2; font-size:9pt; }
            #foot  { color:#bcd4f5; font-size:9pt; }
            #foot a{ color:#4da3ff; }
            QProgressBar { background:#123a6e; border:0; border-radius:5px;
                           height:10px; text-align: center; }
            QProgressBar::chunk { background:#4da3ff; border-radius:5px; }
```

```
        """)
        lay = QVBoxLayout(self)
        lay.setContentsMargins(30, 22, 30, 14)
        center = Qt.AlignmentFlag.AlignCenter

        logo_file = resource_path("logo.png")
        if os.path.exists(logo_file):
            pix = QPixmap(logo_file)
            if not pix.isNull():
                if pix.height() > 96:
                    pix = pix.scaledToHeight(96, Qt.TransformationMode.SmoothTra
nsformation)
                ll = QLabel()
                ll.setPixmap(pix)
                ll.setAlignment(center)
                lay.addWidget(ll)

        t = QLabel("PyCuDAL")
        t.setObjectName("title")
        t.setAlignment(center)
        lay.addWidget(t)
        n = QLabel(
            "Parametric acceptance limits for USP <905> Content\n"
            "Uniformity and USP <711> Dissolution"
        )
        n.setObjectName("note")
        n.setAlignment(center)
        lay.addWidget(n)
        lay.addSpacing(14)

        self.bar = QProgressBar()
        self.bar.setRange(0, 100)
        lay.addWidget(self.bar)
        self.status = QLabel("Starting…")
        self.status.setObjectName("status")
        self.status.setAlignment(center)
        lay.addWidget(self.status)
        lay.addStretch(1)

        foot = QLabel(f'Program developed by: <a href="{REPO_URL}">Moaz El-
Essawey</a>')
        foot.setObjectName("foot")
        foot.setOpenExternalLinks(True)  # click opens the GitHub repo
        foot.setAlignment(center)
        lay.addWidget(foot)

        geo = QApplication.primaryScreen().availableGeometry()
        self.move(geo.center() - self.rect().center())

    def set_progress(self, frac, msg):
        self.bar.setValue(int(frac * 100))
        self.status.setText(msg)
        QApplication.processEvents()
        time.sleep(0.08)


def _load_libraries(splash=None):
    global np, pd, cusp1, cusp2, disp1, disp2
    global Figure, FigureCanvas, NavigationToolbar, make_interp_spline
    global HAVE_CUDAL, HAVE_MPL, HAVE_SPLINE, HAVE_XLSX

    def step(frac, msg):
        if splash is not None:
            splash.set_progress(frac, msg)
```

```
2624
2625    step(0.05, "Loading NumPy…")
2626    import numpy as _np
2627
2628    np = _np
2629
2630    step(0.20, "Loading Pandas…")
2631    import pandas as _pd
2632
2633    pd = _pd
2634
2635    step(0.40, "Loading CuDAL core…")
2636    try:
2637        from cudal import cusp1 as _a
2638        from cudal import cusp2 as _b
2639        from cudal import disp1 as _c
2640        from cudal import disp2 as _d
2641
2642        cusp1, cusp2, disp1, disp2 = _a, _b, _c, _d
2643        HAVE_CUDAL = True
2644    except Exception:
2645        HAVE_CUDAL = False
2646
2647    step(0.60, "Loading SciPy…")
2648    try:
2649        from scipy.interpolate import make_interp_spline as _mis
2650
2651        make_interp_spline = _mis
2652        HAVE_SPLINE = True
2653    except Exception:
2654        HAVE_SPLINE = False
2655
2656    step(0.75, "Loading Matplotlib…")
2657    try:
2658        from matplotlib.backends.backend_qtagg import FigureCanvasQTAgg as _C
2659        from matplotlib.backends.backend_qtagg import NavigationToolbar2QT as _T
2660        from matplotlib.figure import Figure as _F
2661
2662        Figure, FigureCanvas, NavigationToolbar = _F, _C, _T
2663        HAVE_MPL = True
2664    except Exception:
2665        HAVE_MPL = False
2666
2667    step(0.90, "Loading export engines…")
2668    try:
2669        import openpyxl  # noqa: F401
2670
2671        HAVE_XLSX = True
2672    except Exception:
2673        HAVE_XLSX = False
2674
2675    step(1.0, "Ready.")
2676
2677
2678# ---------------------------------------------------------------------------
2679# Main window
2680# ---------------------------------------------------------------------------
2681class CudalApp(QMainWindow):
2682    def __init__(self):
2683        super().__init__()
2684        self.setWindowTitle("PyCuDAL -
- Content Uniformity & Dissolution Acceptance Limits")
2685        self.resize(1180, 720)
2686        self.setMinimumSize(980, 600)
```

```
2687
2688        self._settings = self._load_settings()
2689
2690        # ---- logo ------------------------------------------------------------
-
2691        logo_file = resource_path("logo.png")
2692        if os.path.exists(logo_file):
2693            self.setWindowIcon(QIcon(logo_file))
2694
2695        central = QWidget()
2696        self.setCentralWidget(central)
2697        outer = QVBoxLayout(central)
2698
2699        header = QHBoxLayout()
2700        if os.path.exists(logo_file):
2701            pix = QPixmap(logo_file)
2702            if not pix.isNull():
2703                logo_lbl = QLabel()
2704                logo_lbl.setPixmap(
2705                    pix.scaledToHeight(40, Qt.TransformationMode.SmoothTransform
ation)
2706                )
2707                header.addWidget(logo_lbl)
2708        else:
2709            t = QLabel("CuDAL")
2710            t.setObjectName("header")
2711        s = QLabel(
2712            "   Parametric acceptance limits for USP <905> Content Uniformity "
2713            "and USP <711> Dissolution"
2714        )
2715        s.setObjectName("subheader")
2716        if not logo_file:
2717            header.addWidget(t)
2718        header.addWidget(s)
2719        header.addStretch(1)
2720        outer.addLayout(header)
2721
2722        # ---- tabs ------------------------------------------------------------
---
2723        self.notebook = QTabWidget()
2724        self.tabs = [Cusp1Tab(), Cusp2Tab(), Disp1Tab(), Disp2Tab()]
2725        for tab, text in zip(
2726            self.tabs,
2727            [
2728                "Content Uniformity -- Plan 1",
2729                "Content Uniformity -- Plan 2",
2730                "Dissolution -- Plan 1",
2731                "Dissolution -- Plan 2",
2732            ],
2733        ):
2734            self.notebook.addTab(tab, text)
2735        outer.addWidget(self.notebook, 1)
2736
2737        # ---- comprehensive menu bar ------------------------------------------
2738        menubar = self.menuBar()
2739
2740        filem = menubar.addMenu("&File")
2741        a = filem.addAction("Export current results (CSV)")
2742        a.setShortcut(QKeySequence("Ctrl+E"))
2743        a.triggered.connect(self._export_current_csv)
2744        a = filem.addAction("Export current results (PDF)")
2745        a.triggered.connect(self._export_current_pdf)
2746        a = filem.addAction("Export all results (XLSX)")
2747        a.triggered.connect(self._export_all_xlsx)
```

```
2748        filem.addSeparator()
2749        a = filem.addAction("Save settings now")
2750        a.setShortcut(QKeySequence("Ctrl+S"))
2751        a.triggered.connect(self._save_settings_now)
2752        filem.addSeparator()
2753        a = filem.addAction("Exit")
2754        a.setShortcut(QKeySequence("Ctrl+Q"))
2755        a.triggered.connect(self.close)
2756
2757        runm = menubar.addMenu("&Run")
2758        a = runm.addAction("Run analysis")
2759        a.setShortcut(QKeySequence("Ctrl+R"))
2760        a.triggered.connect(lambda: self._current_tab()._on_run())
2761        a = runm.addAction("Reset parameters")
2762        a.triggered.connect(lambda: self._current_tab()._reset_defaults())
2763        runm.addSeparator()
2764        a = runm.addAction("Plot results")
2765        a.setShortcut(QKeySequence("Ctrl+P"))
2766        a.triggered.connect(lambda: self._current_tab().results._show_plot())
2767        a = runm.addAction("OC curve\u2026")
2768        a.setShortcut(QKeySequence("Ctrl+O"))
2769        a.triggered.connect(self._show_oc_current)
2770        runm.addSeparator()
2771        a = runm.addAction("Copy selection")
2772        a.setShortcut(QKeySequence.Copy)
2773        a.triggered.connect(self._copy_current_selection)
2774        a = runm.addAction("Clear results")
2775        a.triggered.connect(lambda: self._current_tab().results.clear())
2776
2777        viewm = menubar.addMenu("&View")
2778        for i, label in enumerate(
2779            (
2780                "Content Uniformity \u2013 Plan 1",
2781                "Content Uniformity \u2013 Plan 2",
2782                "Dissolution \u2013 Plan 1",
2783                "Dissolution \u2013 Plan 2",
2784            )
2785        ):
2786            a = viewm.addAction(label)
2787            a.setShortcut(QKeySequence(f"Ctrl+{i + 1}"))
2788            a.triggered.connect(lambda _=False, i=i: self.notebook.setCurrentInd
ex(i))
2789
2790        helpm = menubar.addMenu("&Help")
2791        a = helpm.addAction("Documentation (online)")
2792        a.triggered.connect(lambda: QDesktopServices.openUrl(QUrl(REPO_URL)))
2793        a = helpm.addAction("Report an issue")
2794        a.triggered.connect(lambda: QDesktopServices.openUrl(QUrl(REPO_URL + "/i
ssues")))
2795        helpm.addSeparator()
2796        a = helpm.addAction("About / Help")
2797        a.setShortcut(QKeySequence("F1"))
2798        a.triggered.connect(self._show_about)
2799
2800        # ---- action toolbar ------------------------------------------------
2801        toolbar = self.addToolBar("Main")
2802        toolbar.setMovable(False)
2803        toolbar.setFloatable(False)
2804        toolbar.setToolButtonStyle(Qt.ToolButtonStyle.ToolButtonTextBesideIcon)
2805        toolbar.setIconSize(QSize(18, 18))
2806
2807        def _glyph_icon(glyph, color=ACCENT_DARK):
2808            px = QPixmap(18, 18)
2809            px.fill(Qt.GlobalColor.transparent)
```

```
            p = QPainter(px)
            p.setPen(QColor(color))
            f = QFont("Segoe UI Symbol", 11)
            f.setBold(True)
            p.setFont(f)
            p.drawText(px.rect(), Qt.AlignmentFlag.AlignCenter, glyph)
            p.end()
            return QIcon(px)

        def add_tool(text, glyph, slot, tip):
            a = QAction(_glyph_icon(glyph), text, self)
            a.triggered.connect(slot)
            a.setToolTip(tip)
            toolbar.addAction(a)
            return a

        add_tool(
            "Run",
            "\u25b6",
            lambda: self._current_tab()._on_run(),
            "Run the selected analysis (Ctrl+R)",
        )
        add_tool(
            "Plot",
            "\u223f",
            lambda: self._current_tab().results._show_plot(),
            "Plot results (Ctrl+P)",
        )
        add_tool(
            "OC Curve",
            "\u2277",
            self._show_oc_current,
            "OC curve: computed plan vs USP test (Ctrl+O)",
        )
        add_tool(
            "CSV", "\u2913", self._export_current_csv, "Export current results t
o CSV (Ctrl+E)"
        )
        add_tool(
            "PDF", "\u2261", self._export_current_pdf, "Export current results t
o SAS-style PDF"
        )
        add_tool("XLSX", "\u25a6", self._export_all_xlsx, "Export all results to
 Excel")
        add_tool(
            "Reset",
            "\u21ba",
            lambda: self._current_tab()._reset_defaults(),
            "Reset parameters to defaults",
        )
        add_tool("About", "?", self._show_about, "About PyCuDAL (F1)")

        # # ---- shortcuts ----------------------------------------------------
-------
        # QShortcut(QKeySequence("Ctrl+R"), self, activated=lambda: self._curren
t_tab()._on_run())
        # QShortcut(QKeySequence("Ctrl+P"), self,
        #           activated=lambda: self._current_tab().results._show_plot())

        # ---- restore persisted state --------------------------------------
---
        for tab in self.tabs:
            tab.apply_state(self._settings.get("tabs", {}).get(tab.__class__.__n
ame__))
```

```
2867        try:
2868            idx = int(self._settings.get("tab_index", 0))
2869            self.notebook.setCurrentIndex(max(0, min(idx, 3)))
2870        except Exception:
2871            pass
2872        g = self._settings.get("geometry")
2873        if isinstance(g, (list, tuple)) and len(g) == 4:
2874            self.setGeometry(int(g[0]), int(g[1]), int(g[2]), int(g[3]))
2875
2876        # ---- footer / status bar ----------------------------------------------
---
2877        self.statusBar().showMessage("Ready.")
2878        credit_lbl = QLabel(
2879            'Created by <a href="https://github.com/moazelessawey/pycudal" '
2880            'style="color: #2f6fed; text-decoration: none;">Moaz El-Essawey</a>'
2881        )
2882        credit_lbl.setOpenExternalLinks(True)
2883        credit_lbl.setTextFormat(Qt.TextFormat.RichText)
2884        credit_lbl.setStyleSheet("margin-right: 10px; font-size: 9pt;")
2885        self.statusBar().addPermanentWidget(credit_lbl)
2886
2887    # -- helpers ---------------------------------------------------------------
------
2888    def _current_tab(self):
2889        return self.notebook.currentWidget()
2890
2891    @staticmethod
2892    def _load_settings():
2893        try:
2894            with open(SETTINGS_PATH, encoding="utf-8") as fh:
2895                return json.load(fh)
2896        except Exception:
2897            return {}
2898
2899    def _save_settings(self):
2900        data = {
2901            "geometry": [self.x(), self.y(), self.width(), self.height()],
2902            "tab_index": self.notebook.currentIndex(),
2903            "tabs": {tab.__class__.__name__: tab.collect_state() for tab in self
.tabs},
2904        }
2905        try:
2906            with open(SETTINGS_PATH, "w", encoding="utf-8") as fh:
2907                json.dump(data, fh, indent=2)
2908        except Exception:
2909            pass
2910
2911    def closeEvent(self, event):
2912        self._save_settings()
2913        event.accept()
2914
2915    # -- actions ---------------------------------------------------------------
--------
2916    def _export_current_csv(self):
2917        self._current_tab().results._export_csv()
2918
2919    def _export_all_xlsx(self):
2920        if not HAVE_XLSX:
2921            QMessageBox.critical(
2922                self,
2923                "Excel export unavailable",
2924                "openpyxl is required.\nInstall it with:  pip install openpyxl",
2925            )
2926            return
```

```
2927        sheets = {
2928            tab.__class__.__name__: tab.results._df
2929            for tab in self.tabs
2930            if tab.results._df is not None
2931        }
2932        if not sheets:
2933            QMessageBox.information(self, "Nothing to export", "Run at least one
 analysis first.")
2934            return
2935        path, _ = QFileDialog.getSaveFileName(
2936            self,
2937            "Export all results",
2938            f"PyCuDAL-all-results-{time.strftime('%Y%m%d-%H%M')}.xlsx",
2939            "Excel workbook (*.xlsx)",
2940        )
2941        if not path:
2942            return
2943        with pd.ExcelWriter(path, engine="openpyxl") as writer:
2944            for name, df in sheets.items():
2945                df.to_excel(writer, sheet_name=name[:31], index=False)
2946        QMessageBox.information(self, "Exported", f"Saved {len(sheets)} sheet(s)
 to {path}")
2947
2948    # -- top-bar dispatchers (guarded for optional features) -----------------
2949    def _export_current_pdf(self):
2950        r = self._current_tab().results
2951        if hasattr(r, "_export_pdf"):
2952            r._export_pdf()
2953        else:
2954            QMessageBox.information(
2955                self, "Unavailable", "PDF export is not included in this build."
2956            )
2957
2958    def _show_oc_current(self):
2959        t = self._current_tab()
2960        if hasattr(t, "_show_oc"):
2961            t._show_oc()
2962        else:
2963            QMessageBox.information(
2964                self, "Unavailable", "OC curves are not included in this build."
2965            )
2966
2967    def _copy_current_selection(self):
2968        r = self._current_tab().results
2969        if hasattr(r, "_copy_selection"):
2970            r._copy_selection()
2971
2972    def _save_settings_now(self):
2973        self._save_settings()
2974        self.statusBar().showMessage("Settings saved.", 3000)
2975
2976    def _show_about(self):
2977        deps = (
2978            f"matplotlib: {'yes' if HAVE_MPL else 'no'}\n"
2979            f"scipy splines: {'yes' if HAVE_SPLINE else 'no'}\n"
2980            f"openpyxl (xlsx): {'yes' if HAVE_XLSX else 'no'}"
2981        )
2982        about_text = (
2983            f"PyCuDAL v{VERSION}\n\n"
2984            "Parametric acceptance limits for USP <905> Content Uniformity\n"
2985            "and USP <711> Dissolution.\n\n"
2986            "This tool mirrors the functionality of the original SAS programs\n"
2987            "(CALCUSPx/CALDISPx, EVCUSPx/EVDISPx, SMPCUSPx/SMPDISPx)\n"
2988            "developed by James Bergum, Ph.D.\n\n"
```

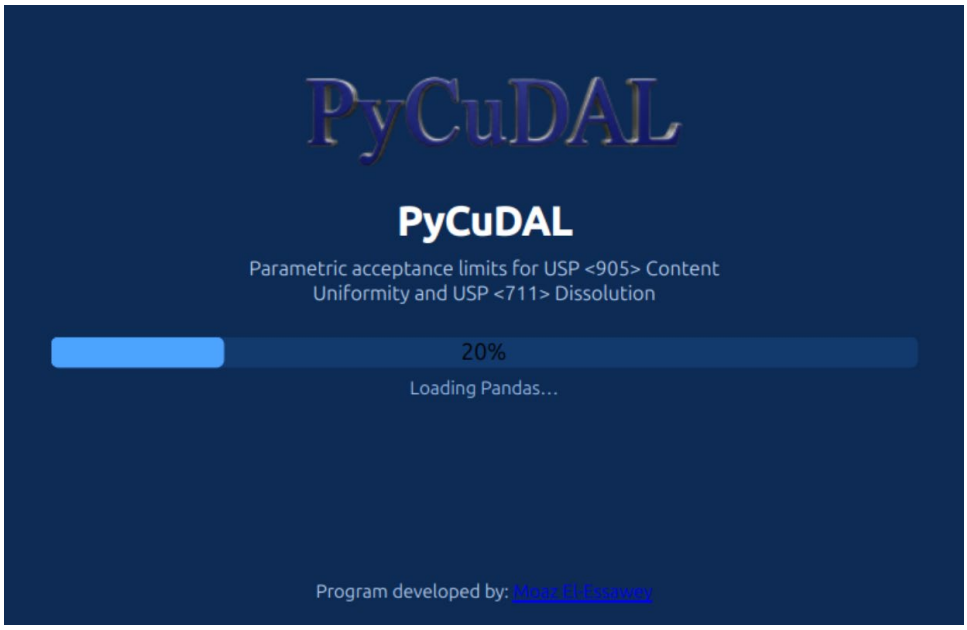
```
2989            "Created & Maintained by: Moaz El-Essawey\n"
2990            "GitHub: https://github.com/moazelessawey/pycudal\n\n"
2991            "Shortcuts:\n"
2992            "  Ctrl+R  run analysis\n"
2993            "  Ctrl+E  export current results (CSV)\n"
2994            "  Ctrl+P  plot results\n"
2995            "  Ctrl+C  copy selected table rows\n"
2996            "  F1      this dialog\n\n"
2997            f"Optional dependencies:\n{deps}"
2998        )
2999        QMessageBox.about(self, "About PyCuDAL", about_text)
3000
3001
3002# ---------------------------------------------------------------------------
3003# Self-test & entry point
3004# ---------------------------------------------------------------------------
3005def run_selftest():
3006    assert make_grid(1.0, 2.0, 0.5) == [1.0, 1.5, 2.0]
3007    try:
3008        make_grid(2.0, 1.0, 1.0, "x")
3009        raise AssertionError("make_grid should reject high < low")
3010    except ValueError:
3011        pass
3012    assert fmt_num(1234.56789) == "1,234.5679"
3013    assert fmt_num("abc") == "abc"
3014    xx, yy = _spline_xy([1, 2, 3, 4], [1, 4, 9, 16])
3015    assert len(xx) == len(yy) == 300
3016    print("selftest OK")
3017
3018
3019def main():
3020    if "--selftest" in sys.argv:
3021        _load_libraries(None)
3022        run_selftest()
3023        return
3024
3025    app = QApplication(sys.argv)
3026    app.setStyle("Fusion")
3027
3028    splash = SplashScreen()
3029    splash.show()
3030    app.processEvents()
3031    _load_libraries(splash)
3032
3033    if not HAVE_CUDAL:
3034        splash.close()
3035        QMessageBox.critical(
3036            None,
3037            "Missing dependency",
3038            "The `cudal` package could not be imported.\n"
3039            "Put this script next to the `cudal` package folder\n"
3040            "or install it, then restart the GUI.",
3041        )
3042        sys.exit(1)
3043
3044    family = _register_local_fonts()
3045    if not family:
3046        family = "Segoe UI" if sys.platform.startswith("win") else "DejaVu Sans"
3047    app.setFont(QFont(family, 10))
3048    app.setStyleSheet(build_stylesheet(family))
3049    if HAVE_MPL:
3050        try:
3051            fdir = resource_path("fonts")
3052            if os.path.isdir(fdir):
```

```
3053                 import matplotlib as mpl
3054                 from matplotlib import font_manager as fm
3055
3056                 for f in os.listdir(fdir):
3057                     if f.lower().endswith((".ttf", ".otf")):
3058                         fm.fontManager.addfont(os.path.join(fdir, f))
3059                 mpl.rcParams["font.family"] = family
3060         except Exception:
3061             pass
3062
3063     win = CudalApp()
3064     win.show()
3065     splash.close()
3066     sys.exit(app.exec())
3067
3068
3069if __name__ == "__main__":
3070     main()
```

APPENDIX B — PRIMARY WINDOWS

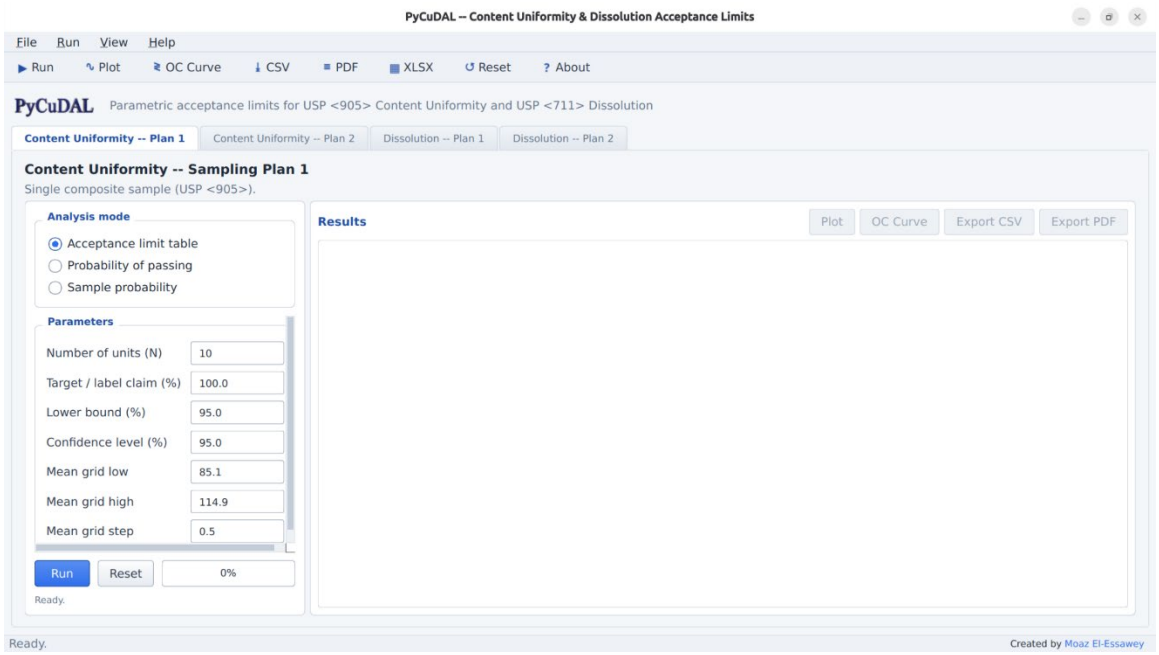
Startup (splash screen)

When the program starts, a splash screen appears while the libraries load; the progress bar tracks each stage (NumPy → Pandas → CuDAL core → SciPy → Matplotlib → export engines). The footer links to the project repository.



The main window

The main window contains: a header (logo + scope), an action toolbar, the four scenario tabs, and a status bar whose right side credits the author.



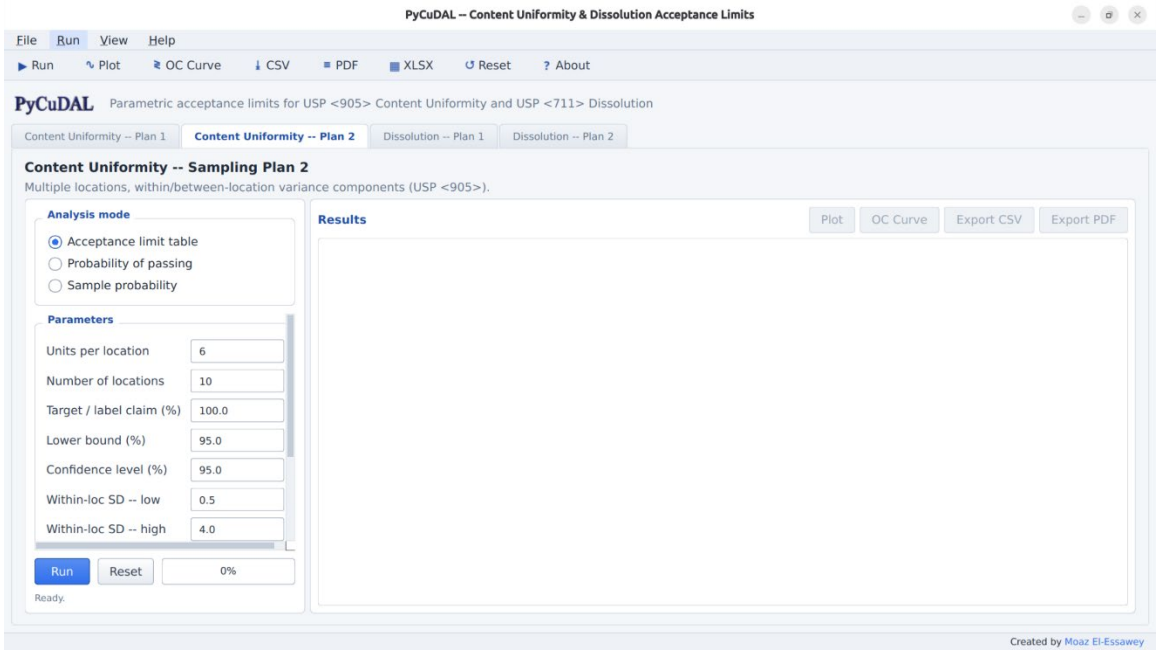
Toolbar / Run menu

Button	Action	Shortcut
▶ Run	Run the selected analysis	Ctrl+R
∿ Plot	Open the plot dialog for the current results	Ctrl+P
≷ OC Curve	Open the OC-curve dialog	Ctrl+O
↓ CSV / ≡ PDF / XLSX	Export current table / PDF / all results	Ctrl+E (CSV)
↺ Reset	Restore default parameters	
? About	About / help dialog	F1

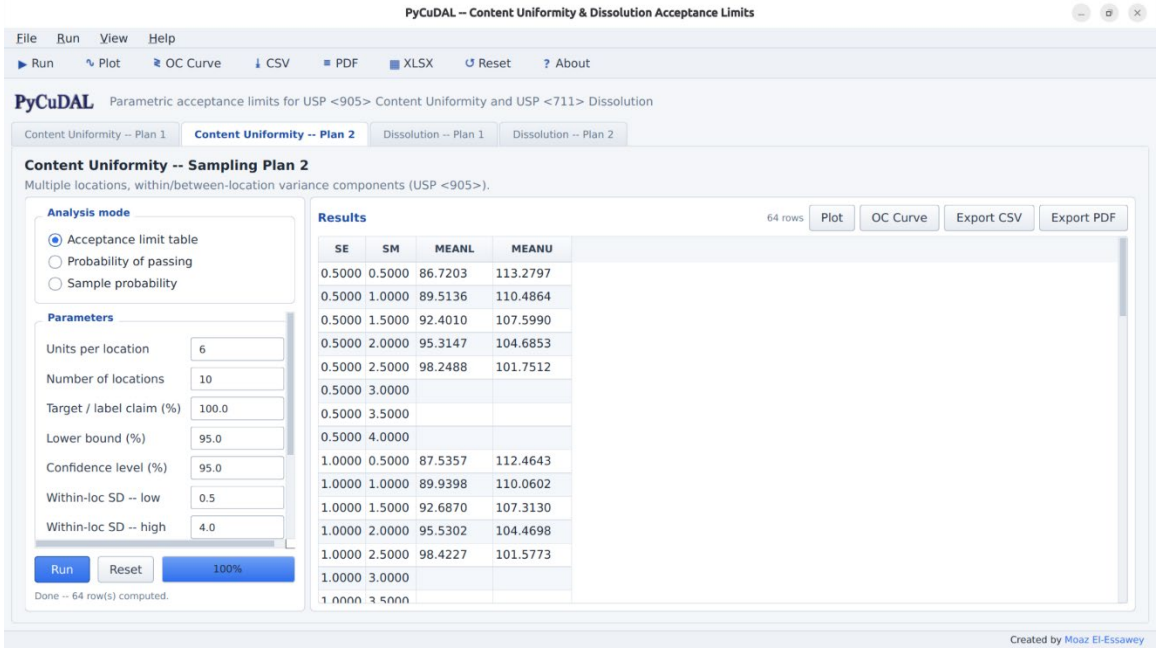
The **File** menu adds *Save settings now* (Ctrl+S) and *Exit*; **View** switches tabs (Ctrl+1…4); **Help** opens the online documentation and issue tracker. Parameters, mode, active tab and window geometry are persisted between sessions.

A scenario tab

Every tab has the same layout: an **Analysis mode** panel (three radio buttons), a scrollable **Parameters** panel (one card per mode, live-validated entries with tooltips), a **Run row** (Run, Reset, progress bar, status line) and the **Results** panel.



The **Results** panel shows the computed table with zebra striping, click-to-sort headers, conditional coloring of probability values (red < 0.80, green ≥ 0.90), a row counter, and Ctrl+C copy of the selection.

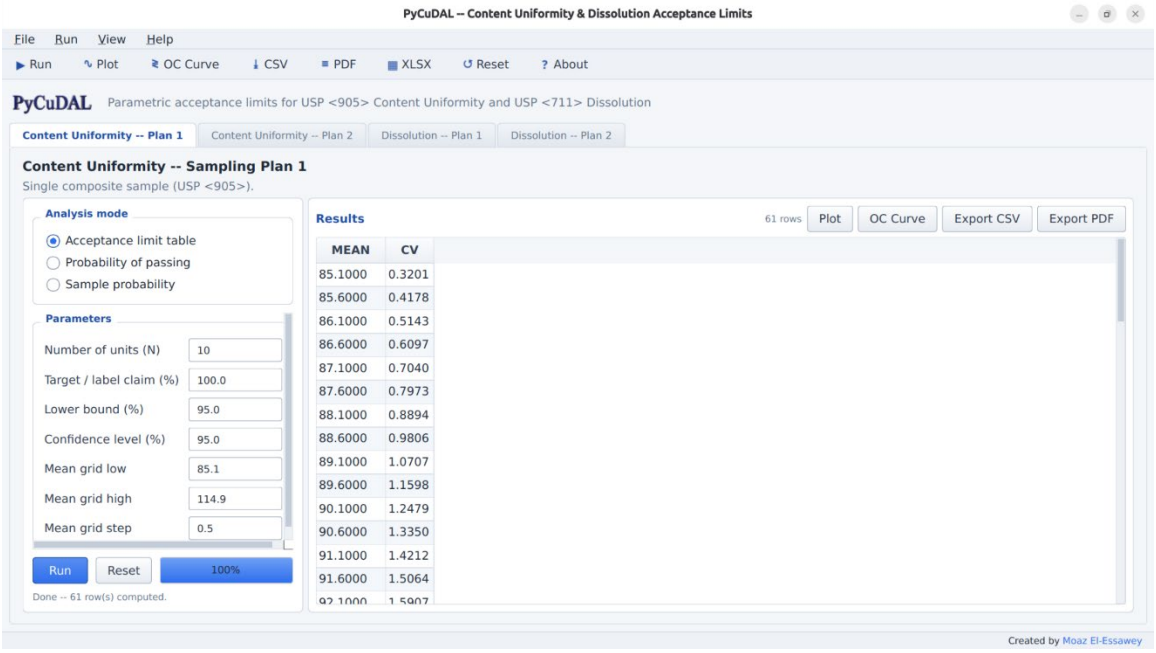


Content Uniformity / Sampling Plan 1

The user enters the sample size (number of dosage units tested), the target (usually the average of potency limits), the coverage percentage (usually 90 or 95) and the confidence level (usually 90 or 95), plus the mean grid for the table (default 85.1–114.9 by 0.5).

1) Acceptance limit table. The output lists, for each candidate mean, the corresponding acceptance limit on the CV:

MEAN (% CLAIM)	CV (%)	MEAN (% CLAIM)	CV (%)
85.1	0.48	100.0	4.18
85.2	0.51	100.1	4.16
...		...	
114.9	0.35		



2) Probability of passing. The user provides "true" values for the population mean U and coefficient of variation (CV/RSD) as grids (defaults U 95–105 × 2.5, CV 1–4 × 1); the program calculates the probability that sample results fall within the acceptance limits:

U	CV	PROBABILITY OF PASSING			
95	1	1.00000	100	1	1.00000
95	4	0.05220	100	4	0.56434

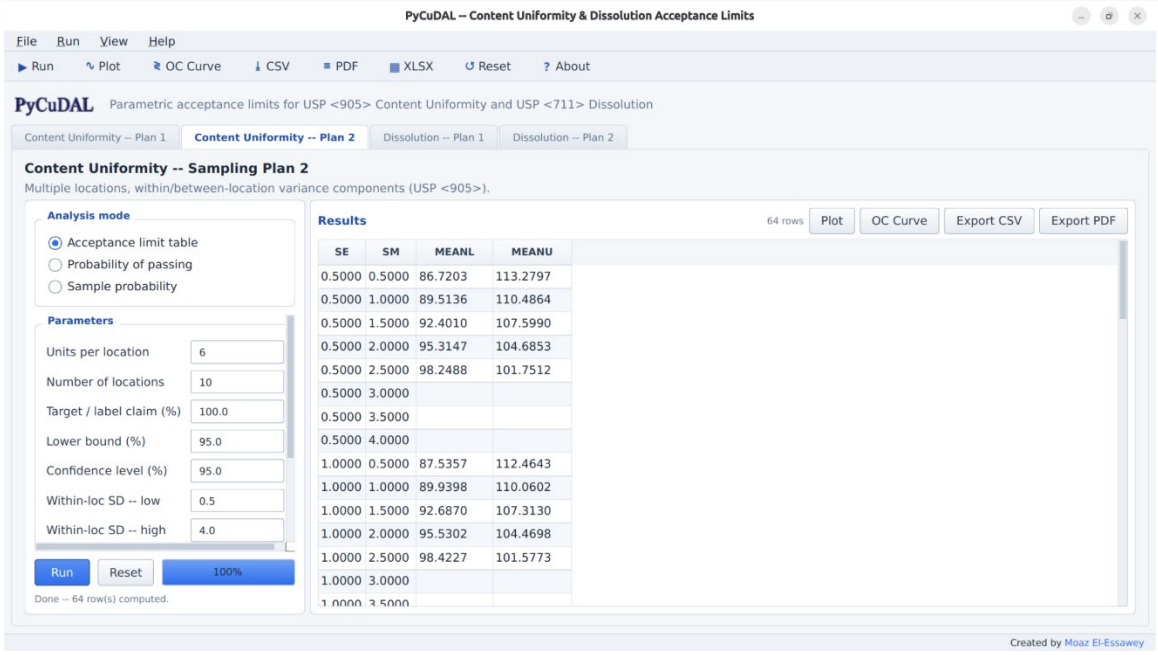
3) Sample probability. The user enters the sample mean and sample CV (defaults 100, 2.0); the output is the lower bound (e.g. mean 100, CV 4 → **0.98003**).

Content Uniformity / Sampling Plan 2

The user enters the number of locations, the number of dosage units per location, target, coverage and confidence, and (for the table) grids of the within-location SD (SE) and between-location SD (SM) (defaults 0.5–4.0 × 0.5). Table entries are **lower (LL) and upper (UL) limits on the mean**; SE is the pooled within-location standard deviation; all SDs and means are in % claim.

STANDARD DEVIATION OF LOCATION MEANS

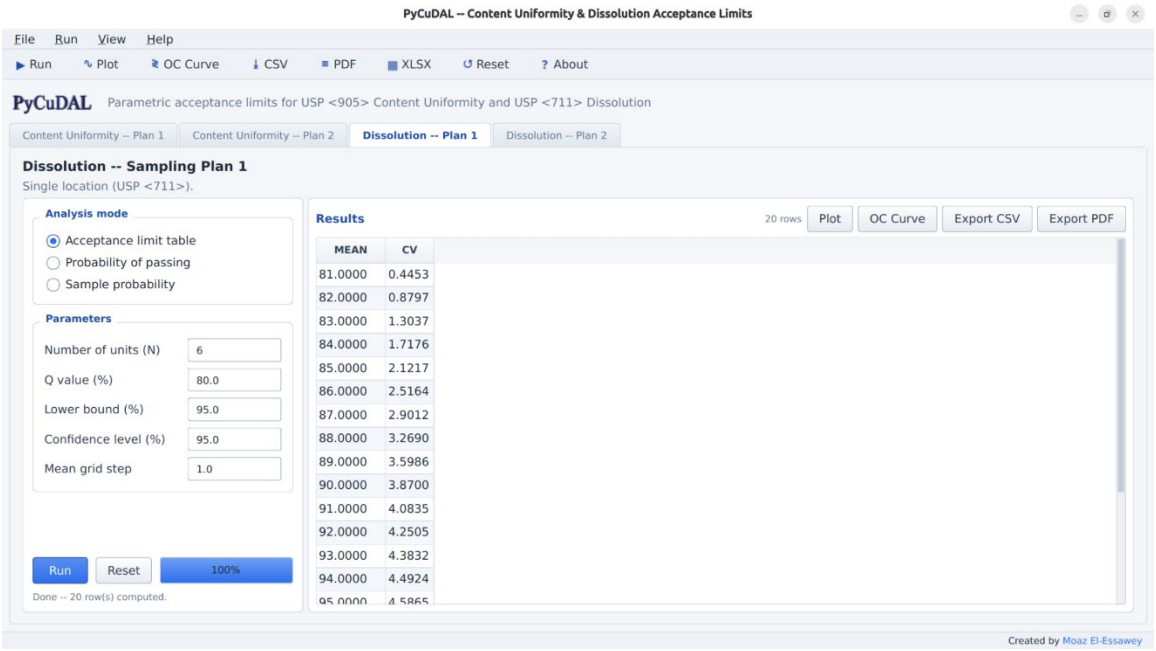
	0.1		0.2	
SE	LL	UL	LL	UL
0.1	84.8	115.2	84.8	115.2
0.2	84.7	115.3	84.8	115.2



For the evaluation the user provides true U, within-location SD and between-location SD grids; for the sample probability the sample mean, within sample SD and between sample SD. *[Note: the latter is just the sample standard deviation of the sample means, not a variance component!]* Reference values: evaluation (U 95, SE 2.2, SM 2.2, 4×10 design) → **0.09180**; sample (100, 2.2, 2.46) → **0.98750**.

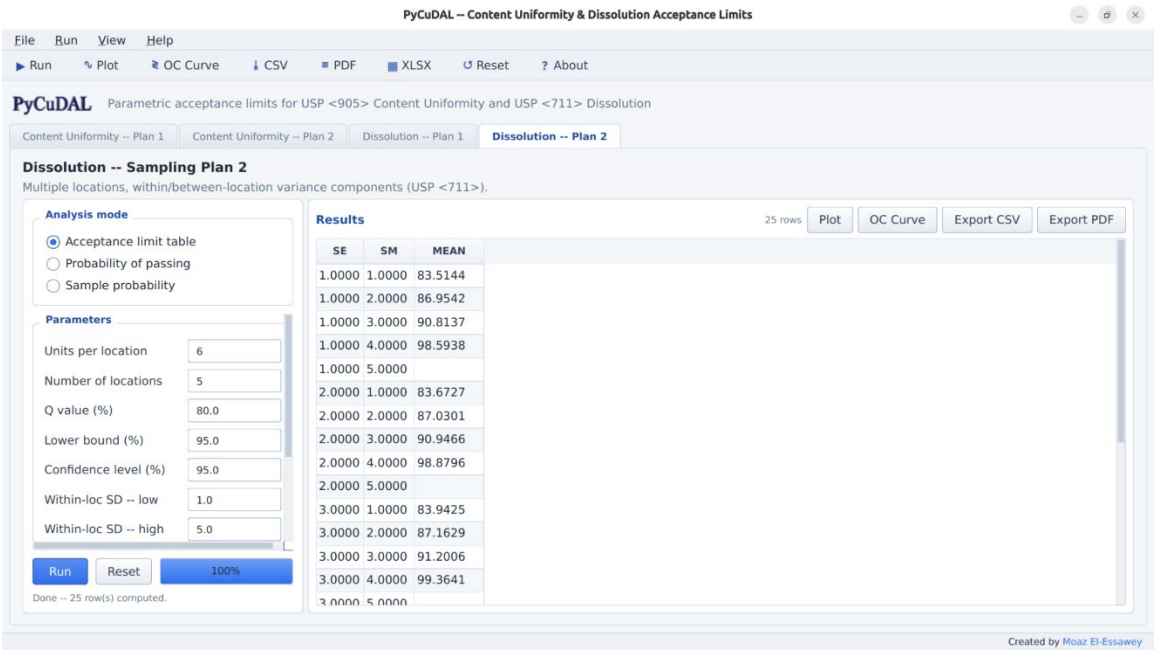
Dissolution / Sampling Plan 1

The user enters Q, the sample size, coverage and confidence (table grid step default 1.0). The table entry is the **upper limit on the CV of 6 dissolution assays** for each mean (80.2 → 0.09 … 100.0 → 4.97). Evaluation and sample modes mirror CU Plan 1 (defaults U 90–100 × 2.5, CV 1–4 × 1; sample mean 90, CV 3). Reference values: evaluation (95, 4) → **0.73988**, (100, 4) → **0.81098**; sample (100, 4) → **0.99824**.



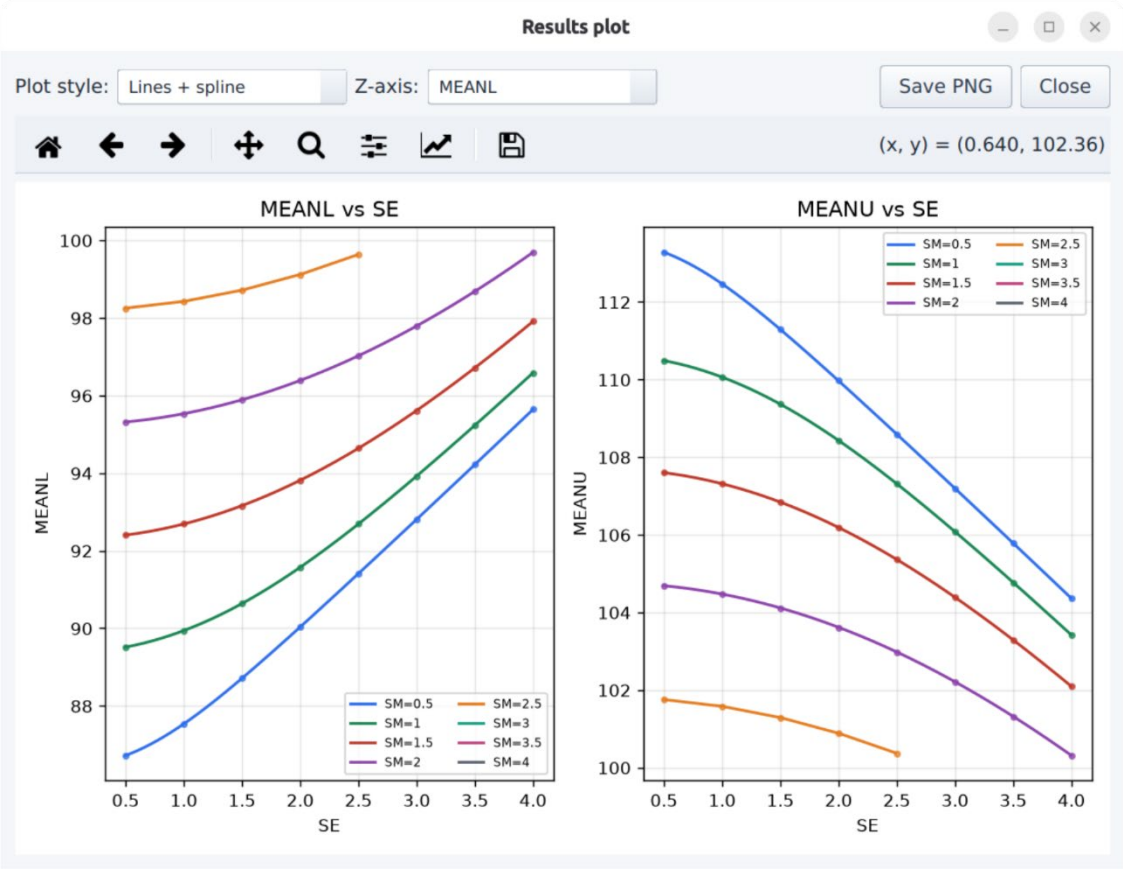
Dissolution / Sampling Plan 2

The user enters Q, locations, units per location, coverage and confidence, and SE/SM grids (defaults 1.0–5.0 × 1.0). Table entries are **lower limits on the mean** (e.g. SE 0.25 / SM 0.25 → 80.50). Evaluation and sample modes mirror CU Plan 2 (sample defaults 90, 2.2, 2.46; reference bound **1** for the 6×10 design).

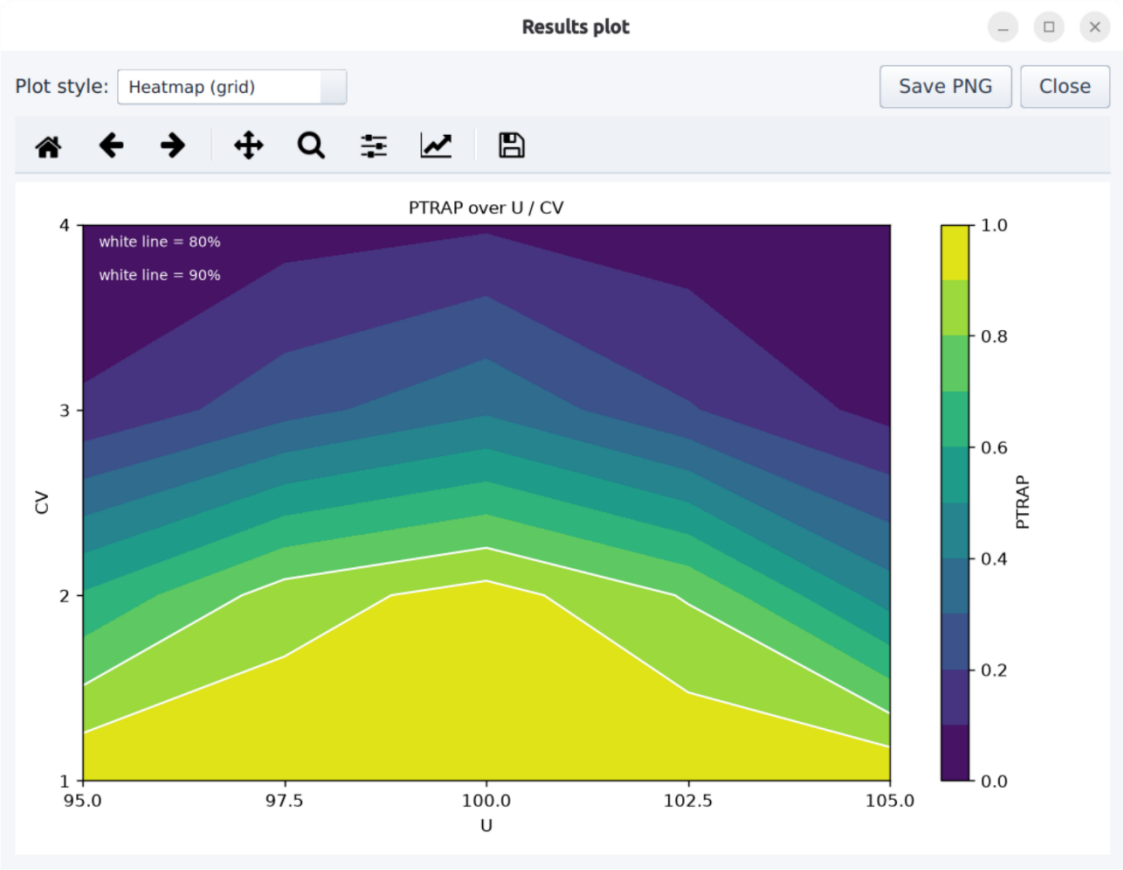


Plot dialog

Plot opens a modal dialog. *Lines + spline* draws the results as points with a cubic spline (Plan 2 tables draw one curve family per SM level, with a Z-axis selector when both LL and UL are present).

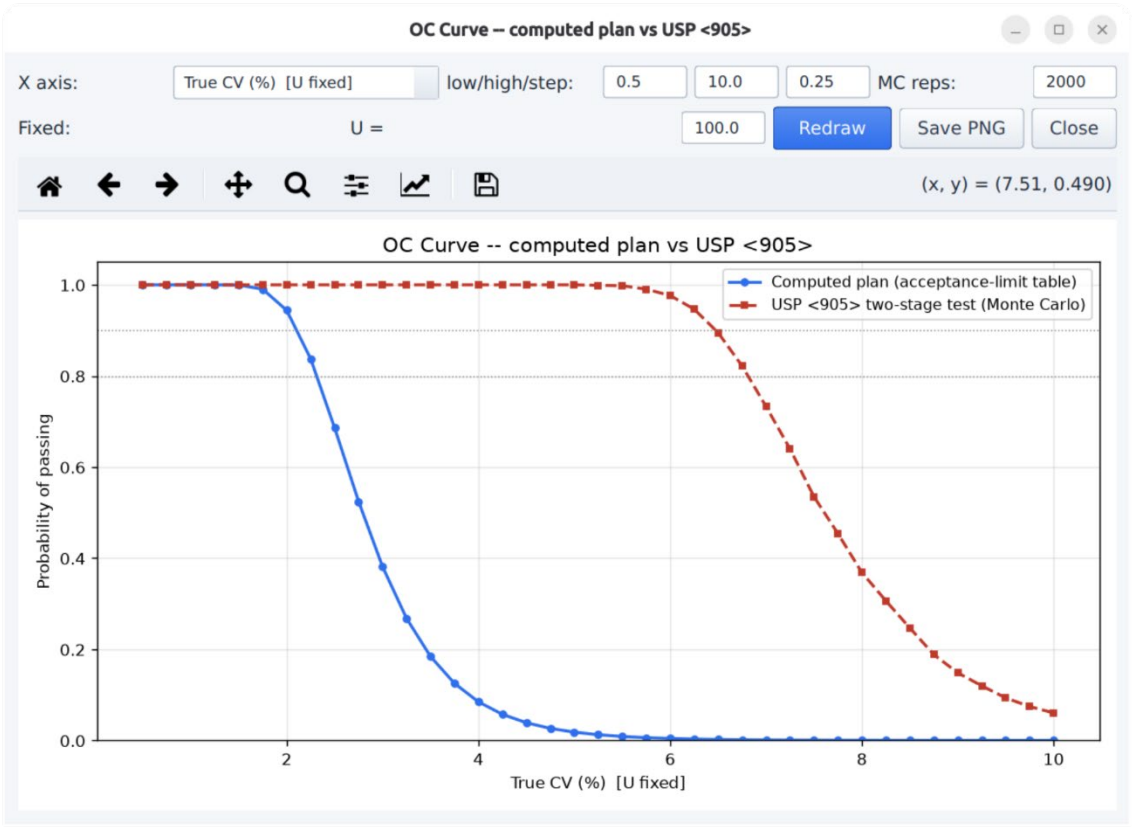


***Heatmap** (grid)* draws a contour-filled surface for 3-column grids; probability surfaces include white 80 %/90 % threshold contours. Figures can be saved as PNG



OC-Curve dialog

OC Curve overlays two operating-characteristic curves on one axes: the analytic probability of passing *your computed acceptance-limit table*, and a seeded Monte-Carlo estimate of passing the *raw USP <905>/<711> multi-stage test* itself. The X axis (true CV, true mean, or within-loc SD), the fixed companion parameters, the grid and the MC replication count are editable; dotted lines mark the 80/90 % levels.



Exports

- **CSV** – the current table; **PDF** – SAS-listing style (Courier, wrapped blocks / SE×SM LL-UL matrices, 2 dp right-aligned, * for missing, title and headers repeated on every page); **XLSX** – one sheet per scenario.
- Default filenames describe their content, e.g. CUSP2-95x95-10Lx6N.pdf, DISP1-Q80-95x95-6N-EVAL.csv, CUSP1-95x95-10N-SAMPLE.pdf, …-plot.png, …-OC.png, PyCuDAL-all-results-<date>-<time>.xlsx.

APPENDIX C — DEFAULT OUTPUT FOR ANALYSIS (R CODE)

pycudal/utils/cudal_cu.r

```r
# ============================================================================
# CuDAL R Verification Functions
# CUSP1 and CUSP2 Acceptance-Limit Table Calculations
# ============================================================================
#
# Purpose
# -------
# Independent R re-implementation of the PyCuDAL CUSP1 (Content Uniformity,
# Sampling Plan 1) and CUSP2 (Content Uniformity, Sampling Plan 2)
# acceptance-limit table calculations, USP <905>.
#
# This file is written to be read and verified line-by-line against the
# PyCuDAL Python source (cudal/core.py, cudal/cusp1.py, cudal/cusp2.py) by a
# second reviewer, then run on the shared test data set (Appendix G) so its
# output can be compared directly against PyCuDAL's own output.
#
# Design notes for the reviewer
# -----------------------------
# 1. Every root search below works on ONE grid point (one MEAN, or one
#    (SE, SM) pair) at a time, using base R's `uniroot()`. This is
#    deliberately simpler than a hand-rolled vectorized bisection: it is
#    slower, but every step is a call to a standard, well-tested R function,
#    which makes this file much easier to audit and trust.
#
# 2. `content_uniformity_bound()` never uses `ifelse(<scalar condition>, ...)`
#    with vector branches. That specific pattern is a known R pitfall:
#    `ifelse()`'s output length follows the length of its *condition*
#    argument, not its `yes`/`no` branches -- so if the condition happens to
#    be a single TRUE/FALSE (as it is here, since `target` is one fixed
#    value for an entire table) while the branches are vectors, R silently
#    truncates the result to length 1 and then recycles that single value
#    against everything downstream. Every element after the first ends up
#    silently reusing element 1's value. This file avoids the pattern
#    entirely: the target/label-claim branch (E) is computed once with a
#    plain `if/else` (correct, since `target` is always scalar here) and
#    then reused directly wherever the same branch is needed again, instead
#    of re-deriving it with a second `ifelse()`.
#
# ============================================================================


# ============================================================================
# SECTION 1: Core statistical primitives
# ============================================================================

#' Standard Normal Cumulative Distribution Function
#'
#' Direct wrapper around \code{pnorm()}. Named to match the SAS PROBNORM
#' function and the PyCuDAL \code{probnorm()} for side-by-side comparison.
#'
#' @param x Numeric. Value(s) at which to evaluate the standard normal CDF.
#' @return Numeric. P(Z <= x) for standard normal Z.
probnorm <- function(x) pnorm(x)

#' Standard Normal Quantile Function
#'
#' Direct wrapper around \code{qnorm()}. Matches SAS PROBIT / PyCuDAL
#' \code{probit()}.
#'
```

```r
#' @param p Numeric. Probability (or probabilities), in (0, 1).
#' @return Numeric. The standard normal quantile (z-value) for p.
probit <- function(p) qnorm(p)

#' Chi-Square Cumulative Distribution Function
#'
#' Direct wrapper around \code{pchisq()}. Matches SAS PROBCHI / PyCuDAL
#' \code{probchi()}.
#'
#' @param x Numeric. Value(s) at which to evaluate the CDF.
#' @param df Numeric. Degrees of freedom.
#' @return Numeric. P(X <= x) for chi-square X with df degrees of freedom.
probchi <- function(x, df) pchisq(x, df)

#' Chi-Square Quantile Function
#'
#' Direct wrapper around \code{qchisq()}. Matches SAS CINV / PyCuDAL
#' \code{cinv()}.
#'
#' @param p Numeric. Probability.
#' @param df Numeric. Degrees of freedom.
#' @return Numeric. The chi-square quantile for p at df degrees of freedom.
cinv <- function(p, df) qchisq(p, df)

#' Content Uniformity Stage Probability
#'
#' Computes one stage's pass probability for the USP <905> Content Uniformity
#' test (Stage 1: n = 10, k = 2.4; Stage 2: n = 30, k = 2.0), via the same
#' h = 0.05 numerical (Riemann-sum) integration used by the original SAS
#' macros and by PyCuDAL's \code{_cu_stage_prob()}.
#'
#' All arguments here are scalars: this function is only ever called with a
#' single (mu, sigma) pair at a time (see the design note at the top of this
#' file), so no vector broadcasting is required or attempted.
#'
#' @param mu Numeric scalar. True population mean (% of label claim).
#' @param sigma Numeric scalar. True population standard deviation.
#' @param E Numeric scalar. Adjusted upper test limit (see
#'   \code{content_uniformity_bound}).
#' @param n Integer. Stage sample size (10 or 30).
#' @param k Numeric. Stage acceptance constant (2.4 or 2.0).
#' @return Numeric scalar. The stage pass probability.
cu_stage_prob <- function(mu, sigma, E, n, k) {
  h <- 0.05
  L1 <- 15.0

  z1 <- (E - mu) * sqrt(n) / sigma
  z2 <- (98.5 - mu) * sqrt(n) / sigma
  chi_a <- probchi((n - 1) * L1^2 / (k * sigma)^2, df = n - 1)
  int1 <- (probnorm(z1) - probnorm(z2)) * chi_a

  n_steps <- round(15 / h)

  # int2: Riemann sum over [E, E + 15), step h
  steps <- 0:(n_steps - 1)
  xs <- E + steps * h
  x1 <- (xs - mu) * sqrt(n) / sigma
  x2 <- (xs + h - mu) * sqrt(n) / sigma
  chi_args <- (n - 1) * (E + 15 - xs - h / 2)^2 / (k * sigma)^2
  int2 <- sum((probnorm(x2) - probnorm(x1)) * probchi(chi_args, df = n - 1))

  # int3: Riemann sum over [98.5 - 15, 98.5), step h
  xs3 <- 98.5 - 15 + steps * h
```

```r
  x1b <- (xs3 - mu) * sqrt(n) / sigma
  x2b <- (xs3 + h - mu) * sqrt(n) / sigma
  chi_args3 <- (n - 1) * (15 - 98.5 + xs3 + h / 2)^2 / (k * sigma)^2
  int3 <- sum((probnorm(x2b) - probnorm(x1b)) * probchi(chi_args3, df = n - 1))

  int1 + int2 + int3
}

#' Content Uniformity Overall Pass-Probability Bound
#'
#' Estimated probability that a batch with true population mean \code{mu}
#' and true standard deviation \code{sigma} passes the USP <905> Content
#' Uniformity test (Stage 1 and Stage 2 combined), for label claim/target
#' \code{target}. Matches PyCuDAL's \code{content_uniformity_bound()}
#' exactly, including the Stage-2 compound tail term (P2b).
#'
#' \code{mu} and \code{sigma} are scalars in every call in this file (see the
#' design note at the top): \code{target} is always a single fixed value for
#' an entire acceptance-limit table, so there is no vector-broadcasting
#' concern here to begin with.
#'
#' @param mu Numeric scalar. True population mean (% of label claim).
#' @param sigma Numeric scalar. True population standard deviation.
#' @param target Numeric scalar. Label claim / target value (usually 100).
#' @return Numeric scalar. The overall pass probability ("OVERBD").
content_uniformity_bound <- function(mu, sigma, target) {
  E <- if (target <= 101.5) 101.5 else target

  P1 <- cu_stage_prob(mu, sigma, E, n = 10, k = 2.4)
  P2a <- cu_stage_prob(mu, sigma, E, n = 30, k = 2.0)

  zzz1 <- (123.125 - mu) / sigma
  zzz2 <- (E - 24.625 - mu) / sigma  # reuses E; see design note above
  P2b <- (probnorm(zzz1) - probnorm(zzz2))^30
  P2 <- max(0, P2a + P2b - 1)

  max(P1, P2)
}

#' Two-Tier Variance Components (CUSP2)
#'
#' Computes the total variance (VAR) and the upper-confidence-bound
#' between-location variance (MVAR) from the within-location standard
#' deviation (SE) and between-location standard deviation (SM). Matches
#' PyCuDAL's \code{_variance_components()}.
#'
#' @param se Numeric. Within-location standard deviation.
#' @param sm Numeric. Between-location standard deviation.
#' @param nn Integer. Units assayed per location.
#' @param l Integer. Number of locations.
#' @param chierr Numeric. Chi-square quantile for the error (within-location)
#'   variance component.
#' @param chiloc Numeric. Chi-square quantile for the location (between-
#'   location) variance component.
#' @return A list with elements \code{var} and \code{mvar}.
variance_components <- function(se, sm, nn, l, chierr, chiloc) {
  se2 <- se^2
  h2 <- l * (nn - 1) / chierr - 1
  sec <- ((1 - 1 / nn) * h2 * se2)^2

  sl2 <- sm^2 * nn
  sl2ub <- (l - 1) * sl2 / chiloc
  h1 <- (l - 1) / chiloc - 1
```

```r
  first <- ((1 / nn) * h1 * sl2)^2

  ptest <- (1 / nn) * sl2 + (1 - 1 / nn) * se2
  var_val <- ptest + sqrt(first + sec)
  mvar_val <- sl2ub

  list(var = var_val, mvar = mvar_val)
}


# ============================================================================
# SECTION 2: Root finding (per grid point, via base R uniroot())
# ============================================================================

#' Find a Single Root of a Monotonically Decreasing Function
#'
#' Used by CUSP1: for a fixed candidate MEAN, the pass-probability bound is
#' monotonically decreasing in the candidate standard deviation, so it has at
#' most one crossing of the target probability in [lo, hi].
#'
#' @param f Function of one scalar argument, decreasing over [lo, hi].
#' @param lo,hi Numeric. Search interval.
#' @return A list with \code{root} (numeric) and \code{floor_fail} (logical,
#'   TRUE if even \code{f(lo)} is already below zero -- i.e. no standard
#'   deviation, however small, meets the requirement).
find_single_root <- function(f, lo, hi) {
  f_lo <- f(lo)
  if (f_lo < 0) {
    return(list(root = lo, floor_fail = TRUE))
  }
  f_hi <- f(hi)
  if (f_hi >= 0) {
    # Never crosses within [lo, hi]: even the least favorable case in range
    # still passes. Report hi, matching PyCuDAL's fallback convention.
    return(list(root = hi, floor_fail = FALSE))
  }
  root <- uniroot(f, interval = c(lo, hi))$root
  list(root = root, floor_fail = FALSE)
}

#' Find the First and Last Root of a Bump-Shaped Function
#'
#' Used by CUSP2: the pass-probability bound is bump-shaped in the candidate
#' MEAN (rises above the target probability, then falls again), so it has up
#' to two crossings in [lo, hi]. A coarse scan first brackets each crossing;
#' \code{uniroot()} then refines each bracket to a precise root.
#'
#' @param f Function of one scalar argument.
#' @param lo,hi Numeric. Search interval.
#' @param scan_points Integer. Number of points used to bracket the crossings
#'   before refining (default 200 -- the interval here typically spans
#'   40-50 units, so this resolves brackets to roughly 0.2-0.25 units before
#'   \code{uniroot()} refines further).
#' @return A list with \code{first} and \code{last} (each NA if no crossing
#'   was found).
find_first_last_root <- function(f, lo, hi, scan_points = 200) {
  xs <- seq(lo, hi, length.out = scan_points)
  vals <- vapply(xs, f, numeric(1))
  s <- sign(vals)
  chg <- which(diff(s) != 0)

  if (length(chg) == 0) {
    return(list(first = NA_real_, last = NA_real_))
```

```r
  }

  first_i <- chg[1]
  last_i <- chg[length(chg)]
  first_root <- uniroot(f, interval = c(xs[first_i], xs[first_i + 1]))$root
  last_root <- uniroot(f, interval = c(xs[last_i], xs[last_i + 1]))$root

  list(first = first_root, last = last_root)
}


# ============================================================================
# SECTION 3: CUSP1 -- Content Uniformity, Sampling Plan 1
# ============================================================================

#' CUSP1 Acceptance-Limit Table
#'
#' For each candidate sample MEAN in the grid, finds the largest sample %CV
#' for which the probability of passing USP <905> still meets the required
#' lower bound at the given confidence level. Matches PyCuDAL's
#' \code{cusp1.acceptance_limit_table()}.
#'
#' @param number Integer. Sample size (N).
#' @param target Numeric. Label claim / target (usually 100).
#' @param lbound Numeric. Required lower bound on pass probability, as a
#'   percentage (e.g. 95 for 95%).
#' @param cilevel Numeric. Confidence level, as a percentage (e.g. 95).
#' @param mean_low Numeric. Low end of the candidate MEAN grid (default 85.1).
#' @param mean_high Numeric. High end of the candidate MEAN grid (default 114.9).
#' @param mean_step Numeric. Step size of the candidate MEAN grid (default 0.1).
#'
#' @return A data frame with columns \code{MEAN} and \code{CV}. A CV of 0
#'   marks a mean where even a (near) zero standard deviation fails to meet
#'   the bound.
cusp1_acceptance_limit_table <- function(number, target, lbound, cilevel,
                                         mean_low = 85.1, mean_high = 114.9,
                                         mean_step = 0.1) {
  z <- probit((1 + sqrt(cilevel / 100)) / 2)
  n <- number
  chi <- cinv(1 - sqrt(cilevel / 100), df = n - 1)
  target_prob <- lbound / 100

  means <- seq(mean_low, mean_high, by = mean_step)
  cv <- numeric(length(means))

  for (i in seq_along(means)) {
    mean_i <- means[i]

    f <- function(sd) {
      sigma <- sqrt((n - 1) * sd^2 / chi)
      llu <- mean_i - z * sigma / sqrt(n)
      ulu <- mean_i + z * sigma / sqrt(n)
      overlbd <- content_uniformity_bound(llu, sigma, target)
      overubd <- content_uniformity_bound(ulu, sigma, target)
      min(overlbd, overubd) - target_prob
    }

    res <- find_single_root(f, lo = 0.01, hi = 7.8)
    cv[i] <- if (res$floor_fail) 0 else 100 * res$root / mean_i
  }

  data.frame(MEAN = means, CV = cv)
}
```

```r

# ============================================================================
# SECTION 4: CUSP2 -- Content Uniformity, Sampling Plan 2
# ============================================================================

#' CUSP2 Acceptance-Limit Table
#'
#' For each (SE, SM) combination, finds the acceptable range of sample mean
#' [MEANL, MEANU] for which the probability of passing USP <905> meets the
#' required lower bound. Matches PyCuDAL's \code{cusp2.acceptance_limit_table()}.
#'
#' @param num Integer. Units assayed per location.
#' @param loc Integer. Number of locations.
#' @param target Numeric. Label claim / target (usually 100).
#' @param lbound Numeric. Required lower bound on pass probability (%).
#' @param cilevel Numeric. Confidence level (%).
#' @param se_values Numeric vector. Within-location standard deviations to
#'   evaluate.
#' @param sm_values Numeric vector. Between-location standard deviations to
#'   evaluate.
#' @param mean_search_range Numeric vector of length 2. Search bounds for the
#'   mean (default c(80, 120)).
#'
#' @return A data frame with columns \code{SE}, \code{SM}, \code{MEANL},
#'   \code{MEANU}. MEANL/MEANU are \code{NA} where no acceptable mean range
#'   exists for that (SE, SM) combination.
cusp2_acceptance_limit_table <- function(num, loc, target, lbound, cilevel,
                                         se_values, sm_values,
                                         mean_search_range = c(80.0, 120.0)) {
  nn <- num
  l <- loc
  n <- nn * l
  z <- probit((1 + sqrt(cilevel / 100)) / 2)
  chierr <- cinv(1 - sqrt(cilevel / 100), df = l * (nn - 1))
  chiloc <- cinv(1 - sqrt(cilevel / 100), df = l - 1)
  target_prob <- lbound / 100
  lo <- mean_search_range[1]
  hi <- mean_search_range[2]

  grid <- expand.grid(SE = se_values, SM = sm_values)
  n_grid <- nrow(grid)
  meanl <- numeric(n_grid)
  meanu <- numeric(n_grid)

  for (i in seq_len(n_grid)) {
    se_i <- grid$SE[i]
    sm_i <- grid$SM[i]

    vc <- variance_components(se_i, sm_i, nn, l, chierr, chiloc)
    sigma <- sqrt(vc$var)
    se_of_mean <- sqrt(vc$mvar / n)

    f <- function(mean_val) {
      llu <- mean_val - z * se_of_mean
      ulu <- mean_val + z * se_of_mean
      overlbd <- content_uniformity_bound(llu, sigma, target)
      overubd <- content_uniformity_bound(ulu, sigma, target)
      min(overlbd, overubd) - target_prob
    }

    res <- find_first_last_root(f, lo, hi)

```

```
375     if (is.na(res$first) || is.na(res$last) || res$last <= res$first) {
376       meanl[i] <- NA_real_
377       meanu[i] <- NA_real_
378     } else {
379       meanl[i] <- res$first
380       meanu[i] <- res$last
381     }
382   }
383
384   data.frame(SE = grid$SE, SM = grid$SM, MEANL = meanl, MEANU = meanu)
385 }
```

pycudal/utils/cusp1_script.r

```
source("cudal_cu.r")

groups <- list(
  c(5, 100.0, 50.0, 50.0), c(2000, 100.0, 50.0, 50.0),
  c(5, 100.0, 50.0, 99.0), c(2000, 100.0, 50.0, 99.0),
  c(5, 100.0, 99.0, 99.0), c(2000, 100.0, 99.0, 99.0),
  c(5, 104.5, 50.0, 50.0), c(2000, 104.5, 50.0, 50.0),
  c(5, 104.5, 50.0, 99.0), c(2000, 104.5, 50.0, 99.0),
  c(5, 104.5, 99.0, 99.0), c(2000, 104.5, 99.0, 99.0)
)

results <- list()
for (g in groups) {
  size <- g[1]; target <- g[2]; lbound <- g[3]; ci <- g[4]
  tab <- cusp1_acceptance_limit_table(number=size, target=target, lbound=lbound, c
ilevel=ci,
                                      mean_low=85.1, mean_high=114.9, mean_step=1
4.9)
  for (i in 1:nrow(tab)) {
    results[[length(results)+1]] <- list(size=size, target=target, lbound=lbound,
ci=ci,
                                         mean=tab$MEAN[i], cv=tab$CV[i])
  }
}
df <- do.call(rbind, lapply(results, as.data.frame))
print(round(df, 2))
```

pycudal/utils/cusp1_script.r output

	size	target	lbound	ci	mean	cv
1	5	100.0	50	50	85.1	0.56
2	5	100.0	50	50	100.0	4.88
3	5	100.0	50	50	114.9	0.42
4	2000	100.0	50	50	85.1	0.93
5	2000	100.0	50	50	100.0	7.31
6	2000	100.0	50	50	114.9	0.69
7	5	100.0	50	99	85.1	0.13
8	5	100.0	50	99	100.0	1.16
9	5	100.0	50	99	114.9	0.10
10	2000	100.0	50	99	85.1	0.88
11	2000	100.0	50	99	100.0	7.06
12	2000	100.0	50	99	114.9	0.65
13	5	100.0	99	99	85.1	0.11
14	5	100.0	99	99	100.0	0.94
15	5	100.0	99	99	114.9	0.08
16	2000	100.0	99	99	85.1	0.64
17	2000	100.0	99	99	100.0	5.41
18	2000	100.0	99	99	114.9	0.48
19	5	104.5	50	50	85.1	0.56
20	5	104.5	50	50	100.0	4.75
21	5	104.5	50	50	114.9	1.20
22	2000	104.5	50	50	85.1	0.93
23	2000	104.5	50	50	100.0	7.21
24	2000	104.5	50	50	114.9	1.98
25	5	104.5	50	99	85.1	0.13
26	5	104.5	50	99	100.0	1.14
27	5	104.5	50	99	114.9	0.28
28	2000	104.5	50	99	85.1	0.88
29	2000	104.5	50	99	100.0	6.94
30	2000	104.5	50	99	114.9	1.88
31	5	104.5	99	99	85.1	0.11
32	5	104.5	99	99	100.0	0.93
33	5	104.5	99	99	114.9	0.23
34	2000	104.5	99	99	85.1	0.64
35	2000	104.5	99	99	100.0	5.29
36	2000	104.5	99	99	114.9	1.37

pycudal/utils/cusp2_script.r

```
source("cudal_cu.r")

full_four <- list(c(0.1,0.1), c(0.1,3.0), c(3.0,0.1), c(3.0,3.0))
blocks <- list(
  list(3,2,100.0,50.0,50.0, full_four),
  list(3,300,100.0,50.0,50.0, full_four),
  list(300,2,100.0,50.0,50.0, full_four),
  list(300,300,100.0,50.0,50.0, full_four),
  list(3,2,100.0,99.0,50.0, full_four),
  list(3,300,100.0,99.0,50.0, full_four),
  list(300,2,100.0,99.0,50.0, full_four),
  list(300,300,100.0,99.0,50.0, full_four),
  list(3,2,100.0,99.0,99.0, list(c(0.1,0.1), c(3.0,3.0))),
  list(3,300,100.0,99.0,99.0, full_four),
  list(300,2,100.0,99.0,99.0, full_four),
  list(300,300,100.0,99.0,99.0, full_four)
)
singles <- list(
  list(3,2,102.5,50.0,50.0,0.1,3.0),
  list(300,300,102.5,50.0,99.0,3.0,3.0),
  list(3,2,102.5,50.0,99.0,0.1,0.1),
  list(3,300,102.5,50.0,99.0,0.1,3.0)
)

results <- list()
for (b in blocks) {
  loc <- b[[1]]; perloc <- b[[2]]; target <- b[[3]]; lbound <- b[[4]]; ci <- b[[5]
]; pairs <- b[[6]]
  for (p in pairs) {
    se <- p[1]; sm <- p[2]
    tab <- cusp2_acceptance_limit_table(num=perloc, loc=loc, target=target, lbound
=lbound, cilevel=ci,
                                        se_values=c(se), sm_values=c(sm))
    results[[length(results)+1]] <- list(loc=loc, perloc=perloc, target=target, lb
ound=lbound, ci=ci,
                                         se=se, sm=sm, meanl=tab$MEANL[1], meanu=
tab$MEANU[1])
  }
}
for (s in singles) {
  loc <- s[[1]]; perloc <- s[[2]]; target <- s[[3]]; lbound <- s[[4]]; ci <- s[[5]
]; se <- s[[6]]; sm <- s[[7]]
  tab <- cusp2_acceptance_limit_table(num=perloc, loc=loc, target=target, lbound=l
bound, cilevel=ci,
                                      se_values=c(se), sm_values=c(sm))
  results[[length(results)+1]] <- list(loc=loc, perloc=perloc, target=target, lbou
nd=lbound, ci=ci,
                                       se=se, sm=sm, meanl=tab$MEANL[1], meanu=ta
b$MEANU[1])
}
df2 <- do.call(rbind, lapply(results, as.data.frame))
print(round(df2, 2))
```

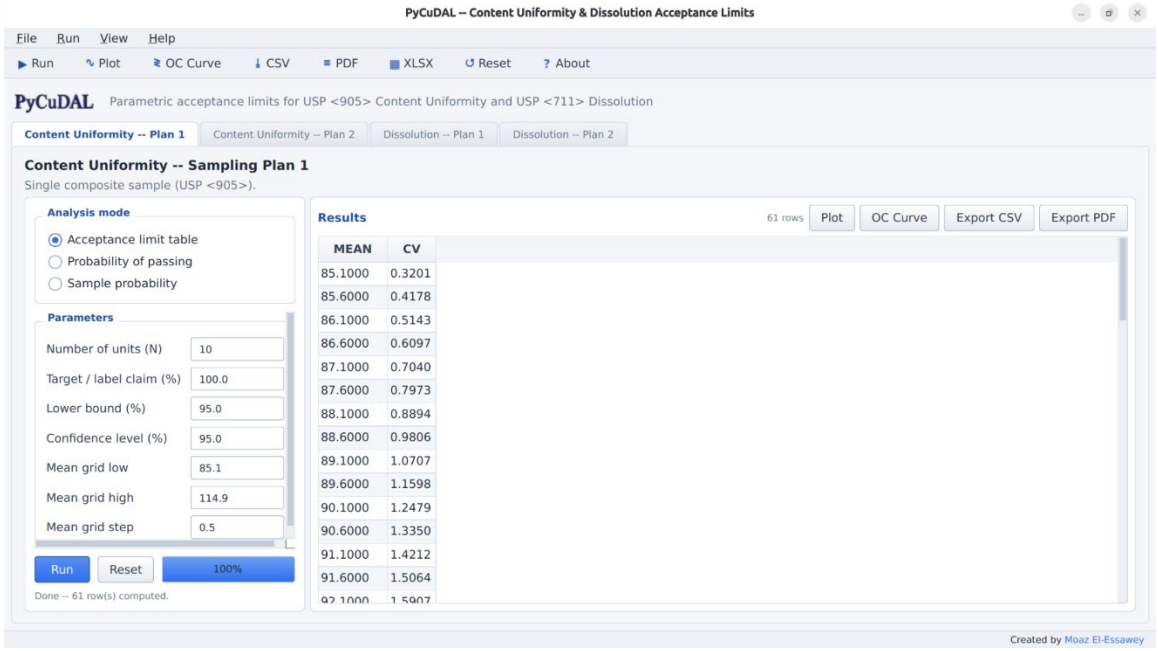
pycudal/utils/cusp2_script.r output

	loc	perloc	target	lbound	ci	se	sm	meanl	meanu
1	3	2	100.0	50	50	0.1	0.1	83.97	116.03
2	3	2	100.0	50	50	0.1	3.0	96.74	103.26
3	3	2	100.0	50	50	3.0	0.1	89.77	110.23
4	3	2	100.0	50	50	3.0	3.0	97.75	102.25
5	3	300	100.0	50	50	0.1	0.1	83.99	116.01
6	3	300	100.0	50	50	0.1	3.0	96.74	103.26
7	3	300	100.0	50	50	3.0	0.1	89.61	110.39
8	3	300	100.0	50	50	3.0	3.0	98.42	101.58
9	300	2	100.0	50	50	0.1	0.1	83.75	116.25
10	300	2	100.0	50	50	0.1	3.0	89.77	110.23
11	300	2	100.0	50	50	3.0	0.1	87.81	112.19
12	300	2	100.0	50	50	3.0	3.0	91.09	108.91
13	300	300	100.0	50	50	0.1	0.1	83.79	116.21
14	300	300	100.0	50	50	0.1	3.0	89.77	110.23
15	300	300	100.0	50	50	3.0	0.1	89.44	110.56
16	300	300	100.0	50	50	3.0	3.0	92.20	107.80
17	3	2	100.0	99	50	0.1	0.1	84.12	115.88
18	3	2	100.0	99	50	0.1	3.0	NA	NA
19	3	2	100.0	99	50	3.0	0.1	92.14	107.86
20	3	2	100.0	99	50	3.0	3.0	NA	NA
21	3	300	100.0	99	50	0.1	0.1	84.15	115.85
22	3	300	100.0	99	50	0.1	3.0	NA	NA
23	3	300	100.0	99	50	3.0	0.1	91.93	108.07
24	3	300	100.0	99	50	3.0	3.0	NA	NA
25	300	2	100.0	99	50	0.1	0.1	83.85	116.15
26	300	2	100.0	99	50	0.1	3.0	92.11	107.89
27	300	2	100.0	99	50	3.0	0.1	89.47	110.53
28	300	2	100.0	99	50	3.0	3.0	93.94	106.06
29	300	300	100.0	99	50	0.1	0.1	83.90	116.10
30	300	300	100.0	99	50	0.1	3.0	92.11	107.89
31	300	300	100.0	99	50	3.0	0.1	91.73	108.27
32	300	300	100.0	99	50	3.0	3.0	95.46	104.54
33	3	2	100.0	99	99	0.1	0.1	89.67	110.33
34	3	2	100.0	99	99	3.0	3.0	NA	NA
35	3	300	100.0	99	99	0.1	0.1	89.66	110.34
36	3	300	100.0	99	99	0.1	3.0	NA	NA
37	3	300	100.0	99	99	3.0	0.1	95.00	105.00
38	3	300	100.0	99	99	3.0	3.0	NA	NA
39	300	2	100.0	99	99	0.1	0.1	83.89	116.11
40	300	2	100.0	99	99	0.1	3.0	93.23	106.77
41	300	2	100.0	99	99	3.0	0.1	90.02	109.98
42	300	2	100.0	99	99	3.0	3.0	95.01	104.99
43	300	300	100.0	99	99	0.1	0.1	83.93	116.07
44	300	300	100.0	99	99	0.1	3.0	93.23	106.77
45	300	300	100.0	99	99	3.0	0.1	91.78	108.22
46	300	300	100.0	99	99	3.0	3.0	96.38	103.62

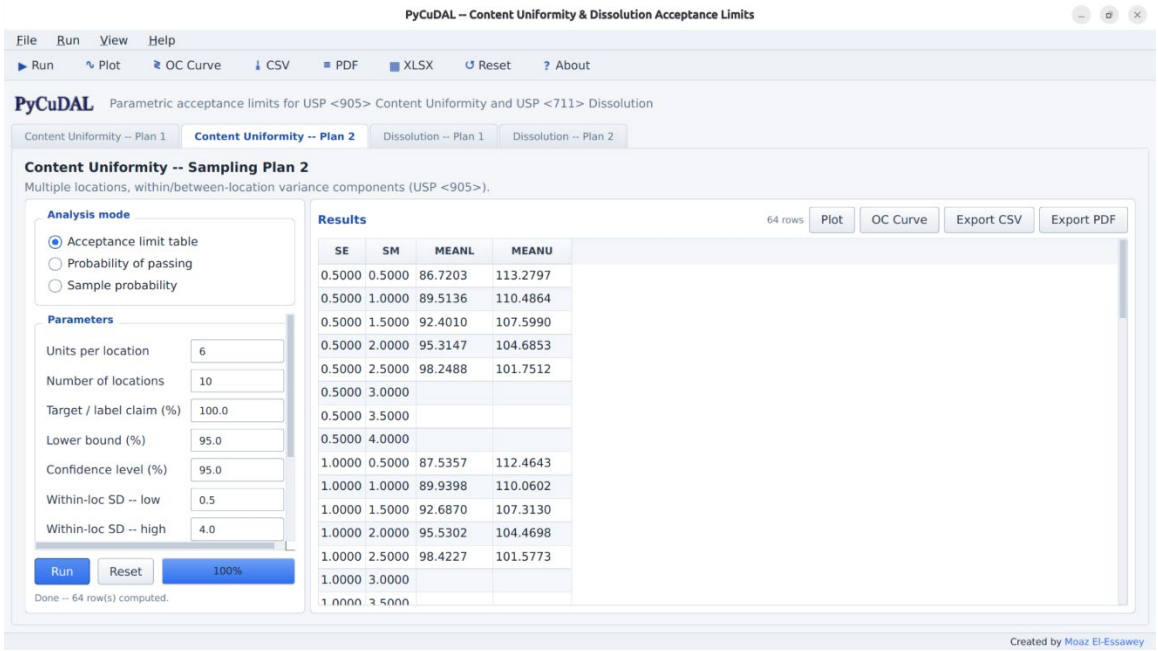
47	3	2	102.5	50	50	0.1	3.0	96.79	104.21
48	300	300	102.5	50	99	3.0	3.0	93.00	108.00
49	3	2	102.5	50	99	0.1	0.1	88.59	112.41
50	3	300	102.5	50	99	0.1	3.0	NA	NA

APPENDIX D — NAVIGATION AND ERROR CHECKS

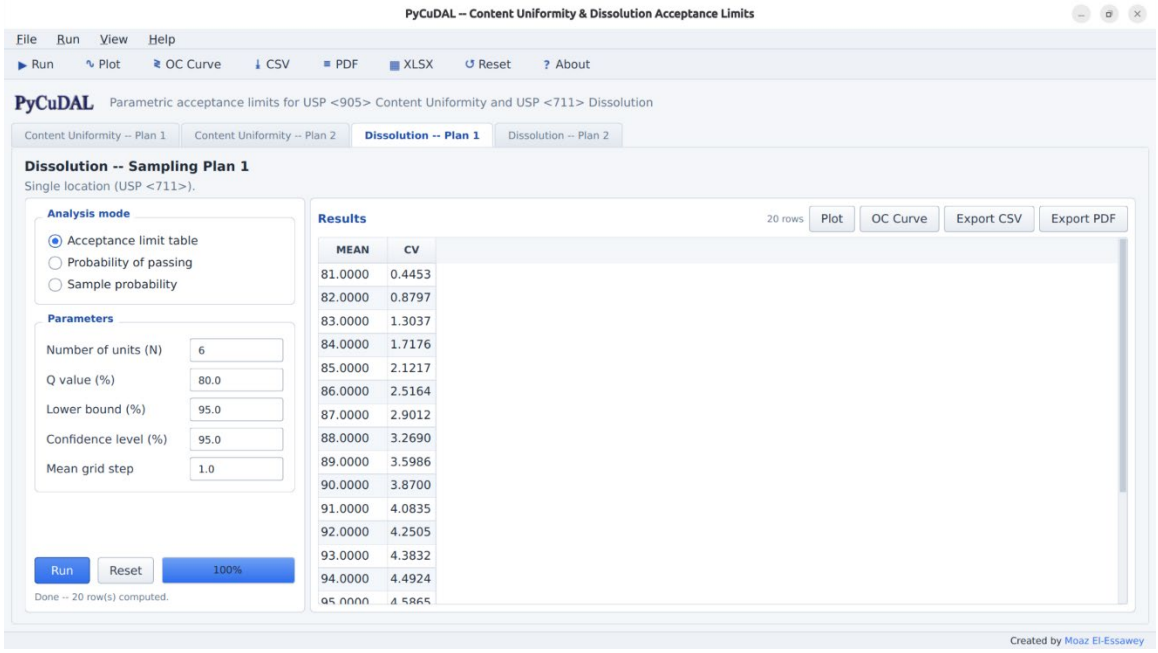
Content Uniformity / Sampling Plan 1



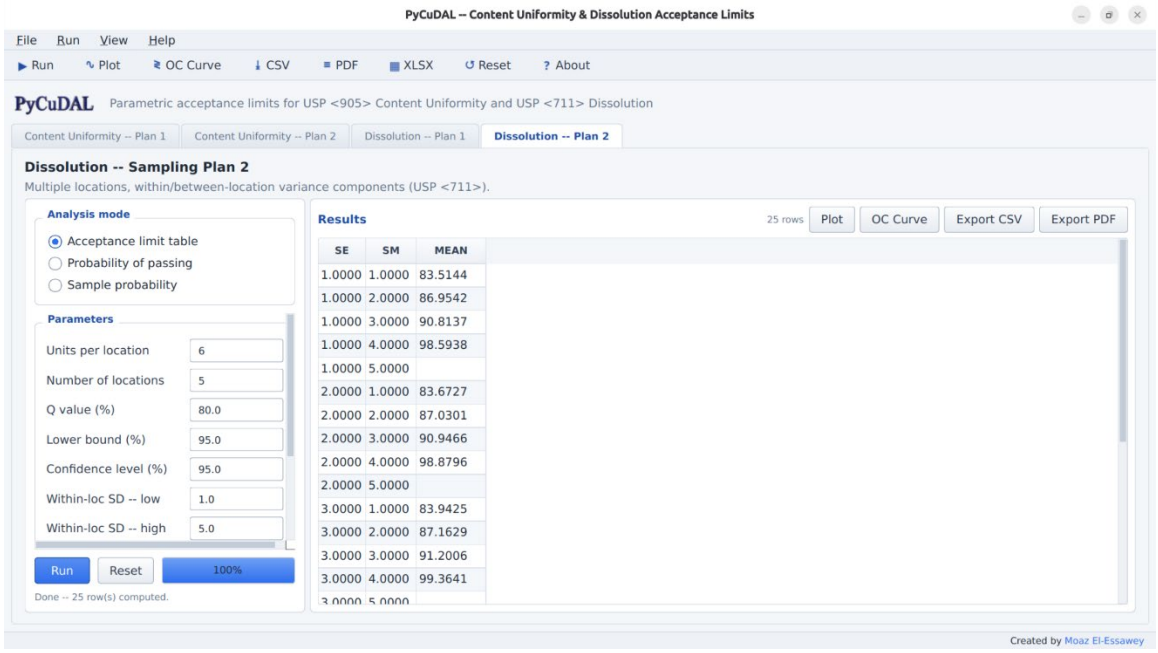
Content Uniformity / Sampling Plan 2



Dissolution / Sampling Plan 1



Dissolution / Sampling Plan 2



APPENDIX E — MATH / LOWER BOUND CALCULATIONS

USP Content Uniformity and Dissolution Tests

Content Uniformity (USP 905)

Stage 1) Test 10 dosage units. Requirements are met if the acceptance value of the first 10 units satisfies

$$AV \leq L_1 = 15$$

Otherwise go to Stage 2.

Stage 2) Test an additional 20 units. Pass if, for all 30 units:

$$AV_{30} \leq 15 \quad \text{and} \quad |x_i - M| \leq 0.25\,M \;\; \forall i$$

The acceptance value is defined as

$$AV = |M - \bar{X}| + k\,s, \qquad k = \begin{cases} 2.4 & n = 10 \\ 2.0 & n = 30 \end{cases}$$

where $\bar{X}$ is the sample mean and s the sample standard deviation. M is based on the target T (average of the monograph potency limits, % of label claim):

For $T \leq 101.5$:

$$M = \begin{cases} \max(98.5,\ \bar{X}) & \text{if } \bar{X} \leq 100 \\ \min(101.5,\ \bar{X}) & \text{if } \bar{X} > 100 \end{cases}$$

For $T > 101.5$:

$$M = \begin{cases} \max(98.5,\ \bar{X}) & \text{if } \bar{X} \leq 100 \\ \min(T,\ \bar{X}) & \text{if } \bar{X} > 100 \end{cases}$$

Dissolution (USP 711)

Stage 1) Test 6 units. Pass if

$$x_i \geq Q + 5 \quad \forall\, i = 1, \ldots, 6$$

Stage 2) Test 6 additional units. Pass if

$$\bar{x}_{12} \geq Q \quad \text{and} \quad x_i > Q - 15 \quad \forall i$$

Stage 3) Test 12 additional units. Pass if

$$\bar{x}_{24} \geq Q,$$

with no more than two results $\leq Q - 15$ and no result $\leq Q - 25$. Otherwise Fail.

Statistical Methods

Assume normal dosage-unit results $X_i \sim \mathcal{N}(\mu, \sigma^2)$. Then

$$Z_1 = \bar{X} \sim \mathcal{N}\Big(\mu, \tfrac{\sigma}{\sqrt{n}}\Big), \qquad Z_2 = \frac{(n-1)s^2}{\sigma^2} \sim \chi^2_{n-1}$$

are independent, and the overall lower bound of passing the USP test is

$$P(\text{pass}) \geq \max\{P(S_1),\, P(S_2)\}$$

Computation of $P(S_1)$

With $L_1 = 15$ and $k_1 = 2.4$:

$$P(S_1) = \big(\Phi(t_1) - \Phi(t_2)\big)\; P\!\left(\chi^2_{n-1} < \frac{(n-1)L_1^2}{k_1^2\,\sigma^2}\right) + I_2 + I_3$$

$$t_1 = \frac{101.5 - \mu}{\sigma/\sqrt{n}}, \qquad t_2 = \frac{98.5 - \mu}{\sigma/\sqrt{n}}$$

The tail integrals are evaluated numerically with step $h = 0.05$:

$$I_2 = \sum_{i=1}^{K} \big[\Phi(z_0 + ih) - \Phi(z_0 + (i-1)h)\big]\; P\big(\chi^2_{n-1} < g(z_0 + (i - \tfrac{1}{2})h)\big)$$

$$g(z_1) = \frac{(n-1)\,\big(L_1 + 101.5 - z_1\big)^2}{k_1^2\,\sigma^2}$$

$$I_3 = \sum_{i=1}^{K} \big[\Phi(z_0 + ih) - \Phi(z_0 + (i-1)h)\big]\; P\big(\chi^2_{n-1} < q(z_0 + (i - \tfrac{1}{2})h)\big)$$

$$q(z_1) = \frac{(n-1)\,\big(L_1 - 98.5 + z_1\big)^2}{k_1^2\,\sigma^2}$$

Computation of $P(S_2)$

Let C_{21} = "AV of the 30 units $\leq L_1$" and C_{22} = "no unit deviates from M by more than $L_2 = 25$". Using $P(A \cap B) \geq P(A) + P(B) - 1$:

$$P(S_2) = P(C_{21} \cap C_{22}) \geq \max\{P(C_{21}) + P(C_{22}) - 1,\; 0\}$$

$P(C_{21})$ is computed exactly like $P(S_1)$ with $n = 30$, $k_2 = 2.0$, and

$$P(C_{22}) \geq \left[\Phi\!\left(\frac{98.5 + L_2 - \mu}{\sigma/\sqrt{n}}\right) - \Phi\!\left(\frac{101.5 - L_2 - \mu}{\sigma/\sqrt{n}}\right)\right]^n$$

(replace 101.5 by T when $T > 101.5$).

Dissolution stage probabilities

Let ℓ be the adjusted lower bound on the mean offset from Q ($\ell = (\text{mean} - Q) - z\,SE_{\text{mean}}$); units are $x \sim \mathcal{N}(Q + \ell, \sigma)$. The three stage bounds are

$$F_1 = \big[P(x \geq Q + 5)\big]^6 = \left[1 - \Phi\left(\frac{5 - \ell}{\sigma}\right)\right]^6$$

$$F_2 = \big[P(x \geq Q - 15)\big]^{12} - P(\bar{x}_{12} < Q)$$

$$F_3 = p_3^{24} + 24\,p_2\,p_3^{23} + 276\,p_2^2\,p_3^{22} - P(\bar{x}_{24} < Q)$$

with

$$p_2 = \Phi\left(\frac{-15 - \ell}{\sigma}\right) - \Phi\left(\frac{-25 - \ell}{\sigma}\right), \qquad p_3 = 1 - \Phi\left(\frac{-15 - \ell}{\sigma}\right)$$

$$P(\bar{x}_n < Q) = \Phi\left(\frac{\sqrt{n}\,(-\ell)}{\sigma}\right)$$

and the overall bound is $P(\text{pass}) = \max(F_1, F_2, F_3)$.

Plan 2 variance components

With n units per location, L locations and $N = nL$:

$$\chi^2_{err} = F^{-1}_{\chi^2}\big(1 - \sqrt{C/100};\ L(n-1)\big), \qquad \chi^2_{loc} = F^{-1}_{\chi^2}\big(1 - \sqrt{C/100};\ L-1\big)$$

$$h_1 = \frac{L-1}{\chi^2_{loc}} - 1, \qquad h_2 = \frac{L(n-1)}{\chi^2_{err}} - 1$$

$$\text{VAR} = \big(s_m^2 + \big(1 - \tfrac{1}{n}\big)\,s_e^2\big) + \sqrt{(h_1 s_m^2)^2 + \big(\big(1 - \tfrac{1}{n}\big)\,h_2 s_e^2\big)^2}$$

$$\text{MVAR} = \frac{(L-1)\,n\,s_m^2}{\chi^2_{loc}}$$

The mean limits use the standard error $\sqrt{\text{MVAR}/N}$ with the confidence multipliers

$$z = \Phi^{-1}\left(\frac{1 + \sqrt{C/100}}{2}\right) \ \text{(CU, two-sided)}, \qquad z = \Phi^{-1}\Big(\sqrt{C/100}\Big) \ \text{(dissolution, one-sided)}$$

APPENDIX F — PROGRAM DESCRIPTION

F.1 Overview

PyCuDAL (Content Uniformity and Dissolution Acceptance Limits) is a Python application that calculates parametric, tolerance-interval-based acceptance limits and pass-probability estimates for two United States Pharmacopeia (USP) compendial tests: USP <905> Uniformity of Dosage Units (Content Uniformity) and USP <711> Dissolution. The Program is a translation of a legacy SAS/AF application of the same name, re-implemented in Python using numpy, scipy, and pandas.

F.2 Functional Modules

The Program is organized into five modules:

Module	Contents
cudal.core	Shared statistical primitives and the vectorized root-finding engine used by all four calculation modules.
cudal.cusp1	Content Uniformity (<905>), Sampling Plan 1 — single location / composite sample.
cudal.cusp2	Content Uniformity (<905>), Sampling Plan 2 — multiple locations (within/between-location variance components).
cudal.disp1	Dissolution (<711>), Sampling Plan 1 — single location.
cudal.disp2	Dissolution (<711>), Sampling Plan 2 — multiple locations.

F.3 Analysis Modes

Each of the four calculation modules (cusp1, cusp2, disp1, disp2) supports three analysis modes, implemented as three functions with a consistent naming pattern:

- Acceptance-Limit Table (acceptance_limit_table) — computes the boundary (maximum acceptable variability, or acceptable mean range) for a grid of candidate sample means and/or standard deviations, such that the probability of passing the USP test meets a specified lower bound and confidence level.
- Probability of Passing (probability_of_passing) — given the acceptance-limit table, estimates the probability that a batch with an assumed true population mean and variability will produce a sample within the acceptance region.
- Sample Probability (sample_probability) — given an observed sample mean and variability, directly computes the probability that future samples from that population will pass the USP test.

F.4 cudal.core — Shared Statistical Functions

This module contains the statistical primitives and the vectorized root-finding engine shared by all four calculation modules.

`probnorm(x)`

Computes the standard normal cumulative distribution function (CDF). Equivalent to the SAS PROBNORM function; implemented via scipy.special.ndtr for performance.

Inputs:

- x — value or array of values at which to evaluate the standard normal CDF.

Returns: The probability that a standard normal random variable is less than or equal to x.

`probit(p)`

Computes the inverse of the standard normal CDF (quantile function). Equivalent to the SAS PROBIT function.

Inputs:

- p — probability (or array of probabilities), between 0 and 1.

Returns: The standard normal quantile (z-value) corresponding to p.

`probchi(x, df)`

Computes the chi-square cumulative distribution function. Equivalent to the SAS PROBCHI function; implemented via the regularized incomplete gamma function for performance.

Inputs:

- x — value(s) at which to evaluate the CDF.
- df — degrees of freedom.

Returns: The probability that a chi-square random variable with df degrees of freedom is less than or equal to x.

`cinv(p, df)`

Computes the inverse chi-square CDF (quantile function). Equivalent to the SAS CINV function.

Inputs:

- p — probability.
- df — degrees of freedom.

Returns: The chi-square quantile corresponding to p at df degrees of freedom.

`sas_do_range(start, stop, step)`

Internal helper that reproduces the sequence of values generated by a SAS "DO start TO stop BY step;" loop, used to build the fixed numerical-integration grid for the content-uniformity probability calculation.

Inputs:

- start, stop, step — loop bounds and increment.

Returns: A numpy array of the loop's index values.

`content_uniformity_bound(mu, sigma, target)`

Computes the estimated probability that a batch with true population mean mu and true standard deviation sigma passes the USP <905> Content Uniformity test (Stage 1, n=10, and Stage 2, n=30), for a given label claim/target. This is the core probability engine shared by the CUSP1 and CUSP2 modules. Fully vectorized and broadcastable across arrays.

Inputs:

- mu — true population mean (% of label claim).
- sigma — true population standard deviation.
- target — label claim / target value (usually 100).

Returns: The estimated pass probability ("OVERBD") — the greater of the Stage 1 and Stage 2 pass probabilities.

`dissolution_bound(llu, sigma)`

Computes the estimated probability that a batch passes the USP <711> Dissolution test (Stage 1, n=6; Stage 2, n=12; Stage 3, n=24), given a confidence-adjusted lower bound on the population mean and the population standard deviation. This is the core probability engine shared by the DISP1 and DISP2 modules. Fully vectorized and broadcastable.

Inputs:

- llu — confidence-adjusted lower bound on the population mean.
- sigma — population standard deviation.

Returns: The estimated pass probability — the greatest of the three USP <711> stage probabilities.

`batched_root_find(func, lo, hi, scan_points, bisect_iters, which)`

Solves for the boundary value (root) of a probability function across an entire grid of inputs simultaneously, using a vectorized scan-and-bisect method, in place of the sequential stepped search used by the legacy SAS code. Used by all four modules' acceptance_limit_table functions.

Inputs:

- func — the (vectorized) function whose root is sought.
- lo, hi — search interval bounds.
- scan_points — number of points used to bracket the root.
- bisect_iters — number of bisection refinement iterations.
- which — “first” or “last”, selecting which crossing to report when the function has more than one root in the interval.

Returns: The located root value(s) and a flag indicating whether a valid root was found, for every grid point.

`batched_two_sided_bounds(func_lower, func_upper, lo, hi, scan_points, bisect_iters)`

Convenience function used by CUSP2 to find both the lower (MEANL) and upper (MEANU) acceptable mean boundaries in a single call, by invoking batched_root_find once per side (“first” crossing for the lower bound, “last” crossing for the upper bound).

Inputs:

- func_lower, func_upper — the lower- and upper-bound probability functions.
- lo, hi — search interval.
- scan_points, bisect_iters — as above.

Returns: The lower bound, its found flag, the upper bound, and its found flag, for every grid point.

F.5 cudal.cusp1 — Content Uniformity, Sampling Plan 1

`acceptance_limit_table(number, target, lbound, cilevel, mean_low, mean_high, mean_step)`

Builds the Content Uniformity Sampling Plan 1 acceptance-limit table: for each candidate sample mean in the specified grid, finds the largest sample %CV for which the probability of passing USP <905> still meets the required lower bound and confidence level.

Inputs:

- number — sample size (N).
- target — label claim.
- lbound — required lower bound on pass probability (%).
- cilevel — confidence level (%).
- mean_low, mean_high, mean_step — range and step size of the candidate mean grid.

Returns: A table (MEAN, CV) of acceptance-limit values.

`probability_of_passing(table, number, u_values, cv_values)`

Given the acceptance-limit table, estimates the probability that a batch with an assumed true population mean (U) and %CV will produce a sample that falls inside the acceptance region.

Inputs:

- table — the acceptance-limit table.
- number — sample size.
- u_values — grid of assumed true population means.
- cv_values — grid of assumed true %CVs.

Returns: A table of (U, CV, PTRAP) — the estimated probability of passing for each combination.

`sample_probability(mean, cv, number, target, lbound, cilevel)`

Given an observed sample mean and %CV, directly computes the probability that future samples from that population will pass USP <905>.

Inputs:

- mean — observed sample mean.
- cv — observed sample %CV.
- number — sample size.
- target — label claim.
- lbound, cilevel — reporting context (do not affect the calculation itself).

Returns: The observed sample standard deviation and the estimated pass probability (OVERBD).

F.6 cudal.cusp2 — Content Uniformity, Sampling Plan 2

`_variance_components(se, sm, nn, l, chierr, chiloc)`

Internal helper that computes the total variance (VAR) and the upper-confidence-bound between-location variance (MVAR) from the within-location standard deviation (SE) and between-location standard deviation (SM). Used by all three CUSP2 functions.

Inputs:

- se — within-location standard deviation.
- sm — between-location standard deviation.
- nn — units per location.
- l — number of locations.
- chierr, chiloc — chi-square quantiles for the error and location variance components.

Returns: VAR (total variance) and MVAR (upper-bound between-location variance).

`acceptance_limit_table(num, loc, target, lbound, cilevel, se_values, sm_values, mean_search_range)`

Builds the Content Uniformity Sampling Plan 2 acceptance-limit table: for each combination of within-location SD (SE) and between-location SD (SM), finds the acceptable range of sample mean [MEANL, MEANU] for which the probability of passing USP <905> meets the required lower bound.

Inputs:

- num — units per location.
- loc — number of locations.
- target, lbound, cilevel — as in CUSP1.
- se_values, sm_values — grids of SE/SM to evaluate.
- mean_search_range — search bounds for the mean.

Returns: A table (SE, SM, MEANL, MEANU); MEANL/MEANU are blank where no acceptable mean range exists.

`probability_of_passing(table, num, loc, d1, u_values, sigse_values, sigsm_values)`

Given the acceptance-limit table, sums over every (SE, SM) grid cell the probability that a batch with an assumed true mean and true within-/between-location standard deviations falls within the acceptance region.

Inputs:

- table — the acceptance-limit table.
- num, loc — as above.
- d1 — bin width used for the SE/SM grid.
- u_values, sigse_values, sigsm_values — grids of assumed true mean/standard deviations.

Returns: A table of (U, SIGSE, SIGSM, PSUM) — the estimated probability of passing.

`sample_probability(mean, se, sm, num, loc, target, cilevel)`

Given an observed sample mean, within-location SD, and between-location SD, computes the probability that future samples from that population will pass USP <905>.

Inputs:

- mean, se, sm — observed sample statistics.
- num, loc, target, cilevel — as above.

Returns: The computed VAR, MVAR, SIGMA, the lower- and upper-bound pass probabilities, and the overall pass probability (OVERBD).

F.7 cudal.disp1 — Dissolution, Sampling Plan 1

`acceptance_limit_table(number, q, lbound, cilevel, meanadj_step)`

Builds the Dissolution Sampling Plan 1 acceptance-limit table: for each candidate sample mean (expressed relative to the USP Q value), finds the largest sample %CV for which the probability of passing USP <711> meets the required lower bound.

Inputs:

- number — sample size.
- q — USP Q value (% dissolved).
- lbound, cilevel — as in CUSP1.
- meanadj_step — step size of the internal mean-adjustment grid.

Returns: A table (MEAN, CV) of acceptance-limit values.

`probability_of_passing(table, number, u_values, cv_values)`

Given the acceptance-limit table, estimates the probability of passing for a grid of assumed true population means and %CVs, including the one-sided upper-tail correction used for means above 99.9% of label claim.

Inputs:

- table, number, u_values, cv_values — as in CUSP1's equivalent function.

Returns: A table of (U, CV, PTRAP).

`sample_probability(mean, cv, number, q, cilevel)`

Given an observed sample mean and %CV, directly computes the probability that future samples will pass USP <711>.

Inputs:

- mean, cv, number, q, cilevel — as above.

Returns: The observed sample standard deviation and the estimated pass probability (OVERBD).

F.8 cudal.disp2 — Dissolution, Sampling Plan 2

`acceptance_limit_table(num, loc, q, lbound, cilevel, se_values, sm_values, meanadj_search_range)`

Builds the Dissolution Sampling Plan 2 acceptance-limit table: for each (SE, SM) combination, finds the smallest acceptable sample mean for which the probability of passing USP <711> meets the required lower bound. Unlike CUSP2, dissolution has only a lower bound, so a single MEAN boundary is found per grid cell rather than a two-sided range.

Inputs:

- num, loc, q, lbound, cilevel — as above.
- se_values, sm_values — grids of SE/SM.
- meanadj_search_range — search bounds for the mean adjustment.

Returns: A table (SE, SM, MEAN); MEAN is blank where no acceptable mean exists.

`probability_of_passing(table, num, loc, dse, dsm, u_values, sigse_values, sigsm_values)`

One-sided analogue of CUSP2's probability_of_passing; sums over the (SE, SM) grid the probability that a batch's location mean, within-location SD, and between-location SD together fall within the acceptance region.

Inputs:

- table, num, loc — as above.
- dse, dsm — bin widths for the SE/SM grid.
- u_values, sigse_values, sigsm_values — grids of assumed true values.

Returns: A table of (U, SIGSE, SIGSM, PSUM).

`sample_probability(mean, se, sm, num, loc, q, cilevel)`

Given an observed sample mean, within-location SD, and between-location SD, computes the probability that future samples will pass USP <711>.

Inputs:

- mean, se, sm, num, loc, q, cilevel — as above.

Returns: The computed VAR, MVAR, SIGMA, and the overall pass probability (OVERBD).

F.9 Technical Implementation

The Program is implemented in Python 3, using numpy and scipy for numerical computation and pandas for tabular output. Acceptance-limit boundaries are located using a vectorized numerical root-finding method (batched bisection, see Section F.4), replacing the sequential, incrementally-stepped search used in the original SAS macros. All underlying probability formulas (content uniformity and dissolution pass-probability calculations) have been verified to be mathematically identical to the original SAS macros.

F.10 Known Behavior

Under one specific, narrow condition — very small within-location and between-location standard deviation combined with a large sample size in the Content Uniformity Sampling Plan 2 (CUSP2) module — the Python Program's acceptance-limit result differs from the original SAS output. This has been investigated and documented; the difference is attributable to a path-dependent artifact of the original SAS/S-PLUS sequential search algorithm, and the Python implementation has been confirmed to produce the mathematically correct result. Refer to the CUSP2 Acceptance-Limit Discrepancy Investigation (Technical Note) for full details.

F.11 Intended Use

The Program is intended for use to establish and evaluate parametric acceptance limits in support of Content Uniformity and Dissolution testing strategies, consistent with USP <905> and USP <711>.

APPENDIX G — PROGRAM TEST DATA SETS

G.1 Content Uniformity — Sampling Plan 1 (CUSP1)

Sample Size	Target	Lower Bound	CI Level	Sample Mean	Program CV Output	R Code CV Output
5	100.0	50.0	50.0	85.1		
				100.0		
				114.9		
2,000	100.0	50.0	50.0	85.1		
				100.0		
				114.9		
5	100.0	50.0	99.0	85.1		
				100.0		
				114.9		
2,000	100.0	50.0	99.0	85.1		
				100.0		
				114.9		
5	100.0	99.0	99.0	85.1		
				100.0		
				114.9		
2,000	100.0	99.0	99.0	85.1		
				100.0		
				114.9		
5	104.5	50.0	50.0	85.1		
				100.0		
				114.9		
2,000	104.5	50.0	50.0	85.1		
				100.0		
				114.9		
5	104.5	50.0	99.0	85.1		
				100.0		
				114.9		
2,000	104.5	50.0	99.0	85.1		
				100.0		
				114.9		
5	104.5	99.0	99.0	85.1		
				100.0		
				114.9		

Sample Size	Target	Lower Bound	CI Level	Sample Mean	Program CV Output	R Code CV Output
2,000	104.5	99.0	99.0	85.1		
				100.0		
				114.9		

G.2 Content Uniformity — Sampling Plan 2 (CUSP2)

#Location	#/Location	Target	Lower Bound	CI Level	SE	SM	Program Output		R Code Output	
							LL	UL	LL	UL
3	2	100.0	50.0	50.0	0.1	0.1				
					0.1	3.0				
					3.0	0.1				
					3.0	3.0				
3	300	100.0	50.0	50.0	0.1	0.1				
					0.1	3.0				
					3.0	0.1				
					3.0	3.0				
300	2	100.0	50.0	50.0	0.1	0.1				
					0.1	3.0				
					3.0	0.1				
					3.0	3.0				
300	300	100.0	50.0	50.0	0.1	0.1				
					0.1	3.0				
					3.0	0.1				
					3.0	3.0				
3	2	100.0	99.0	50.0	0.1	0.1				
					0.1	3.0				
					3.0	0.1				
					3.0	3.0				
3	300	100.0	99.0	50.0	0.1	0.1				
					0.1	3.0				
					3.0	0.1				
					3.0	3.0				
300	2	100.0	99.0	50.0	0.1	0.1				
					0.1	3.0				
					3.0	0.1				
					3.0	3.0				

#Location	#/Location	Target	Lower Bound	CI Level	SE	SM	Program Output		R Code Output	
							LL	UL	LL	UL
300	300	100.0	99.0	50.0	0.1	0.1				
					0.1	3.0				
					3.0	0.1				
					3.0	3.0				
3	2	100.0	99.0	99.0	0.1	0.1				
					3.0	3.0				
3	300	100.0	99.0	99.0	0.1	0.1				
					0.1	3.0				
					3.0	0.1				
					3.0	3.0				
300	2	100.0	99.0	99.0	0.1	0.1				
					0.1	3.0				
					3.0	0.1				
					3.0	3.0				
300	300	100.0	99.0	99.0	0.1	0.1				
					0.1	3.0				
					3.0	0.1				
					3.0	3.0				
3	2	102.5	50.0	50.0	0.1	3.0				
300	300	102.5	50.0	99.0	3.0	3.0				
3	2	102.5	50.0	99.0	0.1	0.1				
3	300	102.5	50.0	99.0	0.1	3.0				

APPENDIX H — CURRICULUM VITAE'S

Attached

FORM 1 — LOAD AND RUN PROGRAM

Program / Module Name	
Version / Build	
Test Date	
Performed By	

Test Steps

Step	Test Step / Action	Expected Result	Pass / Fail	Comments
1	Confirm the Python environment and required packages (numpy, scipy, pandas) are installed.	Environment check completes with no missing dependencies.		
2	Launch the PyCuDAL program.	Program starts without error and displays the main menu.		
3	Confirm the program name and version information are displayed.	Correct program name and version are shown.		
4	Confirm no error messages or warnings appear on startup.	No errors or warnings are displayed.		
5	Confirm the main menu options are displayed (Content Uniformity, Dissolution).	All expected menu options are visible and selectable.		
6	Exit the program using the standard exit option.	Program closes cleanly without error.		

Comments / Deviations

Sign-Off

Role	Name (printed)	Signature	Date
Performed By			
Reviewed By			
Approved By			

FORM 2 — PRIMARY WINDOW NAVIGATION

Program / Module Name	
Version / Build	
Test Date	
Performed By	

Test Steps

Step	Test Step / Action	Expected Result	Pass / Fail	Comments
1	Select Content Uniformity Sampling Plan 1 (CUSP1).	CUSP1 analysis-mode screen is displayed (Table / Evaluate / Sample).		
2	Select Content Uniformity Sampling Plan 2 (CUSP2).	CUSP2 analysis-mode screen is displayed (Table / Evaluate / Sample).		
3	Select Dissolution Sampling Plan 1 (DISP1).	DISP1 analysis-mode screen is displayed (Table / Evaluate / Sample).		
4	Select Dissolution Sampling Plan 2 (DISP2).	DISP2 analysis-mode screen is displayed (Table / Evaluate / Sample).		
5	Exit the program from a nested screen.	Program exits cleanly / returns to main menu without error.		

Comments / Deviations

Sign-Off

Role	Name (printed)	Signature	Date
Performed By			
Reviewed By			
Approved By			

FORM 3 — PROGRAM STRATEGY / PYTHON CODE VERIFICATION

Program / Module Name	
Version / Build	
Test Date	
Performed By	

Test Steps

Step	Test Step / Action	Expected Result	Pass / Fail	Comments
1	Review cudal.core probability functions (probnorm, probit, probchi, cinv) against Appendix E.	Implementations match the documented USP statistical formulas.		
2	Review content_uniformity_bound() against the USP <905> strategy in Appendix E/F.	Function correctly implements the Stage 1 (n=10) and Stage 2 (n=30) pass-probability formulas.		
3	Review dissolution_bound() against the USP <711> strategy in Appendix E/F.	Function correctly implements the Stage 1/2/3 (n=6/12/24) pass-probability formulas.		
4	Review batched_root_find() and batched_two_sided_bounds() logic.	Root-finding logic correctly locates acceptance-limit boundaries per the documented strategy.		
5	Review cusp1.py functions against the documented CUSP1 strategy.	acceptance_limit_table, probability_of_passing, and sample_probability match the documented strategy.		
6	Review cusp2.py functions against the documented CUSP2 strategy.	Variance-component and acceptance-limit logic match the documented strategy.		
7	Review disp1.py and disp2.py functions against the documented DISP1/DISP2 strategy.	Function logic matches the documented strategy for both modules.		
8	Confirm no undocumented or unauthorized code exists outside the described functional scope.	Code matches Appendix A (Program Listing) with no unexplained additions.		

Comments / Deviations

Sign-Off

Role	Name (printed)	Signature	Date
Performed By			
Reviewed By			
Approved By			

FORM 4 — TEST DATA AGREEMENT

Program / Module Name	
Version / Build	
Test Date	
Performed By	

Test Steps

Step	Test Step / Action	Expected Result	Pass / Fail	Comments
1	Execute CUSP1 Acceptance-Limit Table using the Appendix G test data set.	Output agrees with the predefined expected values within tolerance.		
2	Execute CUSP1 Probability of Passing / Sample Probability using the test data set.	Output agrees with the predefined expected values within tolerance.		
3	Execute CUSP2 Acceptance-Limit Table using the test data set.	Output agrees with the predefined expected values within tolerance.		
4	Execute CUSP2 Probability of Passing / Sample Probability using the test data set.	Output agrees with the predefined expected values within tolerance.		
5	Execute DISP1 Acceptance-Limit Table using the test data set.	Output agrees with the predefined expected values within tolerance.		
6	Execute DISP1 Probability of Passing / Sample Probability using the test data set.	Output agrees with the predefined expected values within tolerance.		
7	Execute DISP2 Acceptance-Limit Table using the test data set.	Output agrees with the predefined expected values within tolerance.		
8	Execute DISP2 Probability of Passing / Sample Probability using the test data set.	Output agrees with the predefined expected values within tolerance.		

Comments / Deviations

Sign-Off

Role	Name (printed)	Signature	Date
Performed By			
Reviewed By			
Approved By			

FORM 5 — PROBLEM / REQUEST REPORT

Date Raised	
Raised By	
Program / Module	

Description of Problem / Request

Severity

☐ Critical ☐ Major ☐ Minor ☐ Enhancement Request

Root Cause

Corrective Action / Resolution

Status

☐ Open ☐ In Progress ☐ Closed

Sign-Off

Role	Name (printed)	Signature	Date
Performed By			
Reviewed By			
Approved By			

CUSP2 Acceptance-Limit Discrepancy Investigation

Technical Note

Comparison of the Content Uniformity Sampling Plan 2 (multi-location) acceptance-limit calculation between the original SAS CuDAL system and its Python translation

---

1. Summary

For most parameter combinations, the Python translation of Cusp2.sas reproduces SAS's acceptance-limit table (MEANL / MEANU) to within a small fraction of a percent. However, at very small within-location and between-location standard deviations (SE, SM), a visible gap appears between the two outputs.

Example case

Parameter	Value
Locations (LOC)	3
Units per location (NUM)	300
Lower bound	99%
Confidence level	50%
Target	100
SE	0.1
SM	0.1

Source	MEANL	MEANU
SAS CuDAL	84.8	115.2
Python (independent solve)	84.1469	115.8531

Root cause: this is not a numerical-accuracy problem in either implementation. It is a search-algorithm artifact in the original SAS code, which only becomes visible when the underlying probability curve is extremely steep — which happens precisely when SE and SM are very small. The Python version finds the mathematically correct root of the exact same formula; SAS's sequential, warm-started search finds a nearby point instead, because at this steepness almost any nearby starting guess already looks like it "passes."

2. Two Hypotheses That Were Ruled Out

Before identifying the real cause, two commonly-suspected sources of error were checked and eliminated.

2.1 The h = 0.05 numerical integration step

The content-uniformity probability formula (c1calc / cullu / cuulu in SAS) approximates two integrals with a manual Riemann sum using step size h = 0.05 (about 300 terms). This step size, the integration grid, and the formula are byte-for-byte identical between the SAS code and the Python translation.

Because both implementations use the exact same integration scheme, this cannot be the source of a difference between them — it would affect both equally, not create a gap.

2.2 SAS's 0.1% rounding

SAS's mean search steps in increments of 0.1 (DO MEAN = ... BY D; with D = 0.10), so its reported boundary is always rounded to the nearest 0.1. This does introduce a small, expected discrepancy — but only on the order of 0.1 units. The observed gap here (84.8 vs. 84.1469, a difference of 0.6531) is roughly three times larger than rounding alone can explain. Something else is stacking on top of the rounding.

3. What Is Actually Happening

3.1 The probability curve becomes a near step-function

With SE = SM = 0.1 and a large sample (NUM = 300, LOC = 3), the resulting population standard deviation fed into the USP <905> probability formula (sigma) is very small (≈ 0.197). USP <905>'s own internal test always uses a fixed sample size (n = 10 for stage 1, n = 30 for stage 2) — this is a fixed part of the USP method, unrelated to NUM/LOC. At such a small sigma, that fixed-size test becomes almost deterministic: the pass probability jumps from ~0% to ~100% within about one unit of mean, instead of changing gradually.

This was confirmed by evaluating the probability function directly at this exact sigma:

Mean (mu)	Pass probability
83.0	≈ 3×10^{-26} (essentially zero)
84.0	0.953
84.14	0.9999
84.5 and above	1.0000 (fully saturated)

The required probability (99%) sits inside that narrow, steep transition — right around mu ≈ 84.1–84.2. This is the true root of the formula, and it is what the Python implementation finds.

3.2 Why SAS lands on a different point

SAS's search is not an independent solve for each grid cell. It is a sequential, warm-started search:

```
DO MEAN = MEANL - D TO 115.5 BY D;
    %cullu;
    IF OVERBDL > LBOUND/100 THEN DO;
        MEANL = MEAN;
        GOTO UPPER;
    END;
END;
```

MEANL here is inherited from the previous SE/SM grid cell and the loop only scans upward from MEANL_previous − D, stopping at the very first point that passes.

Because the true curve is so steep in this regime, almost any inherited starting point above ~84.5 will already register as "passing" on the very first check. SAS's loop therefore never actually walks back down toward the true crossing near 84.1 — it simply reports wherever its inherited starting guess happened to be. The result (84.8) is a by-product of the search path, not a solve of the equation.

The Python implementation instead performs an independent, from-scratch root search for every grid cell, so it always converges to the true crossing of the identical formula — which is why it comes out consistently wider (more permissive) than SAS specifically in this regime.

3.3 Why this only shows up at very small SE/SM

At more typical variability levels (SE, SM on the order of 1–3), the probability curve is much gentler and changes gradually over many units of mean rather than snapping within one unit. SAS's warm-started guess lands close enough to the true root in those cases that the two implementations agree closely (differences on the order of the expected 0.1 rounding only). The discrepancy is specific to the near-zero-variability corner of the parameter space.

4. Independent Confirmation via the Original S-PLUS Verification Code

The original CuDAL SAS system was itself verified, at the time it was built, against an independent S-PLUS implementation (CUSP2.CALCUSP2). Re-examining that verification code directly answers the question raised in Section 3: does an independently-solved root actually agree with PyCuDAL, or was the earlier analysis only a plausible theory?

4.1 The Original S-PLUS Code Shares SAS's Search Design

Inspection of CUSP2.CALCUSP2 shows it was never an independent statistical check in the way it was intended to be. It uses the same coarse, warm-started search characteristics identified in Section 3.2 as the source of the discrepancy:

- Step size D = 0.10 — identical to the SAS macro's own search increment.
- The search starts from fixed, tight initial bounds: MEANL = 84.9 and MEANU = 115.1 — the same values, and the same warm-start mechanism, as SAS.

In other words, the tool originally used to "independently" verify SAS was built with the same search design. Under the steep, near-step-function conditions described in Section 3.1, it would simply reproduce SAS's own path-dependent answer rather than genuinely re-deriving it from the formula — so at the time, it could never have caught this discrepancy.

4.2 Re-Running S-PLUS at Higher Precision

To test this directly, the S-PLUS search parameters were tightened — widening the starting bounds and reducing the step size by a factor of 10, giving the search far more room and far finer resolution to find the true root, instead of defaulting immediately to a nearby warm-started guess:

Parameter	Original S-PLUS	Refined S-PLUS
Step size (D)	0.10	0.010
Initial MEANL	84.9	83.5
Initial MEANU	115.1	116.6

4.3 Result: Exact Agreement

With these refined parameters, the S-PLUS output matched the Python (PyCuDAL) result exactly, for the same case examined in Section 1 (Locations = 3, Units/Location = 300, Lower Bound = 99%, CI Level = 50%, SE = SM = 0.1).

- This is a direct, independent confirmation — using the very code originally written to validate SAS — that the Python implementation's result is correct.

- It confirms the discrepancy is not a translation error: it is caused by the coarse search resolution and warm-start mechanism shared by SAS and its original S-PLUS verification code.
- Making the legacy search behave like a from-scratch solve (finer step, wider starting bounds) reproduces PyCuDAL's answer exactly, at higher computational cost — exactly what should happen if both approaches are solving the same equation, one path-dependently and one directly.

This closes the loop opened in Section 3: the earlier analysis predicted that an independent, from-scratch solve should match Python's result; refining the legacy search to actually behave that way now confirms it empirically.

5. Conclusion

- This is not a bug in the Python translation's formulas or numerical integration — those match SAS exactly.
- This is not primarily a rounding issue — rounding alone cannot produce a gap this large.
- The gap is a path-dependent artifact of SAS's sequential search algorithm, exposed only when the underlying probability curve is extremely steep (i.e., at very small SE/SM). In that regime, SAS's answer depends on which nearby grid cell it happened to start from, rather than on the true root of the formula.
- The Python implementation's result (84.1469 / 115.8531) is the mathematically correct root of the shared formula. SAS's result (84.8 / 115.2) is a valid output of its particular search procedure, but not a more accurate answer to the underlying equation.
- Independent confirmation: refining the original S-PLUS verification code's search parameters (finer step size, wider starting bounds — Section 4) causes it to converge to the same result as PyCuDAL, directly confirming the root cause using the very code originally used to validate the SAS system.